Design & Analysis of Algorithms

Complete MCQ Theory Exam Guide

Topic	Questions
Divide & Conquer	49
Dynamic Programming	43
Floyd-Warshall (All-Pairs Shortest Paths)	23
TOTAL	**115**

Each topic is presented in two parts:
Part 1 – Practice Questions (attempt these first, no answers shown)
Part 2 – Answer Key & Detailed Explanations (with reasoning for every option)

Based on core CLRS-style theory: Divide & Conquer, Dynamic Programming, and the Floyd-Warshall All-Pairs Shortest Path algorithm.

Table of Contents

Each section's Answer Key immediately follows that section's practice questions.

Divide & Conquer — Practice Questions

Q1. Consider $T(n) = 2T(n/2) + n \log n$. Using the extended Master theorem (Case 2 with a polylog factor), what is $T(n)$?

A. $\Theta(n \log n)$

B. $\Theta(n \log^2 n)$

C. $\Theta(n^2)$

D. $\Theta(n \log n \log \log n)$

Q2. What is the tightest asymptotic bound for $T(n) = 3T(n/4) + n \log n$, by the Master theorem?

A. $\Theta(n \log n)$

B. $\Theta(n^{\log_4 3})$

C. $\Theta(n^{\log_4 3} \log n)$

D. $\Theta(n^2)$

Q3. In Karatsuba's algorithm for multiplying two n-digit numbers, three recursive multiplications of (n/2)-digit numbers are combined with O(n) extra work. What recurrence and complexity result?

A. $T(n) = 3T(n/2) + O(n)$, $\Theta(n^{1.585})$

B. $T(n) = 4T(n/2) + O(n)$, $\Theta(n^2)$

C. $T(n) = 3T(n/2) + O(n^2)$, $\Theta(n^2)$

D. $T(n) = 2T(n/2) + O(n)$, $\Theta(n \log n)$

Q4. Strassen's algorithm multiplies two n×n matrices using 7 multiplications of (n/2)×(n/2) submatrices plus $O(n^2)$ additions. What is its time complexity, and why does it beat the naive 8-multiplication approach?

A. $\Theta(n^3)$, because 8 recursive calls also give n^3 so there is no real gain

B. $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$, strictly better than $\Theta(n^3)$ for large n

C. $\Theta(n^2 \log n)$, because the additions dominate asymptotically

D. $\Theta(n^{2.5})$, the geometric mean of n^2 and n^3

Q5. What does the recurrence $T(n) = 2T(n/2) + \Theta(1)$ describe, and what is its solution?

A. A balanced divide-and-conquer with constant combine time (e.g., counting nodes in a balanced binary tree); $T(n) = \Theta(n)$

B. Binary search; $T(n) = \Theta(\log n)$

C. Merge sort; $T(n) = \Theta(n \log n)$

D. This recurrence is invalid because combine work must be at least linear

Q6. For T(n) = T(n/3) + T(2n/3) + O(n) analyzed via a recursion tree, what is the depth of the longest root-to-leaf path, and why does the O(n log n) bound still hold despite the tree being unbalanced?

A. Depth is $\log_{3/2} n$ (from repeatedly taking the 2n/3 branch); the bound holds because every level of the tree still sums to exactly cn in total work

B. Depth is $\log_3 n$ (from the n/3 branch only); the bound is therefore not tight

C. Depth is constant; the recurrence solves to O(n)

D. Depth is n; the recurrence solves to $O(n^2)$

Q7. Which is the correct description of the substitution method's inductive step for proving T(n) ≤ cn log n, where T(n) = 2T(⌊n/2⌋) + n?

A. Assume T(k) ≤ ck log k for all k < n, substitute k = ⌊n/2⌋, then algebraically show the resulting expression is ≤ cn log n for suitable c > 0, $n \geq n_0$

B. Assume T(n) ≤ cn log n for the very n being proved, then it trivially holds

C. Prove only the base case T(1) = Θ(1) and conclude the guess holds for all n

D. Substitute the guess into the recurrence and solve for n directly using algebra, without induction

Q8. For T(n) = T(⌊n/2⌋) + T(⌈n/2⌉) + 1, a naive guess T(n) ≤ cn fails the substitution-method induction. What is the standard fix, and what is the true asymptotic bound?

A. Strengthen the guess to T(n) ≤ cn − b for constant b > 0 to absorb the slack from the +1 term; the true bound remains Θ(n)

B. Switch to an $O(n^2)$ guess since O(n) must be structurally wrong

C. Add a multiplicative log factor, guessing T(n) ≤ cn log n instead

D. Increase c without bound until the inequality eventually holds

Q9. Given T(n) = 4T(n/2) + n, what is Θ(T(n)) and which Master theorem case applies?

A. Case 1 (recursive term dominates): $\Theta(n^2)$

B. Case 2 (balanced): $\Theta(n^2 \log n)$

C. Case 3 (driving function dominates): Θ(n)

D. Case 1, but the correct value is Θ(n log n)

Q10. Which recurrence CANNOT be solved directly by the basic (non-extended) Master theorem as stated in CLRS, and why?

A. T(n) = 2T(n/2) + n/log n, because f(n) is asymptotically smaller than n but not by a polynomial factor

B. T(n) = 3T(n/3) + n, because a = b

C. T(n) = T(n/2) + 1, because a = 1

D. $T(n) = 2T(n/2) + n^2$, because f(n) dominates polynomially

Q11. In merge sort's MERGE procedure using sentinel values (∞) at the ends of each subarray, what goes wrong if the sentinels are omitted without adding explicit bounds-checking logic?

A. The merge loop may read past the end of an exhausted subarray, since comparisons continue against a nonexistent element once one side runs out

B. The algorithm still works correctly but degrades to $\Theta(n^2)$ time

C. The algorithm silently produces a correctly sorted array with no side effects

D. The recursion never terminates

Q12. In the divide-and-conquer maximum-subarray algorithm, the crucial extra step is finding the max crossing subarray. What is its time complexity for a subarray of size n, and why?

A. $\Theta(n)$, via two separate linear scans outward from the midpoint (one leftward, one rightward)

B. $\Theta(\log n)$, via binary search on both halves

C. $\Theta(n \log n)$, because it recursively finds crossing subarrays too

D. $\Theta(1)$, because the midpoint value alone determines the crossing max

Q13. What is the overall recurrence and complexity of the divide-and-conquer maximum-subarray algorithm, and how does it compare to Kadane's algorithm?

A. $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$; Kadane's $\Theta(n)$ is asymptotically better

B. $T(n) = 2T(n/2) + \Theta(1) = \Theta(n)$; identical to Kadane's approach

C. $T(n) = T(n/2) + \Theta(n) = \Theta(n)$; identical to Kadane's approach

D. $T(n) = 2T(n/2) + \Theta(n^2) = \Theta(n^2)$; worse than brute force

Q14. In the closest-pair-of-points D&C algorithm, why does the 'strip' merge step examine only a constant number of candidate points per point, rather than all points in the strip?

A. A geometric packing argument shows that within the y-sorted strip, only a small constant number of subsequent points can possibly be closer than δ to any given point, since the recursive δ already bounds minimum spacing in each half

B. The algorithm deliberately ignores boundary points for simplicity, accepting approximate results

C. All points in the strip are automatically guaranteed to be within δ of each other

D. A hash table lookup in $O(1)$ replaces the need for pairwise comparisons

Q15. What is the time complexity of the closest-pair-of-points algorithm when the y-sorted strip order is maintained via a linear-time merge at each recursive level (rather than re-sorting)?

A. $\Theta(n \log n)$

B. $\Theta(n \log^2 n)$

C. $\Theta(n^2)$

D. $\Theta(n)$

Q16. A student claims: 'Any divide-and-conquer algorithm that splits a problem of size n into 2 subproblems of size n/2 must run in Θ(n log n) time.' Is this correct?

A. No — complexity also depends critically on the combine step's cost; e.g., $2T(n/2)+\Theta(1)$ gives $\Theta(n)$, while $2T(n/2)+\Theta(n^2)$ gives $\Theta(n^2)$

B. Yes, this follows directly from the Master theorem regardless of the combine cost

C. No, because the number of subproblems must always be a power of 2 for the theorem to apply

D. Yes, because two half-size subproblems always implies balanced logarithmic depth, which fixes the total to n log n

Q17. For recursive binary search, what is the recurrence, and what subtle bug can arise from computing the midpoint as (low+high)/2 rather than low+(high−low)/2 in a fixed-width-integer language?

A. $T(n) = T(n/2) + \Theta(1) = \Theta(\log n)$; low+high can overflow for large indices, producing an incorrect (possibly negative) midpoint

B. $T(n) = 2T(n/2) + \Theta(1) = \Theta(n)$; there is no overflow risk since indices are always small

C. $T(n) = T(n/2) + \Theta(1) = \Theta(\log n)$; the two formulas are always numerically identical, so there is no bug

D. $T(n) = T(n-1) + \Theta(1) = \Theta(n)$; overflow causes linear-time degradation

Q18. What is the recurrence and complexity for the naive recursive Fibonacci computation F(n)=F(n−1)+F(n−2), and why is it NOT considered efficient divide-and-conquer?

A. $T(n) = T(n-1) + T(n-2) + \Theta(1) \approx \Theta(\varphi^n)$ (exponential); it's inefficient because subproblems overlap massively rather than being independent

B. $T(n) = 2T(n/2) + \Theta(1) = \Theta(n)$; it's inefficient only due to constant-factor overhead

C. $T(n) = T(n/2) + \Theta(1) = \Theta(\log n)$; it is actually very efficient

D. $T(n) = T(n-1) + \Theta(1) = \Theta(n)$; it is already efficient and optimal

Q19. In quickselect (D&C-based selection with expected linear time), what is the WORST-case complexity, and under what condition does it occur with a naive 'always pick the last element' pivot strategy?

A. $\Theta(n^2)$, when the input is already sorted (or reverse-sorted), causing maximally unbalanced partitions at every step

B. $\Theta(n \log n)$, on random inputs, matching quicksort's average case

C. $\Theta(n)$, always, because selection only requires one recursive call regardless of pivot quality

D. $\Theta(\log n)$, because only one side of the partition is ever recursed into

Q20. Randomized quickselect selects a uniformly random pivot at each step. What is its expected time complexity, and why does randomization help despite the existence of a $\Theta(n^2)$ worst case?

A. $\Theta(n)$ expected; randomization ensures that, on average, partitions are reasonably balanced, and adversarial worst-case inputs cannot be constructed in advance since the pivot choice is unknown to any fixed input

B. $\Theta(n \log n)$ expected; matches merge sort exactly regardless of pivot choice

C. $\Theta(n^2)$ expected; randomization doesn't change the expected complexity, only the variance

D. $\Theta(1)$ expected; random pivots always land exactly on the median

Q21. The deterministic median-of-medians algorithm guarantees $\Theta(n)$ worst-case selection by choosing pivots via groups of 5. What recurrence captures its worst-case behavior, and why is group size exactly 5 significant?

A. $T(n) \leq T(n/5) + T(7n/10) + \Theta(n)$; group size 5 is the smallest odd size that guarantees at least ~30% of elements are provably smaller (and 30% larger) than the chosen pivot, keeping both recursive calls' combined size below n

B. $T(n) \leq 2T(n/2) + \Theta(n)$; group size is irrelevant to the final complexity

C. $T(n) \leq T(n/2) + T(n/2) + \Theta(n \log n)$; group size 5 minimizes the log factor specifically

D. $T(n) \leq T(n-5) + \Theta(n)$; group size 5 guarantees removing exactly 5 elements per call

Q22. Why does choosing groups of 3 (instead of 5) in the median-of-medians algorithm fail to guarantee linear worst-case time?

A. With groups of 3, the guaranteed fraction of eliminated elements leads to a recurrence like $T(n) \leq T(n/3) + T(2n/3) + \Theta(n)$, where the fractions sum to exactly 1, causing the recurrence to solve to $\Theta(n \log n)$ instead of $\Theta(n)$

B. Groups of 3 cause the algorithm to be incorrect, not just slower

C. Groups of 3 make the partitioning step take $\Theta(n^2)$ time directly

D. There is no difference; groups of 3 also yield $\Theta(n)$

Q23. Consider the D&C algorithm for computing x^n (integer exponentiation) via: if n is even, $x^n = (x^{n/2})^2$; if odd, $x^n = x \cdot (x^{(n-1)/2})^2$. What is its time complexity, and what is the key advantage over the naive Θ(n) iterative multiplication approach?

A. Θ(log n), because the exponent is halved at each recursive step regardless of parity

B. Θ(n), identical to the naive approach since squaring still requires n multiplications total

C. Θ(n log n), because each squaring operation itself costs Θ(log n)

D. Θ(√n), by analogy to other 'fast' algorithms

Q24. In the classic 'counting inversions' problem solved via modified merge sort, an inversion is a pair (i,j) with i<j and A[i]>A[j]. During the MERGE step, when an element from the right subarray is copied because it's smaller than the current left element, how many new inversions does this single event account for, and why?

A. The number of remaining (uncopied) elements in the left subarray, because each of those left elements is greater than the copied right element and appears earlier in the array

B. Exactly one inversion, corresponding to the single comparison just made

C. The total size of the right subarray, because all right elements are assumed smaller

D. Zero, since inversions are only counted after the merge completes, not during it

Q25. What is the overall time complexity of the inversion-counting algorithm based on modified merge sort, and how does it compare to the brute-force $O(n^2)$ pairwise-comparison approach?

A. Θ(n log n); strictly better than the brute-force $O(n^2)$ approach for large n

B. $\Theta(n^2)$; identical to brute force since every pair must still be considered eventually

C. Θ(n); better than brute force because merging is a single linear pass

D. Θ(log n); because inversions only depend on the recursion depth

Q26. Consider the D&C algorithm for finding both the maximum AND minimum of an array simultaneously by comparing elements in pairs and recursing. What is the tight bound on the number of comparisons, and how does it beat the naive 2n−2 approach (separate max and min scans)?

A. $\lceil 3n/2 \rceil - 2$ comparisons, achieved by comparing elements pairwise first and only comparing each pair's winner to the running max and loser to the running min

B. n^2 comparisons, because every element must be compared against every other

C. n log n comparisons, matching a general D&C recurrence with a log factor

D. 2n comparisons; the D&C approach doesn't actually improve on the naive method

Q27. In the 'skyline problem' (merging building silhouettes) solved via D&C, two skylines of sizes m and n are merged in O(m+n) time by a linear sweep. What is the overall recurrence and complexity for n buildings, assuming an even split at each level?

A. $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$

B. $T(n) = 2T(n/2) + \Theta(n^2) = \Theta(n^2)$

C. $T(n) = T(n/2) + \Theta(n) = \Theta(n)$

D. $T(n) = 2T(n/2) + \Theta(\log n) = \Theta(n)$

Q28. A poorly designed D&C algorithm for computing the sum of an array recurses as SUM(A, 1, n) = SUM(A, 1, n−1) + A[n]. What recurrence does this describe, and is it truly 'divide-and-conquer'?

A. $T(n) = T(n-1) + \Theta(1) = \Theta(n)$; this is decrease-by-one recursion, not true divide-and-conquer, since the problem size shrinks by a constant, not a fraction

B. $T(n) = 2T(n/2) + \Theta(1) = \Theta(n)$; this is legitimate D&C with balanced splitting

C. $T(n) = T(n/2) + \Theta(1) = \Theta(\log n)$; this is an efficient D&C solution

D. $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$; this is inefficient D&C

Q29. For the recurrence $T(n) = 7T(n/3) + n^2$, which Master theorem case applies, and what is T(n)?

A. Case 3: $\Theta(n^2)$, since $f(n)=n^2$ grows polynomially faster than $n^{\log_3 7} \approx n^{1.77}$, and the regularity condition holds

B. Case 1: $\Theta(n^{\log_3 7})$, since the recursive term always dominates when $a>1$

C. Case 2: $\Theta(n^2 \log n)$, since the exponents are 'close enough'

D. The Master theorem does not apply because 7 is not a power of the base 3

Q30. What is the output and time complexity of applying the D&C paradigm to evaluate a polynomial of degree n−1 at a single point x using Horner's rule versus a naive term-by-term approach?

A. Horner's rule: $\Theta(n)$ multiplications via nested evaluation $(((a_n x + a_{n-1})x + \ldots)x + a_0)$; naive term-by-term (computing each x^d separately): $\Theta(n^2)$ multiplications without a shared power ladder

B. Both approaches are $\Theta(n \log n)$ since polynomial evaluation always requires a D&C-based FFT-like structure

C. Horner's rule is $\Theta(n^2)$ because of the nested parentheses; naive is $\Theta(n)$

D. Both are $\Theta(n)$ since only n coefficients need to be touched regardless of method

Q31. Consider T(n) = √2·T(n/2) + log n, where a=√2>1. What is $n^{\log_b a}$, and which Master theorem case applies?

A. $n^{\log_2\sqrt{2}} = n^{0.5}$; since f(n)=log n = $O(n^{0.5-\varepsilon})$ for a suitable small ε>0, f(n) IS polynomially smaller than $n^{0.5}$, so Case 1 applies, giving T(n)=$\Theta(n^{0.5})$

B. $n^{\log_2\sqrt{2}} = n^{0.5}$; log n cannot be polynomially smaller than any power of n, so this falls into a genuine Master theorem gap requiring Akra–Bazzi

C. This is Case 2, since log n and $n^{0.5}$ are 'close enough' by convention

D. The recurrence is invalid because a must be an integer for the Master theorem to apply

Q32. A student incorrectly claims that T(n) = 2T(n/2) + n/2 solves to a DIFFERENT complexity class than T(n) = 2T(n/2) + n. Evaluate this claim.

A. False — both solve to Θ(n log n); constant multiplicative factors on f(n) never change the Master theorem's resulting Θ-class

B. True — n/2 is smaller than n, so the first recurrence is Θ(n) while the second is Θ(n log n)

C. True — the first is Θ(n log n) while the second is $\Theta(n^2)$ since whole-number coefficients push it to a higher case

D. False — both are $\Theta(n^2)$ because doubling recursive calls always squares the complexity

Q33. For the D&C convex hull algorithm that splits points by x-coordinate, recursively finds hulls of each half, then merges via finding upper and lower tangent lines in O(n) time, what is the overall complexity, and what dominates it in practice (after an initial sort)?

A. Θ(n log n) overall; dominated by the initial Θ(n log n) sort by x-coordinate, since the recursive merging itself is Θ(n log n) via T(n)=2T(n/2)+Θ(n)

B. $\Theta(n^2)$ overall; dominated by an O(n) tangent-finding step performed naively at every possible pair of points

C. Θ(n) overall; the D&C structure eliminates the need for any sorting

D. $\Theta(n \log^2 n)$ overall; each merge step requires an additional log n factor for tangent-line binary search

Q34. In tournament-style D&C algorithms for finding the second-largest element among n elements (using a single-elimination tournament followed by re-examining only elements the winner directly beat), what is the total comparison count, and how does it improve on the naive 2n−3 approach (find max, then find max excluding it)?

A. $n + \lceil \log_2 n \rceil - 2$ comparisons: n−1 for the tournament to find the max, plus $\lceil \log_2 n \rceil - 1$ more among the $\leq \lceil \log_2 n \rceil$ elements the max directly defeated

B. 2n−2 comparisons, identical to naively finding max then min

C. $n^2/2$ comparisons, since every pair must eventually be compared in a tournament bracket

D. $\log^2 n$ comparisons, since tournaments are inherently logarithmic in every dimension

Q35. For merge sort applied to a LINKED LIST (rather than an array), what critical difference in the divide step changes compared to array-based merge sort, and why doesn't this change the overall Θ(n log n) complexity?

A. Finding the middle of a linked list to split it requires an O(n) traversal (e.g., via a slow/fast pointer technique) rather than O(1) index arithmetic, but this Θ(n) cost per level is already subsumed by the existing Θ(n) merge cost at that level, so the total remains Θ(n log n)

B. Linked-list merge sort is actually Θ(n^2) because pointers cannot be indexed in constant time at all

C. The divide step becomes free (Θ(1)) since linked lists don't require finding a midpoint

D. Linked-list merge sort requires converting to an array first, adding an unavoidable Θ(n log n) overhead on top of the normal algorithm

Q36. A recursive D&C algorithm for matrix transposition splits an n×n matrix into four (n/2)×(n/2) quadrants, recursively transposes each quadrant, then swaps the (already-transposed) off-diagonal quadrants. What is the recurrence, and what is a subtle correctness requirement for the swap step?

A. $T(n) = 4T(n/2) + \Theta(n^2) = \Theta(n^2 \log n)$ if implemented naively with an explicit swap-copy, but the swap of two already-transposed off-diagonal blocks can actually be done via in-place pointer/index swapping in Θ(1) 'bookkeeping' if the algorithm is designed carefully, giving Θ(n^2) overall; the requirement is that recursive transposition must happen BEFORE the block swap, not after, or the wrong elements get swapped

B. $T(n) = 4T(n/2) + \Theta(1) = \Theta(n^2)$ always, with no ordering requirement between transposition and swapping

C. $T(n) = T(n/2) + \Theta(n^2) = \Theta(n^2)$, since only one quadrant needs recursive processing

D. $T(n) = 4T(n/2) + \Theta(n^3) = \Theta(n^3)$, matching naive matrix multiplication's complexity by coincidence

Q37. Which statement correctly distinguishes 'divide-and-conquer' from 'dynamic programming' as paradigms, based on how each handles subproblems?

A. D&C assumes subproblems are independent (non-overlapping), so results aren't cached; DP exploits overlapping subproblems and caches results (memoization/tabulation) to avoid redundant recomputation

B. D&C always uses more memory than DP because it doesn't share subproblem solutions

C. DP never uses recursion, only iteration, while D&C is purely recursive

D. D&C and DP are identical paradigms with only naming differences from different textbook traditions

Q38. For the Master theorem, why is the 'regularity condition' ($a \cdot f(n/b) \leq c \cdot f(n)$ for some $c<1$ and sufficiently large n) specifically required for Case 3 but not for Cases 1 or 2?

A. Case 3 involves f(n) dominating the recursive term, and without the regularity condition, f(n) could oscillate or grow too irregularly across recursive levels such that the sum of costs across all levels doesn't actually converge to being dominated by the top level's cost (unlike Cases 1 and 2, where the polynomial comparison alone is sufficient due to the structure of geometric sums)

B. The regularity condition is actually required for all three cases equally; it's just often omitted from casual statements of Cases 1 and 2 for brevity

C. Case 3 requires it because a must be irrational in that case, and the condition compensates for rounding errors

D. There is no real mathematical reason; it's a historical artifact from how the theorem was first published, not a technical necessity

Q39. Consider a flawed 'proof' that P = NP using divide-and-conquer: 'Any NP problem of size n can be divided into 2 subproblems of size n/2, solved recursively, and combined in polynomial time, so by the Master theorem it runs in polynomial time.' What is the fundamental flaw?

A. The claim that ANY NP problem can be divided into 2 independent half-size subproblems that can be combined in polynomial time is simply false for problems believed to be NP-hard; the Master theorem is a tool for solving KNOWN correct recurrences, not for establishing that such a decomposition exists in the first place

B. The Master theorem only works for sorting algorithms, so it cannot be applied to general NP problems at all

C. The flaw is a computational one: the Master theorem's case analysis was done incorrectly, not the underlying premise

D. There is no flaw; this is actually a valid proof technique that complexity theorists have overlooked

Q40. In-place merge sort variants attempt to reduce the $\Theta(n)$ auxiliary space of standard merge sort. What is the time-complexity trade-off typically incurred by achieving $O(1)$ extra space for the merge step (e.g., via block-based in-place merging techniques)?

A. The merge step's time complexity typically increases (e.g., to $O(n \log n)$ for the merge alone in some techniques, or the constant factor increases substantially), so achieving $O(1)$ space is not 'free' — there's a genuine time-space trade-off

B. There is no trade-off; in-place merging achieves both $\Theta(n)$ time and $O(1)$ space simultaneously with no downsides, which is why all modern implementations use it

C. In-place merging is impossible for merge sort under any circumstances

D. In-place merging always converts merge sort into an unstable sort with no time complexity change

Q41. For the recurrence describing the number of leaves in the recursion tree of T(n) = 4T(n/2) + n (used e.g. in some matrix algorithms), how many leaves are there at the base of the tree, and how does this relate to the Θ(n²) final answer?

A. $n^{\log_2 4}$ = n² leaves, each contributing Θ(1) base-case work, so leaf-level work alone accounts for Θ(n²) — matching the Case-1 conclusion that the leaves (bottom level) dominate the total cost

B. $\log_2 n$ leaves, since the tree has logarithmic depth

C. 4 leaves, since each internal node has exactly 4 children by definition

D. n leaves, matching the linear f(n) term exactly

Q42. A hybrid sorting algorithm uses merge sort for large subarrays but switches to insertion sort once subarray size falls below a threshold k (a common real-world optimization). Why does this hybrid approach typically run FASTER in practice than pure merge sort, despite both being Θ(n log n) asymptotically for appropriate fixed thresholds?

A. Insertion sort has a much smaller constant factor for small inputs (fewer overhead operations per element, better cache locality, no recursive call overhead), so replacing the bottom few levels of merge sort's recursion (which contribute a large fraction of total recursive CALLS, even if not asymptotic time) with insertion sort reduces real-world constant-factor overhead without changing the asymptotic class

B. The hybrid approach is actually Θ(n) because insertion sort is faster than merge sort in all cases

C. There is no real speedup; this is a common misconception perpetuated without empirical basis

D. The hybrid approach changes the recurrence to T(n) = 2T(n/2) + Θ(log n), which is asymptotically faster than standard merge sort

Q43. Which of the following correctly identifies why T(n) = T(n/2) + T(n/2) + O(1) is DIFFERENT from T(n) = T(n/2) + O(1), despite both having only 'n/2'-sized subproblems mentioned?

A. The first has TWO independent recursive calls (each contributing separately to total work), giving T(n)=2T(n/2)+O(1)=Θ(n); the second has only ONE recursive call, giving T(n)=T(n/2)+O(1)=Θ(log n) — the coefficient 'a' (number of subproblems) matters enormously even when subproblem SIZE is identical

B. They are identical; writing T(n/2) twice versus once is just a stylistic difference with no mathematical consequence

C. The first is slower because O(1) is added twice, contributing O(2) total, which changes the growth rate to quadratic

D. The second recurrence is invalid because a single recursive call cannot constitute 'divide-and-conquer'

Q44. In the classic D&C 'find peak element in an array' problem (an element greater than both neighbors, with boundary conventions), the algorithm compares the midpoint to its neighbor and recurses into the half guaranteed to contain a peak. What is the time complexity, and why is correctness guaranteed even though the array need not be sorted?

A. $\Theta(\log n)$; correctness follows because whichever direction has a strictly larger neighboring element is guaranteed to contain at least one peak, due to the array's finite length and boundary behavior (treating out-of-bounds as $-\infty$), regardless of the array's overall ordering

B. $\Theta(n)$; the array must be scanned entirely since it isn't sorted, so binary-search-style logic cannot apply

C. $\Theta(\log^2 n)$; an extra log factor arises from needing to verify both neighbors at every step

D. This problem cannot be solved via divide-and-conquer since peaks can appear anywhere, so no informative decision is possible at the midpoint

Q45. Consider a buggy implementation of merge sort where the base case is written as 'if (high - low <= 1) return;' instead of the correct 'if (low >= high) return;'. For a 2-element subarray, what happens?

A. The 2-element subarray is returned unsorted (the base case incorrectly triggers and skips sorting), producing an incorrectly sorted overall array for certain inputs

B. The algorithm crashes immediately due to an index-out-of-bounds error

C. The algorithm still works correctly because merge sort doesn't need to explicitly handle 2-element subarrays

D. The algorithm runs correctly but in $\Theta(n^2)$ time instead of $\Theta(n \log n)$

Q46. Which best explains why the D&C paradigm is particularly well-suited for PARALLEL computation, using merge sort as the canonical example?

A. The two recursive calls on independent subarrays (left half, right half) have no data dependencies on each other, so they can be executed concurrently on separate processors/threads, with only the final merge step requiring synchronization/sequential combination of results

B. D&C algorithms are inherently sequential and cannot be parallelized due to the recursive call stack

C. Parallelizing D&C requires no synchronization at all, even during the combine/merge step

D. Only iterative (non-recursive) algorithms can be parallelized effectively; D&C's recursive nature is a fundamental obstacle

Q47. For T(n) = 8T(n/2) + Θ(n²) (the naive matrix-multiplication recurrence before Strassen's optimization), what is T(n), and how does it relate to the direct definition-based Θ(n³) matrix multiplication algorithm?

A. Θ(n³), by Master theorem Case 1 (since $n^{\log_2 8}=n^3$ dominates Θ(n²)); this matches the standard triple-nested-loop matrix multiplication complexity exactly, confirming the recursive block-based decomposition is asymptotically equivalent to the direct method

B. Θ(n² log n), by Case 2, strictly better than triple-nested loops

C. Θ(n²), by Case 3, matching the combine step exactly

D. Θ(n².807), by accidentally applying Strassen's exponent to the wrong recurrence

Q48. A D&C algorithm to find the majority element (an element occurring > n/2 times) works by recursively finding candidate majority elements in each half, then verifying by counting. If each half's candidate is found recursively and a final Θ(n) counting pass verifies the true majority, what is the recurrence and complexity, and what subtlety must the combine step handle?

A. T(n) = 2T(n/2) + Θ(n) = Θ(n log n); the combine step must correctly recognize that a global majority element (if one exists) must be a majority element in AT LEAST ONE of the two halves, requiring both halves' candidates to be checked, not just one arbitrarily chosen candidate

B. T(n) = 2T(n/2) + Θ(1) = Θ(n); no verification is needed since the recursive candidates are always correct

C. T(n) = T(n/2) + Θ(n) = Θ(n); only one half needs to be searched since majority elements are always found in the first half by definition

D. T(n) = 2T(n/2) + Θ(n²) = Θ(n²); verification requires comparing every pair of elements from both halves

Q49. Which statement correctly captures a KEY limitation of applying the Master theorem to recurrences arising from randomized divide-and-conquer algorithms, such as randomized quicksort?

A. The Master theorem, as classically stated, applies to WORST-CASE deterministic recurrences of the exact form T(n)=aT(n/b)+f(n) with fixed a and b; randomized algorithms like quicksort have recurrences that depend on a RANDOM split point (varying 'a' and subproblem sizes across runs), so their AVERAGE-CASE analysis typically requires different techniques (like the indicator random variable method) rather than a direct Master theorem application

B. The Master theorem cannot be used for ANY algorithm that includes comparisons, since comparisons introduce implicit randomness

C. Randomized algorithms are always analyzed using the Master theorem's Case 2 by convention, regardless of their actual structure

D. There is no limitation; the Master theorem directly and rigorously handles randomized splits without modification

Divide & Conquer — Answer Key & Explanations

Q1. Correct Answer: B — Θ(n log² n)

Here a=2, b=2, so $n^{\log_b a} = n$. f(n) = n log n = $n^1 \cdot \log^1 n$, matching the extended Master theorem's Case 2 ($f(n) = \Theta(n^{\log_b a} \cdot \log^k n)$ with k=1), giving $T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n) = \Theta(n \log^2 n)$. (A) is the answer for the plain recurrence 2T(n/2)+n without the extra log factor — picking the 'simpler-looking' option is a common trap. (C) wildly overestimates by ignoring the actual computation of $n^{\log_b a}$. (D) invents a log log n term that does not arise from this recurrence's structure at all.

Q2. Correct Answer: A — Θ(n log n)

a=3, b=4, so $n^{\log_4 3} \approx n^{0.79}$. Since f(n) = n log n grows polynomially faster than $n^{0.79}$ ($f(n) = \Omega(n^{\log_4 3 + \varepsilon})$ for some ε>0), and the regularity condition $a \cdot f(n/b) \leq c \cdot f(n)$ holds, this is Case 3, giving Θ(n log n) — the driving function dominates. (B) is correct only if f(n) were absent entirely (T(n)=3T(n/4) alone), a classic distractor from forgetting the additive term. (C) incorrectly blends Case 2's log-multiplier onto a Case 3 scenario. (D) massively overestimates the true growth rate.

Q3. Correct Answer: A — T(n) = 3T(n/2) + O(n), Θ($n^{1.585}$)

Karatsuba reduces 4 naive multiplications to 3 via algebraic manipulation (computing (a+b)(c+d) and subtracting known products), giving T(n) = 3T(n/2) + O(n). By the Master theorem, a=3, b=2, so $n^{\log_2 3} \approx n^{1.585}$ dominates the O(n) term (Case 1), giving $T(n) = \Theta(n^{1.585})$. (B) is the naive grade-school recurrence before Karatsuba's optimization — the entire point of Karatsuba is avoiding that 4th multiplication. (C) incorrectly inflates the combine step to $O(n^2)$, which would erase the benefit of reducing the multiplication count. (D) is the merge-sort recurrence shape, structurally unrelated to Karatsuba's 3-way split.

Q4. Correct Answer: B — Θ($n^{\log_2 7}$) ≈ Θ($n^{2.807}$), strictly better than Θ(n³) for large n

With a=7, b=2: T(n) = 7T(n/2) + O(n²) solves to $\Theta(n^{\log_2 7}) = \Theta(n^{2.807\ldots})$ by Master theorem Case 1, since $\log_2 7 \approx 2.807 > 2$ means the recursive term dominates the $O(n^2)$ combine work. This beats the naive $\Theta(n^3)$ obtained from 8 recursive multiplications ($T(n)=8T(n/2)+O(n^2) = \Theta(n^3)$ since $\log_2 8 = 3$). (A) falsely claims 8 multiplications also yield n^3 'with no gain' — they do yield n^3, but that IS worse than Strassen's 2.807 exponent, so the reasoning is backwards. (C) invents a log factor not implied by Case 1. (D) is a fabricated 'averaging' heuristic with no grounding in the Master theorem.

Q5. Correct Answer: A — A balanced divide-and-conquer with constant combine time (e.g., counting nodes in a balanced binary tree); T(n) = Θ(n)

By the Master theorem: a=2, b=2, $n^{\log_2 2} = n$. f(n)=Θ(1) is polynomially smaller than n (Case 1), so T(n) = Θ(n). This matches problems like counting nodes in a balanced binary tree, where each call does O(1) work but there are Θ(n) total calls. (B) confuses this with T(n)=T(n/2)+Θ(1) (a single recursive call, not two), which gives binary search's Θ(log n) — swapping one recursive call for two changes the answer entirely. (C) is the merge sort recurrence T(n)=2T(n/2)+Θ(n), which has linear (not constant) combine work. (D) is false; recurrences with sub-linear combine steps are common and perfectly valid.

Q6. Correct Answer: A — Depth is $\log_{3/2} n$ (from repeatedly taking the 2n/3 branch); the bound holds because every level of the tree still sums to exactly cn in total work

The tree is unbalanced: the shortest path (always taking n/3) has length $\log_3 n$, but the longest path (always taking 2n/3) has length $\log_{3/2} n$. Crucially, every level — despite the imbalance in branch sizes — sums to exactly cn (since the two children's sizes always add to n), and there are at most $\log_{3/2} n$ levels, giving total cost O(n log n), matching the substitution-method proof. (B) accounts for only the shorter path, underestimating the tree's true depth. (C) and (D) misunderstand unbalanced-but-still-logarithmic recursion: the imbalance doesn't collapse the depth to constant, nor does per-level work blow up to make the bound quadratic.

Q7. Correct Answer: A — Assume T(k) ≤ ck log k for all k < n, substitute k = ⌊n/2⌋, then algebraically show the resulting expression is ≤ cn log n for suitable c > 0, n ≥ n_0

The substitution method uses strong induction: assume the bound holds for all smaller values (the hypothesis), substitute into the recurrence, and show algebraically that the bound then holds for n — often requiring subtraction of a lower-order term to absorb slack, plus separate verification of the base case(s). (B) is circular reasoning: you cannot assume what you are trying to prove for the same n. (C) is incomplete; the base case says nothing about the inductive step, where the real work (handling the +n term) happens. (D) misdescribes the method; there is no direct algebraic 'solving for n' — it is a proof technique, not equation-solving.

Q8. Correct Answer: A — Strengthen the guess to T(n) ≤ cn − b for constant b > 0 to absorb the slack from the +1 term; the true bound remains Θ(n)

This is CLRS's classic 'subtract a lower-order term' technique: naively substituting T(n) ≤ cn into T(n) ≤ c⌊n/2⌋ + c⌈n/2⌉ + 1 = cn + 1 fails due to the extra +1. Strengthening to T(n) ≤ cn − b lets the −b terms combine to cancel the +1, so induction succeeds, and the asymptotic bound remains Θ(n). (C) is wrong: this recurrence solves to Θ(n), not Θ(n log n), since there is no per-level combine cost scaling with n (unlike merge sort's Θ(n) combine step). (B) is a non-sequitur; a failed proof attempt for a linear guess doesn't imply quadratic growth. (D) doesn't work: no matter how large c is, cn + 1 ≤ cn is always false — the fix is structural (subtracting a constant), not a larger constant factor on the same form.

Q9. Correct Answer: A — Case 1 (recursive term dominates): Θ(n²)

a=4, b=2 → $n^{\log_2 4} = n^2$. f(n)=n is polynomially smaller than n^2 ($n = O(n^{2-\varepsilon})$ for ε=1), satisfying Case 1, so T(n) = Θ(n²). (B) misapplies Case 2, which requires $f(n) = \Theta(n^{\log_b a})$ exactly (here that would mean f(n)=Θ(n²), but f(n)=n is not) — a frequent error is assuming any 'close-looking' exponent lands in Case 2. (C) inverts which function dominates; f(n)=n is smaller, not larger, than $n^{\log_b a}=n^2$, so Case 3 cannot apply. (D) gets the case number right but the final value wrong, revealing confusion about computing $n^{\log_b a}$ for a=4, b=2 (which is n², not n log n).

Q10. Correct Answer: A — T(n) = 2T(n/2) + n/log n, because f(n) is asymptotically smaller than n but not by a polynomial factor

The Master theorem requires f(n) to be polynomially smaller, polynomially larger (with the regularity condition), or polynomially equal (within log factors, per the extended version) to $n^{\log_b a}$. Here f(n) = n/log n is smaller than $n=n^{\log_2 2}$ but not by a polynomial factor (n / (n/log n) = log n, which grows slower than any n^{ε}), falling into a gap the basic Master theorem doesn't cover — it needs more advanced tools like the Akra–Bazzi method. (B) is a standard Case 2 application (a=b=3, $f(n)=n=n^{\log_3 3}$), giving Θ(n log n), with no gap at all. (C) is the classic binary search recurrence, solved directly by Case 2, giving Θ(log n). (D) is squarely Case 3 ($f(n)=n^2$ dominates $n^{\log_2 2}=n$ polynomially), solvable directly, giving Θ(n²).

Q11. Correct Answer: A — The merge loop may read past the end of an exhausted subarray, since comparisons continue against a nonexistent element once one side runs out

Sentinels let the merge loop's comparison stay well-defined without a separate 'copy remaining elements' step. Without them and without alternative bounds-checking, once one subarray is fully consumed, the loop keeps comparing against an out-of-range index — undefined behavior or an explicit crash depending on the language. (B) is wrong; this is a correctness/safety bug, not merely a performance regression. (C) is false — this is precisely the kind of subtle bug that corrupts output or crashes rather than being benign. (D) confuses an indexing bug with non-termination; MERGE is not recursive, and MERGE-SORT's recursion terminates via shrinking subarray size regardless of any bug inside MERGE.

Q12. Correct Answer: A — Θ(n), via two separate linear scans outward from the midpoint (one leftward, one rightward)

FIND-MAX-CROSSING-SUBARRAY scans leftward from mid to find the best left extension, then rightward from mid+1 for the best right extension — two linear, non-recursive passes, Θ(n) total. This step is exactly why the overall recurrence is T(n) = 2T(n/2) + Θ(n), matching merge sort's shape. (B) wrongly assumes crossing sums are monotonic enough for binary search; they are not, so every boundary extension must be examined. (C) incorrectly makes the crossing step itself recursive; only the two half-subarray searches recurse, the crossing step is separate and linear. (D) drastically underestimates the work; the best crossing point cannot be found without scanning outward from the midpoint.

Q13. Correct Answer: A — T(n) = 2T(n/2) + Θ(n) = Θ(n log n); Kadane's Θ(n) is asymptotically better

As established, the crossing-subarray step costs Θ(n), giving the merge-sort-shaped recurrence T(n) = 2T(n/2) + Θ(n) = Θ(n log n) by Master theorem Case 2. This is asymptotically worse than Kadane's Θ(n) DP approach, though CLRS presents the D&C version pedagogically before introducing the faster linear algorithm. (B) wrongly treats the crossing step as constant time, which would require somehow knowing the max crossing sum without scanning. (C) mismodels the recursion with only one recursive call instead of two (D&C searches both halves). (D) fabricates a worse-than-brute-force outcome; Θ(n log n) is strictly better than the Θ(n^2) brute-force approach.

Q14. Correct Answer: A — A geometric packing argument shows that within the y-sorted strip, only a small constant number of subsequent points can possibly be closer than δ to any given point, since the recursive δ already bounds minimum spacing in each half

The key lemma: if strip points are sorted by y-coordinate, any point has at most a small constant number of subsequent strip points (a well-known bound, often stated as at most 7) that could possibly be closer than δ — because points closer than δ to each other cannot be packed too densely into a δ×2δ region without violating that δ is already the minimum distance within each recursive half. This keeps the merge step at Θ(n), giving overall Θ(n log n). (B) is false; the classical closest-pair algorithm is exact, not approximate. (C) overstates the guarantee: being in the strip only means being within δ horizontally of the dividing line, not close to every other strip point. (D) fabricates a mechanism; no hashing is involved — the bound is purely geometric.

Q15. Correct Answer: A — Θ(n log n)

Maintaining y-sorted order via merging (like merge sort's merge step) rather than re-sorting at every level gives the recurrence T(n) = 2T(n/2) + Θ(n), which solves to Θ(n log n) — the standard, optimized version. (B) results from the naive variant that re-sorts the strip by y-coordinate at every level using an O(n log n) sort, giving T(n) = 2T(n/2) + Θ(n log n); this is valid but suboptimal, and a very common 'almost right' distractor. (C) would only occur with a brute-force all-pairs approach, defeating the purpose of divide-and-conquer. (D) is impossible for comparison-based geometric closest-pair at this level of generality.

Q16. Correct Answer: A — No — complexity also depends critically on the combine step's cost; e.g., 2T(n/2)+Θ(1) gives Θ(n), while 2T(n/2)+Θ(n²) gives Θ(n²)

This is a critical trap: fixing a=2, b=2 only fixes $n^{\log_b a} = n$; the overall complexity depends entirely on how f(n) (the combine cost) compares to n. With f(n)=Θ(1), Case 1 gives Θ(n); with f(n)=Θ(n), Case 2 gives Θ(n log n) (merge sort); with f(n)=Θ(n²), Case 3 gives Θ(n²). (B) is false precisely because it ignores this dependency — the Master theorem's entire mechanism is comparing f(n) to $n^{\log_b a}$. (C) is a fabricated constraint; a and b need not be powers of 2, though non-integer subdivision requires floor/ceiling handling. (D) repeats the same flawed logic just rephrased — logarithmic depth alone doesn't fix per-level cost.

Q17. Correct Answer: A — T(n) = T(n/2) + Θ(1) = Θ(log n); low+high can overflow for large indices, producing an incorrect (possibly negative) midpoint

Binary search makes one recursive call on a half-sized problem plus constant work, giving T(n) = T(n/2) + Θ(1) = Θ(log n) by Master theorem Case 2 (a=1, b=2, f(n)=Θ(1)=$n^{\log_2 1}$=n^0). The famous documented bug (even seen in production Java) is that low+high can exceed the maximum representable integer when both are large, silently overflowing and corrupting the midpoint; low+(high−low)/2 avoids ever summing two large values. (B) wrongly doubles the recursive calls; binary search discards half the array and recurses on only one side. (C) is factually wrong about the bug — the formulas are mathematically equal in infinite precision but NOT identical under fixed-width overflow, which is the entire point. (D) misapplies both the recurrence and the bug's consequence — overflow corrupts an index; it doesn't degrade the algorithm to linear time.

Q18. Correct Answer: A — T(n) = T(n−1) + T(n−2) + Θ(1) ≈ Θ(φ^n) (exponential); it's inefficient because subproblems overlap massively rather than being independent

Naive Fibonacci branches into two calls of size n−1 and n−2 — a decrease-by-constant (not divide-by-fraction) recursion, and because the subproblems overlap massively (F(n−2) is recomputed independently within both the F(n−1) and F(n) call trees), total work grows exponentially, approximately Θ(φ^n) where φ is the golden ratio (≈1.618). This motivates why memoization/DP (exploiting overlapping subproblems) beats naive recursion — true efficient divide-and-conquer assumes independent, non-overlapping subproblems. (B) and (C) incorrectly model this as size-halving recursion, but Fibonacci's recursion reduces size by a constant amount, so the Master theorem doesn't even apply in that form. (D) drastically underestimates the branching factor; there are two recursive calls, not one, which is exactly what drives the exponential blowup.

Q19. Correct Answer: A — Θ(n²), when the input is already sorted (or reverse-sorted), causing maximally unbalanced partitions at every step

With a naive last-element pivot on already-sorted (or reverse-sorted) input, every partition step splits off just one element, leaving a subproblem of size n−1 — giving T(n) = T(n−1) + Θ(n) = Θ(n²), the worst case. Even though quickselect recurses into only one side (unlike quicksort's two), a consistently unbalanced split still produces quadratic behavior because the linear partition cost is paid n times over a linearly-shrinking sequence of sizes. (B) misattributes Θ(n log n) — that's quicksort's average-case complexity, not quickselect's (quickselect's expected case is Θ(n), better than quicksort's, since it discards one side). (C) is false; a single recursive call doesn't guarantee good complexity if the partition itself is unbalanced — the pivot quality matters enormously. (D) is wrong; only one side IS recursed into (that part is true of quickselect), but that alone doesn't make it Θ(log n) since the sizes can shrink by only 1 each time rather than halving.

Q20. Correct Answer: A — Θ(n) expected; randomization ensures that, on average, partitions are reasonably balanced, and adversarial worst-case inputs cannot be constructed in advance since the pivot choice is unknown to any fixed input

With a uniformly random pivot, the expected recurrence sums geometrically decreasing subproblem sizes across possible pivot ranks, yielding E[T(n)] = Θ(n) — a classic result proved via indicator random variables (similar to randomized quicksort's analysis, but selection needs only one recursive branch, improving the constant). Crucially, no fixed input sequence can force the worst case every time, because the pivot is chosen independently of the input at each call, so an adversary who doesn't see the random choices in advance cannot construct a guaranteed bad case. (B) is wrong; quickselect's expected complexity is linear, strictly better than n log n. (C) misunderstands the effect of randomization; it does change the expected complexity dramatically (from potentially always-quadratic with adversarial fixed pivots to linear in expectation) — it's not merely a variance reduction. (D) is false; random pivots don't guarantee landing on the median; they only guarantee good behavior on average across the randomness.

Q21. Correct Answer: A — T(n) ≤ T(n/5) + T(7n/10) + Θ(n); group size 5 is the smallest odd size that guarantees at least ~30% of elements are provably smaller (and 30% larger) than the chosen pivot, keeping both recursive calls' combined size below n

Median-of-medians recursively finds the median of ~n/5 group-medians (cost T(n/5)), uses it as a pivot, and recurses into the larger partition side, which is provably at most 7n/10 elements (cost T(7n/10)), plus Θ(n) for grouping/partitioning. Since 1/5 + 7/10 = 9/10 < 1, the recurrence solves to Θ(n) via the substitution method (this specific fractional sum being strictly less than 1 is what makes the linear bound provable — smaller groups like 3 would fail this inequality). (B) discards the crucial group-size-driven fractional split, misrepresenting the whole point of the algorithm. (C) fabricates an incorrect recurrence shape and an irrelevant justification about a 'log factor' — there's no log factor to minimize here since the final answer is linear. (D) drastically misunderstands the algorithm's reduction; it doesn't remove a constant number of elements per call, it removes a constant fraction.

Q22. Correct Answer: A — With groups of 3, the guaranteed fraction of eliminated elements leads to a recurrence like $T(n) \leq T(n/3) + T(2n/3) + \Theta(n)$, where the fractions sum to exactly 1, causing the recurrence to solve to $\Theta(n \log n)$ instead of $\Theta(n)$

With groups of 3, only about half of the group-medians are provably less than the pivot (roughly n/6 elements guaranteed on each side from full groups), leading to a worst-case recursion where the larger partition can be as large as roughly 2n/3, giving $T(n) \leq T(n/3) + T(2n/3) + \Theta(n)$. Here the fractions n/3 and 2n/3 sum to exactly n (i.e., the coefficients sum to 1), which is precisely the boundary case where the substitution method's inductive proof fails to produce a strictly decreasing bound, and the recurrence instead solves to $\Theta(n \log n)$, not $\Theta(n)$ — this is why groups of 5 (giving $1/5+7/10=9/10<1$) were specifically chosen. (B) is false; groups of 3 still work correctly, just less efficiently in the worst case. (C) is a fabricated and unrelated claim; the partitioning step itself is unaffected and remains $\Theta(n)$. (D) is factually incorrect and misses the entire mathematical point of why 5 was chosen over smaller odd numbers.

Q23. Correct Answer: A — $\Theta(\log n)$, because the exponent is halved at each recursive step regardless of parity

This is 'fast exponentiation' (exponentiation by squaring): the recurrence is $T(n) = T(n/2) + \Theta(1)$ regardless of whether n is even or odd (the odd case just adds one extra constant-time multiplication), which solves to $\Theta(\log n)$ by Master theorem Case 2 ($a=1$, $b=2$, $f(n)=\Theta(1)=n^0$). This is a dramatic improvement over the naive $\Theta(n)$ approach of multiplying x by itself $n-1$ times. (B) describes the naive approach's complexity, missing the entire point of why the recursive halving strategy is introduced. (C) incorrectly assumes each multiplication itself costs $\Theta(\log n)$ in this model — at this level of the course, multiplication of numbers (not matrices) is standardly treated as $\Theta(1)$ per operation, and even accounting for bit-complexity, that's not the intended justification here. (D) is a fabricated complexity with no derivation basis in this recurrence.

Q24. Correct Answer: A — The number of remaining (uncopied) elements in the left subarray, because each of those left elements is greater than the copied right element and appears earlier in the array

When a right-subarray element R[j] is copied ahead of the remaining left-subarray elements, every one of those remaining left elements L[i], L[i+1], ..., L[end] is (a) positioned before R[j] in the original array's relative order and (b) greater than R[j] (since the left subarray is sorted and R[j] was smaller than the current left element, hence smaller than all subsequent ones too) — so each remaining left element forms an inversion with R[j]. This is the crucial insight that lets inversion-counting piggyback on merge sort's $\Theta(n \log n)$ structure. (B) drastically undercounts; the trick's entire value is that ONE copy operation can resolve MANY inversions in $O(1)$ additional work by just adding a count, not enumerating each pair individually. (C) incorrectly assumes ALL right elements are smaller than ALL left elements, which isn't given by a single comparison. (D) is wrong; the counting must happen precisely during the merge, using the invariant about sorted subarrays, or the $\Theta(n \log n)$ efficiency is lost.

Q25. Correct Answer: A — Θ(n log n); strictly better than the brute-force O(n²) approach for large n

The modified merge sort retains merge sort's exact recurrence T(n) = 2T(n/2) + Θ(n) (the O(1) extra work per copy to increment the inversion counter doesn't change the asymptotic merge cost), giving Θ(n log n) overall — a substantial improvement over the O(n²) brute-force approach that explicitly checks every pair. (B) incorrectly assumes that because the concept 'inversions' is about pairs, the algorithm must inspect pairs individually — but the merge-based trick avoids this via the bulk-counting insight. (C) underestimates the true complexity; a single merge pass is Θ(n), but there are Θ(log n) levels of merging in total, not just one merge. (D) misunderstands the recurrence entirely; recursion depth alone doesn't determine total work when there's linear work performed at every level.

Q26. Correct Answer: A — ⌈3n/2⌉ − 2 comparisons, achieved by comparing elements pairwise first and only comparing each pair's winner to the running max and loser to the running min

The tournament-style approach processes elements in pairs: 1 comparison per pair to find the pair's local max and min (n/2 comparisons for n/2 pairs), then compares each pair's max against the running overall max and each pair's min against the running overall min (2 more comparisons per pair beyond the first), totaling ⌈3n/2⌉ − 2 in the tight analysis — beating the naive approach's 2n−2 (which separately scans for max, using n−1 comparisons, and min, using another n−1). (B) wildly overestimates; this problem does not require all-pairs comparison. (C) invents a log factor that doesn't apply; this is a linear-comparison-count problem, not one governed by a recursive divide that changes the comparison count by a logarithmic factor in this particular formulation (the 3n/2 bound stands regardless of how the recursion is structured, as it's proven via a general comparison-counting/adversary argument). (D) is factually wrong; the whole point of this classical example is that the smarter pairing strategy DOES reduce the comparison count below 2n−2.

Q27. Correct Answer: A — T(n) = 2T(n/2) + Θ(n) = Θ(n log n)

Splitting n buildings into two halves and merging their resulting skylines in linear time (via a coordinated sweep comparing heights at overlapping x-ranges) gives the classic merge-sort-shaped recurrence T(n) = 2T(n/2) + Θ(n) = Θ(n log n) by Master theorem Case 2. (B) drastically overstates the merge cost; the linear-sweep merge is specifically designed to avoid quadratic behavior by processing critical x-coordinates in sorted order rather than comparing all pairs of edges. (C) incorrectly halves the number of recursive calls; the skyline problem genuinely needs both halves solved and then merged, not just one. (D) understates the merge cost to Θ(log n), which is far too little work to correctly merge two silhouettes of total size n — this would only be plausible if the merge step didn't have to examine each building's contribution.

Q28. Correct Answer: A — T(n) = T(n−1) + Θ(1) = Θ(n); this is decrease-by-one recursion, not true divide-and-conquer, since the problem size shrinks by a constant, not a fraction

This recurrence peels off one element per call, giving T(n) = T(n−1) + Θ(1), which unrolls to Θ(n) — asymptotically no worse than a simple loop, but it is 'decrease-and-conquer' (reducing by a constant amount), not divide-and-conquer in the CLRS sense (which specifically involves splitting into smaller-by-a-fraction subproblems, like n/2). This distinction matters conceptually even though the complexity here happens to match a simple linear scan. (B) and (C) incorrectly claim a halving recursion that isn't present in the given pseudocode — the problem explicitly shows SUM(A,1,n−1), a decrement, not SUM(A,1,n/2), a division. (D) incorrectly attributes Θ(n) work per call when only Θ(1) work (the addition of A[n]) happens outside the recursive call.

Q29. Correct Answer: A — Case 3: $\Theta(n^2)$, since $f(n)=n^2$ grows polynomially faster than $n^{\log_3 7} \approx n^{1.77}$, and the regularity condition holds

a=7, b=3, so $n^{\log_3 7} \approx n^{1.771}$. Since $f(n)=n^2$ is polynomially larger ($n^2 = \Omega(n^{1.771+\varepsilon})$ for small ε), and the regularity condition $a \cdot f(n/b) = 7 \cdot (n/3)^2 = (7/9)n^2 \leq c \cdot n^2$ holds for $c=7/9<1$, this is Case 3, giving $\Theta(n^2)$. (B) misapplies Case 1 as a blanket rule; Case 1 specifically requires f(n) to be polynomially SMALLER than $n^{\log_b a}$, which is false here since n^2 is larger. (C) misapplies Case 2, which requires the two functions to match (not merely be 'close'); 2 and 1.771 differing by roughly 0.23 is NOT within the same polynomial class in the Master theorem's precise sense — the theorem provides an exact test, not a fuzzy proximity judgment. (D) is a fabricated restriction; a need not be a power of b for the Master theorem to apply.

Q30. Correct Answer: A — Horner's rule: $\Theta(n)$ multiplications via nested evaluation ($(\ldots(a_n x + a_{n-1})x + \ldots)x + a_0$); naive term-by-term (computing each x^d separately): $\Theta(n^2)$ multiplications without a shared power ladder

Horner's rule restructures the polynomial to require exactly n−1 multiplications and n−1 additions via nested evaluation, giving $\Theta(n)$ — this is not literally a recursive D&C algorithm but is often taught alongside D&C evaluation strategies as an efficient alternative and building block for polynomial-related D&C algorithms (like the FFT). The naive approach recomputes each power x^d independently (e.g., via repeated multiplication) without reusing prior work, costing $\Theta(i)$ per term and summing to $\Theta(n^2)$ overall. (B) incorrectly asserts that O(n log n) FFT-style evaluation is the ONLY way to evaluate a polynomial; single-point evaluation via Horner's rule is a distinct and much simpler $\Theta(n)$ technique, unrelated to multi-point FFT evaluation. (C) has the complexities backwards. (D) ignores that the two approaches genuinely differ in complexity because the naive approach fails to share/reuse partial power computations.

Q31. Correct Answer: A — $n^{\log_2 \sqrt{2}} = n^{0.5}$; since $f(n)=\log n = O(n^{0.5-\varepsilon})$ for a suitable small $\varepsilon>0$, f(n) IS polynomially smaller than $n^{0.5}$, so Case 1 applies, giving $T(n)=\Theta(n^{0.5})$

a=√2, b=2, so $n^{\log_b a} = n^{\log_2 \sqrt{2}} = n^{0.5}$. The key test for Case 1 is whether $f(n) = O(n^{\log_b a - \varepsilon})$ for SOME constant $\varepsilon>0$ — and log n indeed satisfies $\log n = O(n^{0.5-\varepsilon})$ for, say, $\varepsilon=0.4$ (since log n grows slower than $n^{0.1}$, let alone $n^{0.5}$), so Case 1 genuinely applies here, giving $T(n) = \Theta(n^{0.5})$. This question specifically tests the common student overreaction of assuming 'any log n term automatically breaks the Master theorem' — that's only true when f(n) sits in the polynomial GAP between being smaller and being of the exact matching order (as in the earlier n/log n example), not whenever a logarithm appears at all. (B) is the trap: students pattern-match 'log' to 'special case / gap' without actually checking whether the polynomial-gap condition for Case 1 is satisfied — here it clearly is. (C) misapplies Case 2, which requires $f(n)=\Theta(n^{\log_b a})$ exactly (within possible log-factor extensions), not vague 'closeness'. (D) is a fabricated restriction; irrational values of a are perfectly valid, as long as the resulting asymptotic comparisons can be carried out.

Q32. Correct Answer: A — False — both solve to Θ(n log n); constant multiplicative factors on f(n) never change the Master theorem's resulting Θ-class

Θ-notation absorbs constant factors: f(n) = n/2 = Θ(n), so despite the surface-level difference in the coefficient, T(n) = 2T(n/2) + Θ(n) in BOTH cases, and both solve identically to Θ(n log n) by Case 2. This tests whether students understand that Θ(n) and Θ(n/2) are literally the same asymptotic class, not different ones — a very common student misconception is to treat a constant-factor change in the driving function as altering which Master theorem case applies. (B) and (C) fall for exactly this misconception, incorrectly treating n/2 as asymptotically distinct from n. (D) is unrelated nonsense with no mathematical basis; 'doubling recursive calls' doesn't generically square complexity — that depends entirely on the base b and the driving function, per the Master theorem.

Q33. Correct Answer: A — Θ(n log n) overall; dominated by the initial Θ(n log n) sort by x-coordinate, since the recursive merging itself is Θ(n log n) via T(n)=2T(n/2)+Θ(n)

After an initial Θ(n log n) sort by x-coordinate (needed once, not per recursive level), the D&C merge step (finding the upper and lower tangents connecting two hulls) can be done in O(n) time with careful pointer-walking, giving T(n) = 2T(n/2) + Θ(n) = Θ(n log n) for the recursive part — and since the sort is also Θ(n log n), the two combine (not multiply) to remain Θ(n log n) overall. (B) incorrectly assumes a naive $O(n^2)$ tangent search; the whole point of the efficient D&C convex hull algorithm is avoiding this via a smarter walking technique. (C) is false; sorting by x-coordinate is a necessary prerequisite step for this specific divide strategy to make geometric sense (so the 'left half'/'right half' split is spatially meaningful), and it cannot be skipped. (D) fabricates an unnecessary extra log factor; well-known implementations achieve the O(n) merge without needing an internal binary search over tangent candidates.

Q34. Correct Answer: A — n + ⌈log₂n⌉ − 2 comparisons: n−1 for the tournament to find the max, plus ⌈log₂n⌉−1 more among the ≤⌈log₂n⌉ elements the max directly defeated

A single-elimination tournament to find the overall maximum takes exactly n−1 comparisons (each comparison eliminates exactly one contender). The key insight: the second-largest element MUST have lost directly to the overall maximum at some round (since it beat everyone else it faced along the way but couldn't beat the eventual champion), and the maximum played at most ⌈log₂n⌉ matches, so there are at most ⌈log₂n⌉ candidates for second place — finding the best among them takes ⌈log₂n⌉−1 more comparisons via another mini-tournament, giving the improved total of n + ⌈log₂n⌉ − 2, strictly better than the naive 2n−3 for n > a small constant. (B) restates the naive approach's complexity, missing the clever tournament-tracking optimization entirely. (C) drastically overcounts; a well-run tournament never requires quadratic comparisons since half the field is eliminated at each round. (D) is a fabricated bound with no derivation; log²n is not achievable here since finding the max ALONE already requires a linear (n−1) number of comparisons in a comparison-based model.

Q35. Correct Answer: A — Finding the middle of a linked list to split it requires an O(n) traversal (e.g., via a slow/fast pointer technique) rather than O(1) index arithmetic, but this Θ(n) cost per level is already subsumed by the existing Θ(n) merge cost at that level, so the total remains Θ(n log n)

In an array, computing the midpoint index is O(1) arithmetic, but in a singly linked list, you must physically traverse to find the middle node (commonly via the slow/fast, or 'tortoise and hare,' pointer technique), which is an O(n) operation. However, since the merge step at each level of recursion is ALSO already Θ(n), adding another Θ(n) find-the-middle step per level doesn't change the per-level total beyond a constant factor — so the overall complexity remains Θ(n log n), a subtle but important point about how extra linear-time steps can be 'absorbed' without changing the recurrence's Θ-class. (B) is false; this reflects a misunderstanding of what actually changes — the complexity class is preserved, not degraded to quadratic. (C) is wrong; finding the midpoint is not free for linked lists, unlike arrays. (D) is a fabricated and unnecessary requirement; well-implemented linked-list merge sort operates directly on the list structure without ever converting to an array.

Q36. Correct Answer: A — T(n) = 4T(n/2) + Θ(n²) = Θ(n² log n) if implemented naively with an explicit swap-copy, but the swap of two already-transposed off-diagonal blocks can actually be done via in-place pointer/index swapping in Θ(1) 'bookkeeping' if the algorithm is designed carefully, giving Θ(n²) overall; the requirement is that recursive transposition must happen BEFORE the block swap, not after, or the wrong elements get swapped

Transposing an n×n matrix inherently touches Θ(n²) elements, so the theoretically optimal complexity is Θ(n²) — achieved if the swap of the two off-diagonal quadrants (after each is individually transposed) is done efficiently rather than with wasteful extra copying, giving T(n) = 4T(n/2) + Θ(n²/4)-ish work that still totals Θ(n²) via the Master theorem's Case 1 boundary reasoning (careful accounting shows the geometric sum of Θ(n²) work across levels converges to Θ(n²), not Θ(n² log n), when implemented with in-place swaps). The subtle correctness point: each off-diagonal quadrant must be fully transposed internally BEFORE being swapped into its new position, otherwise you'd be swapping un-transposed data into the wrong place, corrupting the result. (B) incorrectly claims Θ(1) combine work, which cannot be right since swapping two (n/2)×(n/2) blocks inherently touches Θ(n²) elements. (C) incorrectly reduces to only one recursive call; transposition must recursively handle all four quadrants (two diagonal ones in place, two off-diagonal ones that get swapped). (D) fabricates a cubic complexity with no justification; transposition never requires more than touching each element a constant number of times.

Q37. Correct Answer: A — D&C assumes subproblems are independent (non-overlapping), so results aren't cached; DP exploits overlapping subproblems and caches results (memoization/tabulation) to avoid redundant recomputation

The defining distinction (as CLRS emphasizes in the DP chapter's introduction) is that D&C algorithms like merge sort or quicksort partition into independent subproblems whose solutions don't overlap, so there is nothing to cache or reuse — while DP problems (like matrix-chain multiplication or Fibonacci) have overlapping subproblems, where naive recursion would redundantly recompute the same subproblem many times, motivating memoization (top-down caching) or tabulation (bottom-up table-filling) to achieve efficiency. (B) is not a defining or generally true characteristic; memory usage depends on the specific algorithm and problem, not the paradigm label itself. (C) is false; DP frequently uses recursion too (memoized/top-down DP is explicitly recursive), and the recursion-vs-iteration axis is orthogonal to the D&C-vs-DP distinction. (D) is false and elides an important conceptual distinction that CLRS explicitly draws out, precisely because failing to recognize overlapping subproblems and using naive D&C-style recursion (like naive Fibonacci) leads to exponential blowup where DP would give polynomial time.

Q38. Correct Answer: A — Case 3 involves f(n) dominating the recursive term, and without the regularity condition, f(n) could oscillate or grow too irregularly across recursive levels such that the sum of costs across all levels doesn't actually converge to being dominated by the top level's cost (unlike Cases 1 and 2, where the polynomial comparison alone is sufficient due to the structure of geometric sums)

In Case 3, f(n) dominates, so intuitively the total cost should be Θ(f(n)) (dominated by the root's cost) — but this requires that the cost doesn't 'blow up' when descending into subproblems, i.e., that $a \cdot f(n/b)$ (the total f-cost one level down) is boundedly smaller than f(n) itself by a constant factor $c<1$; without this, pathological f(n) could cause the recursive levels to sum to more than Θ(f(n)), invalidating the clean 'top level dominates' conclusion. In Cases 1 and 2, the polynomial/exact-match comparison between f(n) and $n^{\log_b a}$ is sufficient on its own because the underlying geometric series argument (either growing or staying constant across levels) works out without needing this extra regularity guarantee — the mathematics of those cases is more forgiving. (B) is incorrect; the standard CLRS statement of the Master theorem genuinely does not require regularity for Cases 1 and 2. (C) is a fabricated and nonsensical justification involving irrational numbers with no basis in the actual theorem. (D) dismisses a real mathematical necessity as a mere historical accident, which is false — the condition is essential to the proof's correctness for Case 3.

Q39. Correct Answer: A — The claim that ANY NP problem can be divided into 2 independent half-size subproblems that can be combined in polynomial time is simply false for problems believed to be NP-hard; the Master theorem is a tool for solving KNOWN correct recurrences, not for establishing that such a decomposition exists in the first place

This question tests conceptual understanding of what the Master theorem actually proves: it solves a recurrence T(n) = aT(n/b) + f(n) GIVEN that such a recurrence correctly models an algorithm — it says nothing about whether a valid, correct, polynomial-time-combinable decomposition into independent half-size subproblems actually EXISTS for a given problem. For NP-hard problems (like 3-SAT or Hamiltonian cycle, believed not to have polynomial-time algorithms), no one has found such a decomposition, and the structure of these problems (e.g., subproblems are not independent, or combining partial solutions is itself intractable) resists this kind of clean splitting — the 'proof' simply assumes the hard part (finding the decomposition) is possible without justification, which is circular. (B) is factually wrong; the Master theorem is a general tool applicable to any divide-and-conquer recurrence, not restricted to sorting. (C) misdiagnoses the flaw as a computational/arithmetic error in applying cases, when the real flaw is a much deeper conceptual gap in the premise itself. (D) is absurd; this exact style of flawed reasoning is a classic 'proof' used to illustrate why the Master theorem cannot be used to establish the EXISTENCE of an efficient decomposition — only to analyze one that's already given.

Q40. Correct Answer: A — The merge step's time complexity typically increases (e.g., to O(n log n) for the merge alone in some techniques, or the constant factor increases substantially), so achieving O(1) space is not 'free' — there's a genuine time-space trade-off

While the standard MERGE procedure achieves Θ(n) time using Θ(n) auxiliary space, true in-place variants (achieving O(1) or O(log n) extra space) generally incur increased time complexity for the merge step itself — some classical in-place merge techniques cost as much as O(n log n) for a single merge (making the WHOLE sort potentially O(n log² n)), or they retain O(n) merge time but with substantially larger constant factors and code complexity, illustrating a genuine space-time trade-off rather than a free lunch. (B) is false and represents a common misconception; if truly free in-place merging with no time cost existed, it would be the universal standard, but in practice most production sorts (like Timsort) accept Θ(n) auxiliary space for merging because the time trade-offs of true in-place merging are usually not worth it. (C) overstates the difficulty; in-place merge sort variants DO exist in the literature, just with more complex implementations and different complexity trade-offs. (D) incorrectly and unnecessarily ties space complexity to stability; a sort can be modified for lower space usage while remaining unrelated in principle to whether it preserves relative order of equal elements, though some specific in-place techniques do sacrifice stability as an additional (separate) trade-off, not an automatic consequence of the space reduction itself.

Q41. Correct Answer: A — $n^{\log_2 4} = n^2$ leaves, each contributing Θ(1) base-case work, so leaf-level work alone accounts for $\Theta(n^2)$ — matching the Case-1 conclusion that the leaves (bottom level) dominate the total cost

The recursion tree for $T(n)=4T(n/2)+n$ has branching factor $a=4$ and depth $\log_2 n$, so the number of leaves is $a^{\log_2 n} = 4^{\log_2 n} = n^{\log_2 4} = n^2$ (using the identity $a^{\log_b n} = n^{\log_b a}$). Since each leaf does Θ(1) base-case work, the leaves alone contribute $\Theta(n^2)$ total — and because this recurrence falls under Master theorem Case 1 ($n^{\log_b a}=n^2$ dominates $f(n)=n$), the leaf-level cost indeed dominates the entire tree's total cost, confirming the $\Theta(n^2)$ answer via this alternative recursion-tree reasoning (rather than direct Master theorem application). (B) confuses the number of leaves with the tree's DEPTH; $\log_2 n$ is the depth, not the leaf count. (C) confuses the branching factor at each single node with the TOTAL leaf count across the whole tree, which compounds multiplicatively across all levels of depth. (D) incorrectly matches the leaf count to the driving function's coefficient, which is a coincidental-looking but incorrect connection — the leaf count comes from a^{depth}, unrelated to the value of f(n).

Q42. Correct Answer: A — Insertion sort has a much smaller constant factor for small inputs (fewer overhead operations per element, better cache locality, no recursive call overhead), so replacing the bottom few levels of merge sort's recursion (which contribute a large fraction of total recursive CALLS, even if not asymptotic time) with insertion sort reduces real-world constant-factor overhead without changing the asymptotic class

This reflects a genuinely important practical algorithm-engineering insight (used in real implementations like Timsort and many C++ STL sort implementations): while merge sort is asymptotically Θ(n log n) regardless of the threshold k (as long as k is a fixed constant, this doesn't change the Big-Θ class), insertion sort has very low constant-factor overhead for small inputs (simple loops, good cache behavior, no recursive call/stack overhead, and it's even adaptive — fast on nearly-sorted small runs), so eliminating the bottom several levels of merge sort's recursion (which involve a very large NUMBER of recursive calls, each with function-call overhead, even though their combined asymptotic contribution is still Θ(n) per level) yields a meaningful constant-factor speedup in practice without changing the asymptotic complexity class at all. (B) is false; insertion sort is $\Theta(n^2)$ in general and is only used here for small, bounded-size subarrays specifically because its overhead is low in that narrow regime — it does not make the WHOLE algorithm Θ(n). (C) is incorrect; this is a well-documented, empirically validated optimization technique, not a myth. (D) fabricates a nonsensical recurrence; the combine (merge) step's cost is unaffected by this optimization and remains Θ(n) per level.

Q43. Correct Answer: A — The first has TWO independent recursive calls (each contributing separately to total work), giving T(n)=2T(n/2)+O(1)=Θ(n); the second has only ONE recursive call, giving T(n)=T(n/2)+O(1)=Θ(log n) — the coefficient 'a' (number of subproblems) matters enormously even when subproblem SIZE is identical

This directly tests whether students grasp that in aT(n/b)+f(n), the coefficient a (how MANY subproblems) is just as important as b (how much smaller each is) — T(n)=T(n/2)+T(n/2)+O(1) is literally 2T(n/2)+O(1) (two separate calls, e.g., processing left and right halves of a tree independently), which is Case 1 ($n^{\log_2 2}=n$ dominates O(1)), giving Θ(n); whereas T(n)=T(n/2)+O(1) has only ONE recursive call (e.g., binary search discarding half the input), which is also technically Case 2 ($n^{\log_2 1}=n^0=1=\Theta(f(n))=\Theta(1)$), giving Θ(log n) — a dramatically different growth rate from the same-looking 'n/2' term. (B) is a serious misconception; writing the recursive term once versus twice fundamentally changes 'a' in the Master theorem and produces different complexity classes (Θ(n) vs Θ(log n)) — this is not stylistic at all. (C) fundamentally misunderstands asymptotic notation; O(1)+O(1)=O(1) still (constant plus constant is still constant), that's not what differs between the two recurrences — what differs is the NUMBER of recursive calls, not the O(1) term. (D) is false; single-recursive-call algorithms like binary search are canonical, well-accepted, efficient divide-and-conquer algorithms.

Q44. Correct Answer: A — Θ(log n); correctness follows because whichever direction has a strictly larger neighboring element is guaranteed to contain at least one peak, due to the array's finite length and boundary behavior (treating out-of-bounds as -∞), regardless of the array's overall ordering

Despite the array not being sorted, this is a classic D&C application: at any midpoint, if the element to the right is larger, that side is guaranteed (by a simple argument treating positions beyond the array boundary as having value -∞) to contain at least one peak — because as you keep moving in the direction of increasing values, you either keep increasing (eventually forced to stop increasing at the boundary, creating a peak) or you find a local peak directly. This gives T(n)=T(n/2)+O(1)=Θ(log n). (B) incorrectly assumes sortedness is required for ANY logarithmic-time divide strategy; this problem is a canonical counterexample showing that a weaker structural guarantee (the existence of SOME peak in the chosen direction) suffices for halving the search space, without full sortedness. (C) fabricates an unnecessary extra log factor; only O(1) comparisons (comparing the midpoint to its immediate neighbors) are needed per level, not a nested logarithmic search. (D) is false and represents a failure to see the clever invariant that makes this problem solvable in logarithmic time despite the lack of global order.

Q45. Correct Answer: A — The 2-element subarray is returned unsorted (the base case incorrectly triggers and skips sorting), producing an incorrectly sorted overall array for certain inputs

With low and high as inclusive indices spanning a 2-element subarray, high−low = 1, so the buggy condition 'high−low ≤ 1' incorrectly treats this AS a base case and returns WITHOUT sorting or recursing further — but a genuine base case should only be a single element (high==low, i.e., high−low==0); a 2-element (or larger) subarray still needs to be divided/sorted. This means any 2-element subarray that happens to be out of order at that point in the recursion is never fixed, producing a final array that can be incorrectly sorted (specific adjacent pairs left unswapped). (B) is incorrect; this bug doesn't cause an indexing crash, it causes a silent logical/correctness error — arguably worse since it may go unnoticed. (C) is false; correctly implemented merge sort absolutely must recurse into (and correctly merge) 2-element subarrays; skipping this step breaks correctness. (D) misdiagnoses the bug's effect entirely; this isn't a performance bug, it's a correctness bug — the algorithm may run FASTER (since it does less work) but produces WRONG output, which is a far more serious flaw than a slowdown.

Q46. Correct Answer: A — The two recursive calls on independent subarrays (left half, right half) have no data dependencies on each other, so they can be executed concurrently on separate processors/threads, with only the final merge step requiring synchronization/sequential combination of results

The defining property that makes D&C naturally parallelizable is precisely the independence of subproblems: in merge sort, sorting the left half and sorting the right half share no data and can genuinely run simultaneously on different cores/threads (this is the basis of practical parallel merge sort implementations), with the only necessarily sequential/synchronized part being the merge step where the two independently-computed results must be combined. (B) is factually backwards; the clean subproblem independence is exactly what ENABLES parallelism, not an obstacle — many high-performance parallel sorting libraries exploit exactly this structure. (C) is incorrect; the merge (combine) step genuinely does require its inputs (the two sorted halves) to be ready, so SOME synchronization is unavoidable at that point, even if the recursive calls themselves are parallel. (D) is false and overly broad; recursive algorithms are commonly and effectively parallelized (e.g., via fork-join frameworks specifically designed for exactly this recursive divide-and-conquer pattern), so recursion itself is not an obstacle to parallelism.

Q47. Correct Answer: A — Θ(n³), by Master theorem Case 1 (since $n^{\log_2 8}=n^3$ dominates Θ(n²)); this matches the standard triple-nested-loop matrix multiplication complexity exactly, confirming the recursive block-based decomposition is asymptotically equivalent to the direct method

With a=8 (eight recursive submatrix multiplications in the naive block decomposition: splitting each of two n×n matrices into four (n/2)×(n/2) blocks requires 8 total submatrix multiplications to compute all four blocks of the result), b=2, we get $n^{\log_2 8}=n^3$, which dominates the Θ(n²) combine/addition work (Case 1), giving Θ(n³) — exactly matching the complexity of the straightforward triple-nested-loop algorithm, confirming that this particular recursive decomposition offers no asymptotic advantage (which is precisely WHY Strassen's insight of reducing from 8 to 7 multiplications was such a significant breakthrough, since it broke this equivalence). (B) misapplies Case 2, which requires exact (not merely 'positive') matching between f(n) and $n^{\log_b a}$; here Θ(n²) is much smaller than Θ(n³), not equal to it. (C) misapplies Case 3, which requires f(n) to DOMINATE, but here f(n)=Θ(n²) is smaller than $n^{\log_b a}=n^3$, the opposite of Case 3's requirement. (D) confuses this recurrence (with a=8, the naive approach) with Strassen's distinct recurrence (with a=7); the exponent $n^{2.807}$ specifically comes from $\log_2 7$, not $\log_2 8$, and applies only to the Strassen-optimized version.

Q48. Correct Answer: A — T(n) = 2T(n/2) + Θ(n) = Θ(n log n); the combine step must correctly recognize that a global majority element (if one exists) must be a majority element in AT LEAST ONE of the two halves, requiring both halves' candidates to be checked, not just one arbitrarily chosen candidate

This is a classic and subtle D&C application: the key correctness insight is that if an element is a majority element (>n/2 occurrences) in the WHOLE array, it must be a majority element in at least one of the two halves (a pigeonhole-style argument: if it weren't, it couldn't possibly reach a majority count when combined) — so the combine step must take BOTH candidates returned by the two recursive calls and verify each with a Θ(n) counting pass over the full array to determine the true overall majority (if any), giving T(n)=2T(n/2)+Θ(n)=Θ(n log n). (B) incorrectly skips the essential verification step; a candidate that's a local majority in one half is not guaranteed to be a GLOBAL majority without an explicit count. (C) incorrectly claims only one half needs searching and that majority elements always appear in 'the first half,' which has no basis — the majority element could be concentrated in either half or split across both. (D) drastically overestimates the verification cost; a single Θ(n) linear counting pass per candidate suffices, not a quadratic all-pairs comparison.

Q49. Correct Answer: A — The Master theorem, as classically stated, applies to WORST-CASE deterministic recurrences of the exact form T(n)=aT(n/b)+f(n) with fixed a and b; randomized algorithms like quicksort have recurrences that depend on a RANDOM split point (varying 'a' and subproblem sizes across runs), so their AVERAGE-CASE analysis typically requires different techniques (like the indicator random variable method) rather than a direct Master theorem application

The Master theorem as presented in CLRS is fundamentally built for the deterministic recurrence form T(n)=aT(n/b)+f(n), where every subproblem is guaranteed to be exactly size n/b; randomized quicksort's recurrence, by contrast, involves a RANDOM pivot that creates subproblems of varying (random) sizes across different executions, so its expected-case Θ(n log n) analysis is instead typically proven via other methods, such as defining indicator random variables for each pair of elements being compared and computing the expected number of comparisons directly — a fundamentally different (though related in spirit) analytical approach, not a literal application of the Master theorem's case-matching machinery. (B) is a nonsensical overgeneralization; comparisons themselves don't introduce randomness (only the RANDOM PIVOT CHOICE does, in the randomized variant specifically). (C) is fabricated; there's no such 'convention,' and applying Case 2 without justification would be mathematically unsound. (D) is false and directly contradicts the actual limitation being tested; the Master theorem, in its standard form, is simply not designed for random/varying subproblem sizes.

Dynamic Programming — Practice Questions

Q50. What are the two key properties a problem must exhibit for dynamic programming to be applicable, as identified in CLRS?

A. Optimal substructure and overlapping subproblems

B. Greedy-choice property and matroid structure

C. Convexity and monotonicity

D. NP-hardness and polynomial reducibility

Q51. In the rod-cutting problem, why does the naive recursive solution CUT-ROD (without memoization) run in $\Theta(2^n)$ time despite there being only n distinct subproblems?

A. Because the recursion tree revisits the same subproblems (e.g., cutting a rod of length k) exponentially many times without caching, since each call to CUT-ROD(n) branches into recursive calls for every smaller length independently, and those calls further re-branch

B. Because the problem doesn't actually have only n distinct subproblems; there are exponentially many distinct rod configurations

C. Because the recursion depth is exponential, not because of redundant recomputation

D. Because each subproblem itself takes exponential time to solve, even without recursion

Q52. For the CUT-ROD problem, what is the recurrence for the optimal revenue r(n), and how many 'choices' does each subproblem consider?

A. $r(n) = \max$ over $1 \le i \le n$ of $(p_i + r(n-i))$, with n choices considered per subproblem (one for each possible first-cut length i)

B. $r(n) = r(n/2) + r(n/2)$, with exactly 2 choices per subproblem

C. $r(n) = p_n$ (the price of the whole uncut rod), with 1 choice per subproblem

D. $r(n) = \max(r(n-1), r(n-2))$, with 2 choices per subproblem, analogous to Fibonacci

Q53. What is the crucial difference between the 'top-down with memoization' and 'bottom-up' (tabulation) approaches to dynamic programming, in terms of which subproblems get solved?

A. Top-down memoization solves ONLY the subproblems that are actually needed (as determined by the recursive calls), potentially skipping some that bottom-up would compute; bottom-up systematically fills the entire table in a fixed order, computing every subproblem regardless of whether it's strictly necessary for the final answer

B. Top-down and bottom-up always solve exactly the same set of subproblems, just in different orders

C. Bottom-up always uses less memory than top-down, since it never uses recursion

D. Top-down cannot be used for any problem that bottom-up can solve, and vice versa

Q54. In the matrix-chain multiplication problem, m[i,j] denotes the minimum number of scalar multiplications to compute the matrix chain product $A_iA_{i+1}...A_\gamma$. What is the recurrence for m[i,j] when i<j, and why must ALL split points k be tried?

A. m[i,j] = min over $i \le k < j$ of $(m[i,k] + m[k+1,j] + p_{i-1}p_kp_\gamma)$; all split points must be tried because the optimal split location isn't known in advance, and different splits can yield vastly different total multiplication counts due to how matrix dimensions interact

B. m[i,j] = m[i,j−1] + m[j,j], with only one split point needed (the last matrix)

C. m[i,j] = $p_i \cdot p_\gamma$, computed directly without any recursive split

D. m[i,j] = min(m[i+1,j], m[i,j−1]), trying only two options: remove the first or last matrix

Q55. For matrix-chain multiplication with n matrices, what is the total running time of the standard bottom-up DP algorithm, and where does this complexity come from?

A. $\Theta(n^3)$: $\Theta(n^2)$ subproblems (all pairs $i \le j$), each requiring up to $\Theta(n)$ choices of split point k to minimize over

B. $\Theta(n^2)$: $\Theta(n)$ subproblems, each requiring $\Theta(n)$ choices

C. $\Theta(n \log n)$: $\Theta(n)$ subproblems solved via a divide-and-conquer-style recursive split

D. $\Theta(2^n)$: exponential, since every possible full parenthesization must be enumerated

Q56. Why does the matrix-chain multiplication problem require filling the DP table m[i,j] in order of increasing CHAIN LENGTH (j−i), rather than simply row-by-row or column-by-column?

A. Because computing m[i,j] requires m[i,k] and m[k+1,j] for all valid k, both of which represent STRICTLY SHORTER sub-chains than [i,j]; filling by increasing chain length guarantees these shorter-chain values are already computed when needed

B. Because row-by-row filling is computationally impossible for this specific recurrence

C. Because chain length order minimizes the total number of table entries needed

D. There's no such requirement; any fill order produces correct results as long as all cells are eventually filled

Q57. In the Longest Common Subsequence (LCS) problem, the recurrence for c[i,j] (LCS length of prefixes X[1..i] and Y[1..j]) is: c[i,j] = c[i−1,j−1]+1 if $x_i=y_Y$, else max(c[i−1,j], c[i,j−1]). What subtle case does this recurrence handle when $x_i=y_Y$, and why is it NOT simply max(c[i−1,j], c[i,j−1], c[i−1,j−1]+1)?

A. When $x_i=y_Y$, the matching character MUST be included in an optimal LCS ending at these positions (proven via a cut-and-paste/exchange argument), so only c[i−1,j−1]+1 is needed — including the other two terms would be redundant since they can never exceed c[i−1,j−1]+1 when characters match

B. Because computing all three terms would be too slow, so the algorithm sacrifices correctness for speed

C. Because c[i−1,j] and c[i,j−1] are undefined when $x_i=y_Y$

D. There's no real difference; both formulations always give identical results in every case, so the simpler one is just stylistic preference

Q58. What is the time and space complexity of the standard LCS DP algorithm for two strings of length m and n, and what technique can reduce the space complexity if only the LCS LENGTH (not the actual subsequence) is needed?

A. Θ(mn) time and Θ(mn) space for the full table; space can be reduced to Θ(min(m,n)) by keeping only the current and previous row/column, since each cell only depends on the immediately preceding row and column

B. Θ(mn) time and Θ(m+n) space always, with no further reduction possible

C. Θ(m+n) time and Θ(mn) space, since only the diagonal needs to be filled

D. Θ(mn^2) time and Θ(mn) space, due to needing all three predecessor cells at every step

Q59. Given two strings 'ABCBDAB' and 'BDCABA', a student computes an LCS DP table but makes an off-by-one error, treating c[i,j] as comparing X[i] to Y[j] directly (0-indexed) while still using a table sized (m+1)×(n+1) with 1-indexed CLRS-style recurrence logic. What is the most likely resulting bug?

A. An index-shift mismatch causing comparisons between the WRONG characters (e.g., comparing X[0] when the logic assumes X[1] was intended), leading to an incorrect LCS length or subsequence due to systematically comparing shifted character pairs

B. The algorithm will crash immediately with an array-index-out-of-bounds exception on the very first comparison

C. The bug has no effect because 0-indexing and 1-indexing are mathematically equivalent for this recurrence

D. The LCS length will always come out exactly one character too long due to a double-counting effect

Q60. What is the key insight that allows the 0/1 Knapsack problem to be solved via dynamic programming in pseudo-polynomial O(nW) time (n items, capacity W), and why is this NOT considered truly polynomial time?

A. The DP table is indexed by (item index, remaining capacity), exploiting the fact that the optimal solution for a given capacity only depends on optimal solutions for smaller capacities and fewer items; it's pseudo-polynomial because W appears as a NUMBER in the input (its magnitude, not the number of bits needed to represent it), so runtime is technically exponential in the BIT-LENGTH of W

B. The DP achieves true polynomial time because W is always bounded by a small constant in practice

C. The insight is sorting items by value-to-weight ratio, which is what makes it polynomial

D. 0/1 Knapsack cannot actually be solved by dynamic programming; only the fractional variant can

Q61. A common student error in 0/1 Knapsack DP is iterating over capacity w in the WRONG order when using a 1D space-optimized array (instead of a full 2D table). What specific bug results from iterating w from LOW to HIGH (ascending) instead of HIGH to LOW (descending) in the space-optimized version?

A. Items get counted MULTIPLE times (effectively turning it into an UNBOUNDED knapsack, where each item can be reused), because updating dp[w] using $dp[w-weight_i]$ with ascending order means $dp[w-weight_i]$ may have ALREADY been updated to include item i earlier in the same pass

B. The algorithm crashes due to negative array indices

C. There is no difference; both iteration orders always produce identical, correct results for 0/1 Knapsack

D. The algorithm undercounts value by skipping every other item due to index parity issues

Q62. For the Longest Increasing Subsequence (LIS) problem, the $O(n^2)$ DP defines dp[i] = length of the LIS ending exactly at index i. What is the recurrence, and why must the algorithm take the MAXIMUM over ALL dp[i] values at the end (rather than just returning dp[n−1])?

A. dp[i] = 1 + max(dp[j] for all j<i where A[j]<A[i]), defaulting to 1 if no such j exists; the overall LIS could END at ANY index, not necessarily the last one, so the final answer requires scanning all dp[i] values to find the true maximum

B. dp[i] = dp[i−1] + 1 always, since the LIS must include every element in order

C. dp[i] = max(dp[i−1], dp[i−2]), analogous to a Fibonacci-style recurrence

D. dp[n−1] alone always gives the correct answer, since the LIS by definition must end at the last element of the array

Q63. What is the improved O(n log n) algorithm for LIS, and what is the key data structure / invariant it maintains that the naive O(n²) DP doesn't exploit?

A. It maintains an auxiliary array 'tails' where tails[k] stores the SMALLEST possible tail value of an increasing subsequence of length k+1 found so far, using binary search to find and update the correct position in O(log n) per element, exploiting the fact that this tails array is always sorted

B. It uses a hash table to store all previously seen elements for O(1) lookup, making each step O(1) instead of O(n)

C. It sorts the array first in O(n log n) time, then the LIS is trivially the entire sorted array

D. It uses a segment tree to answer range-maximum queries, achieving O(log n) per element but requiring O(n²) preprocessing

Q64. Consider the DP recurrence for counting the number of distinct ways to make change for amount V using coin denominations $c_1,\ldots,c_n$ (unlimited supply of each, order doesn't matter — i.e., combinations, not permutations). Why does iterating coins in the OUTER loop and amount in the INNER loop produce a DIFFERENT (correct, combination-counting) result than iterating amount in the outer loop and coins in the inner loop (which counts permutations)?

A. With coins in the outer loop, each coin denomination is 'fully processed' (potentially used any number of times) before moving to the next denomination, which enforces a canonical ORDER on how coins are combined (preventing the same SET of coins used in different orders from being double-counted); with amount in the outer loop, every amount reconsiders ALL coins fresh at each step, allowing the same combination to be counted once for each ORDERING of its coins, giving a permutation count instead

B. There is no actual difference; both loop orders always produce identical counts for this problem

C. The coins-outer version is simply a performance optimization with no effect on which value is computed, only how fast it's computed

D. Amount-outer counting is always wrong and produces negative numbers due to integer underflow

Q65. For the 'minimum edit distance' (Levenshtein distance) DP problem between strings of length m and n, allowing insertion, deletion, and substitution (each cost 1), what is the recurrence for dp[i][j], and what does each of the three terms in the min() represent?

A. dp[i][j] = min(dp[i−1][j]+1, dp[i][j−1]+1, dp[i−1][j−1] + (0 if X[i]=Y[j] else 1)), representing deletion from X, insertion into X (to match Y), and substitution (or match) respectively

B. dp[i][j] = dp[i−1][j−1] + 1 always, since edit distance only ever requires substitution operations

C. dp[i][j] = min(dp[i−1][j], dp[i][j−1]) only, ignoring the diagonal case entirely since substitution is never optimal

D. dp[i][j] = |i−j|, computed directly via a closed-form formula without any recursive table-filling

Q66. In the edit-distance DP table, what do the FIRST ROW and FIRST COLUMN (base cases) represent, and why are they initialized as dp[0][j]=j and dp[i][0]=i?

A. dp[0][j]=j represents transforming an EMPTY string into the first j characters of Y, requiring exactly j insertions; dp[i][0]=i represents transforming the first i characters of X into an EMPTY string, requiring exactly i deletions

B. dp[0][j] and dp[i][0] are arbitrary initialization values with no semantic meaning, chosen only to avoid undefined table lookups

C. dp[0][j]=j and dp[i][0]=i because the empty string always has edit distance 0 to any string, and the formula is a typo/convention rather than a meaningful cost

D. These base cases represent the WORST-CASE edit distance, used only as safety upper bounds that get overwritten during the main computation

Q67. Which of the following is the BEST argument for why the Longest Common Subsequence problem exhibits optimal substructure, matching the formal proof style CLRS uses (via 'cut-and-paste' contradiction)?

A. Suppose Z is a longest common subsequence (LCS) of X and Y, but its prefix Z' (with the last character removed) is NOT the longest common subsequence of the corresponding prefixes of X and Y; then there would exist a STRICTLY LONGER common subsequence Z'' of those prefixes, and appending the removed final character to Z'' would produce a common subsequence of X and Y strictly longer than Z — contradicting that Z was the LCS

B. LCS trivially has optimal substructure because all dynamic programming problems automatically have this property by definition, with no proof needed

C. LCS has optimal substructure only when X and Y are sorted alphabetically, otherwise the property fails

D. The optimal substructure of LCS follows directly from the fact that strings are finite in length, with no need to consider prefixes or subsequences at all

Q68. CLRS presents the 'longest simple path' problem as a case WITHOUT clean optimal substructure (unlike shortest path). Why does decomposing an optimal longest simple path through an intermediate vertex w NOT guarantee that the two sub-paths are themselves optimal (longest) simple paths between their respective endpoints?

A. Because the two 'optimal' sub-paths, if chosen independently to each be the longest simple path between their endpoints, might SHARE vertices with each other, violating the SIMPLICITY (no-repeated-vertex) constraint of the combined path — so the pieces can't always be selected independently and then combined

B. Because longest simple path never has any valid decomposition through intermediate vertices at all

C. Because edge weights in longest-path problems are always negative, breaking the substructure argument used for shortest paths

D. Because the longest simple path problem is always solvable in polynomial time, making the substructure question moot

Q69. In the Optimal Binary Search Tree (OBST) problem, given keys with access probabilities, the DP recurrence for e[i,j] (expected cost of optimal BST containing keys $k_i..k_Y$) involves a term w(i,j) = sum of probabilities from i to j. Why is this w(i,j) term ADDED at every recursive split, rather than just used once?

A. Because whichever key k_r is chosen as the root of the subtree spanning i to j, EVERY key in that range has its depth increased by 1 compared to if it were the root itself, so the TOTAL additional cost across all these keys (weighted by their probabilities) is exactly w(i,j), and this must be added at EVERY level of the recursion where the subtree is 're-rooted' deeper in the overall tree

B. The w(i,j) term is added multiple times purely due to a formatting convention in CLRS with no real mathematical necessity

C. w(i,j) represents the total number of keys in the range, unrelated to depth or cost considerations

D. The term is added because OBST always requires exactly $\log_2 n$ levels of recursion, and w(i,j) compensates for this fixed depth

Q70. What is the time complexity of the standard OBST dynamic programming algorithm for n keys, and how does it compare structurally to matrix-chain multiplication's complexity?

A. $\Theta(n^3)$, structurally analogous to matrix-chain multiplication: $\Theta(n^2)$ subproblems (pairs i,j) each requiring O(n) choices of root to minimize over

B. $\Theta(n^2)$, since OBST only needs to consider each key once as a potential root across the whole algorithm

C. $\Theta(n \log n)$, since OBST can be solved via a balanced-tree-construction shortcut

D. $\Theta(2^n)$, since all possible binary tree SHAPES must be enumerated exhaustively

Q71. For the classic 'coin change: minimum number of coins' DP problem with denominations {1,3,4} and target amount 6, a GREEDY approach (always take the largest denomination that fits) gives 4+1+1=3 coins. What does the correct DP approach give, and why does greedy fail here?

A. DP gives 3+3=2 coins, strictly better than greedy's 3-coin solution; greedy fails because it makes locally optimal choices (largest coin first) without considering how that choice affects the REMAINING amount's own optimal decomposition, and for this particular denomination set, greedy's early commitment to '4' forecloses the better '3+3' option

B. DP and greedy always agree for coin change problems, so both would give 3 coins

C. DP gives 6 coins (six 1s), which is worse than greedy, proving greedy is actually better in this case

D. The problem has no valid DP solution since 4 doesn't evenly divide 6

Q72. In bitmask DP for the Traveling Salesman Problem (TSP), the state is typically dp[mask][i] = minimum cost to visit exactly the set of cities in 'mask', ending at city i. What is the time complexity of this approach for n cities, and why is it considered a significant improvement over brute-force despite still being exponential?

A. $\Theta(n^2 \cdot 2^n)$ time; while still exponential, this is a massive improvement over the brute-force $\Theta(n!)$ approach (trying every permutation of cities), since 2^n grows dramatically slower than $n!$ for even moderately large n

B. $\Theta(n \log n)$, since bitmask DP is fundamentally a divide-and-conquer optimization technique

C. $\Theta(2^n)$ alone, since the number of states is the only factor that matters, with no per-state transition cost

D. $\Theta(n!)$, identical to brute force, since bitmask DP still must consider every possible tour

Q73. For a code-tracing question: given the recursive memoized function `def f(n, memo={}): if n in memo: return memo[n]; if n<=1: return n; memo[n]=f(n-1,memo)+f(n-2,memo); return memo[n]`, what subtle Python bug exists regarding the mutable default argument `memo={}`, and what is its practical consequence?

A. The default dictionary `memo={}` is created ONCE when the function is DEFINED (not each time it's called), so it PERSISTS across separate top-level calls to f() unless a fresh memo is explicitly passed each time — this can cause STALE or unexpectedly shared cached results across logically distinct invocations, though for a pure function like Fibonacci this happens to be harmless (even beneficial for caching) but would be dangerous if memo needed to be reset between independent uses

B. This code has no bugs; mutable default arguments are always recreated fresh on each call in Python

C. The code will throw a RecursionError immediately on any input due to the memo dictionary being empty

D. The bug causes the function to always return 0 regardless of input, due to the memo dictionary overriding the base case

Q74. Trace through this recursive code: `def mystery(n): if n==0: return 0; return n + mystery(n-1)`. If called with a large n WITHOUT memoization, and this were mistakenly assumed to require exponential time due to 'looking recursive like Fibonacci,' what is the ACTUAL time complexity, and why does it differ fundamentally from naive Fibonacci despite both being unmemoized recursion?

A. $\Theta(n)$, because this function makes only ONE recursive call per invocation (not two, like Fibonacci), so there is no branching/overlap at all — it's simply a linear chain of calls, structurally nothing like the exponential-blowup pattern of naive Fibonacci

B. $\Theta(2^n)$, identical to naive Fibonacci, since both are recursive and unmemoized

C. $\Theta(n^2)$, because each call does O(n) work summing up all previous results

D. $\Theta(\log n)$, because recursive functions are always logarithmic unless explicitly stated otherwise

Q75. Given the matrix-chain multiplication dimension array p = [30, 35, 15, 5, 10, 20, 25] (for 6 matrices $A_1 \ldots A_6$), what is the dimension of matrix A_3 specifically, and how many total scalar multiplications would a NAIVE left-to-right (fully left-associative) parenthesization require, versus the true optimal (which CLRS computes as 15,125)?

A. A_3 has dimensions 15×5 (using p[2]×p[3]); left-to-right naive multiplication would require substantially MORE than 15,125 multiplications (the specific naive total for this exact sequence, computable by summing p[i-1]·p[i]·p[6] style costs along a fixed left-associative order, is known to be significantly higher, illustrating why the choice of parenthesization dramatically affects efficiency even though the FINAL matrix product is identical regardless of association

B. A_3 has dimensions 5×10, and left-to-right is always optimal for any matrix chain by definition

C. A_3 has dimensions 35×15, and the naive approach always matches the optimal exactly, making DP unnecessary

D. There is not enough information to determine A_3's dimensions from the array p alone

Q76. For the Fibonacci sequence computed via BOTTOM-UP tabulation (an array fib[0..n] filled iteratively), what is the space complexity if implemented naively with a full array versus the OPTIMIZED version using just two variables, and why does this optimization NOT generalize easily to problems like LCS's full table reconstruction?

A. Naive: Θ(n) space; optimized: Θ(1) space, since fib[i] only depends on fib[i−1] and fib[i−2] — this two-variable trick works because Fibonacci's dependency is on a FIXED, small, constant number of previous states, whereas problems needing full BACKTRACKING (like reconstructing the actual LCS string, not just its length) require retaining more of the table's history to trace the solution path, which the O(1)-space version discards

B. Both versions always require Θ(n) space; there is no possible space optimization for Fibonacci

C. The optimized version requires Θ(log n) space due to the recursive call stack, even in the iterative version

D. Space optimization is impossible for any DP problem that has more than one state variable

Q77. Consider a 2D grid DP problem: 'count the number of distinct paths from top-left to bottom-right of an m×n grid, moving only RIGHT or DOWN.' What is the recurrence, and how does its solution relate to the BINOMIAL COEFFICIENT C(m+n−2, m−1)?

A. dp[i][j] = dp[i−1][j] + dp[i][j−1] (with dp[0][0]=1 and appropriate boundary handling), and the closed-form answer equals C(m+n−2, m−1), since any path consists of exactly (m−1) DOWN moves and (n−1) RIGHT moves in SOME order, and choosing WHICH positions (out of the total m+n−2 moves) are DOWN moves is a combinatorial selection problem

B. dp[i][j] = dp[i−1][j−1] + 1, with no combinatorial interpretation possible for grid path counting

C. dp[i][j] = i×j directly, without any recursive relationship or need for a DP table at all

D. The recurrence dp[i][j] = dp[i−1][j] + dp[i][j−1] is correct, but there is NO closed-form combinatorial equivalent; it must always be computed via the full DP table

Q78. For the same grid-path-counting DP, if certain cells are marked as OBSTACLES (impassable), how must the recurrence be modified, and why does this modification BREAK the clean binomial-coefficient closed form from the unobstructed case?

A. dp[i][j] = 0 if cell (i,j) is an obstacle, else dp[i−1][j] + dp[i][j−1] (with appropriate boundary handling); this breaks the closed form because the binomial-coefficient formula assumed EVERY possible interleaving of moves was a VALID path, but obstacles selectively INVALIDATE specific paths in a way that generally has no simple combinatorial closed-form expression, especially when obstacle positions are arbitrary

B. Obstacles have no effect on the recurrence; the same binomial coefficient formula still applies regardless of obstacle placement

C. dp[i][j] must be multiplied by −1 for obstacle cells, preserving the closed-form solution exactly

D. Obstacles make the problem unsolvable by dynamic programming; only exhaustive path enumeration works

Q79. In subset-sum DP (determine if some subset of a given set sums to exactly target T), the boolean DP table is dp[i][s] = True if some subset of the first i elements sums to exactly s. What is the base case dp[i][0] for ALL i (including i=0), and why is this seemingly 'trivial' base case actually crucial for correctness?

A. dp[i][0] = True for all i, including i=0, because the EMPTY subset (choosing NONE of the available elements) always sums to exactly 0, regardless of how many elements are available to choose from; this must be correctly initialized, or the recurrence (which builds on dp[i-1][s] and dp[i-1][s-a_i]) would incorrectly propagate False for achievable sums that rely on 'not including' certain elements to reach 0 partway through a larger sum calculation

B. dp[i][0] = False for all i, since a sum of exactly 0 is impossible to achieve with any nonempty or empty subset

C. dp[i][0] is undefined and must be explicitly excluded from the table entirely

D. dp[i][0] = True only when i=0, and False for all i>0, since larger subsets can never sum to 0

Q80. What is the time and space complexity of the subset-sum DP for n elements and target sum T, and why is this considered PSEUDO-polynomial (similar to 0/1 Knapsack)?

A. $\Theta(nT)$ time and $\Theta(nT)$ space (reducible to $\Theta(T)$ space via rolling rows); pseudo-polynomial because T's numeric MAGNITUDE (not its bit-length, $\log_2 T$) directly determines the running time, so for exponentially large T (relative to its bit representation), the algorithm's true runtime relative to INPUT SIZE becomes exponential

B. $\Theta(n \log T)$ time, fully polynomial, since T only contributes a logarithmic factor

C. $\Theta(2^n)$ time always, since subset-sum inherently requires trying all 2^n possible subsets

D. $\Theta(n^2)$ time, independent of T entirely, since T only affects space, not time

Q81. Which of these statements about the WEIGHTED INTERVAL SCHEDULING DP problem is CORRECT regarding why a greedy 'earliest finish time first' approach (which works for the UNWEIGHTED version) fails once weights are introduced?

A. With weights, a job with a very high weight but a slightly later finish time might be far more valuable to include than several lower-weight jobs with earlier finish times, so greedily prioritizing EARLIEST FINISH alone (ignoring weight entirely) can lead to a SUBOPTIMAL total weight, whereas the correct DP approach (sorting by finish time, then using p(j) = the last job compatible with j, with recurrence $OPT(j)=\max(OPT(j-1), w_Y+OPT(p(j)))$ correctly considers BOTH options (include or exclude each job) at every step

B. Greedy earliest-finish-time still works perfectly even with weights, since finish time alone always determines optimal scheduling regardless of value

C. The weighted version cannot be solved by DP at all; it requires integer linear programming instead

D. Weights only affect the FINAL total sum reported, not which specific jobs are selected, so greedy selection remains correct with weights simply summed at the end

Q82. For the weighted interval scheduling recurrence OPT(j) = max(OPT(j−1), w_Y+OPT(p(j))), what determines whether job j is INCLUDED in an optimal solution, based on comparing the two terms, and what does p(j) precisely represent?

A. Job j is included if and only if w_Y+OPT(p(j)) > OPT(j−1) (strictly better to include it, given the best achievable using only jobs compatible with j); p(j) is the LARGEST index i<j such that job i's finish time is ≤ job j's start time (i.e., the latest job that doesn't overlap/conflict with job j)

B. Job j is always included if its weight w_Y is positive, regardless of any comparison with other options

C. p(j) represents the total number of jobs that conflict with job j, used purely for counting purposes

D. Job j is included based on a coin flip in the randomized version of this algorithm, since the deterministic version doesn't exist

Q83. Consider the 'palindrome partitioning: minimum cuts' DP problem (find minimum number of cuts to partition a string into palindromic substrings). Why does this problem typically require a TWO-PHASE DP approach (first precomputing which substrings ARE palindromes, then computing minimum cuts), rather than a single unified DP pass?

A. Checking whether an arbitrary substring is a palindrome naively takes O(length) time, and doing this check REPEATEDLY inside the minimum-cuts recurrence (without precomputation) would multiply the overall complexity unnecessarily; precomputing a 2D boolean table isPalin[i][j] in $O(n^2)$ time upfront allows each subsequent palindrome-check during the cuts-DP to be an O(1) lookup instead, keeping the overall complexity at $O(n^2)$ instead of a worse $O(n^3)$

B. A two-phase approach is used purely for code readability, with no actual effect on time complexity

C. Single-phase DP is mathematically impossible for this problem due to a logical contradiction in the recurrence

D. The two phases are needed because palindrome-checking and cut-counting are unrelated problems that happen to share the same input string by coincidence

Q84. In a code-tracing scenario, given the recurrence isPalin[i][j] = (s[i]==s[j]) AND isPalin[i+1][j−1] for the palindrome-precomputation table, what critical base/edge cases must be handled SEPARATELY, and why does naive application of this formula fail for very short substrings?

A. Substrings of length 1 (i==j) are trivially palindromes (isPalin[i][i]=True) and substrings of length 2 (j==i+1) require checking ONLY s[i]==s[j] (since isPalin[i+1][j-1] would reference an INVALID range where i+1>j−1, i.e., an empty or nonsensical range); without handling these separately, the formula would either access out-of-bounds/undefined table entries or incorrectly require satisfying a condition on a non-existent inner substring

B. No special handling is needed; the general formula works correctly for all substring lengths including length 1 and 2 without modification

C. Only length-1 substrings need special handling; length-2 substrings are automatically handled correctly by the general formula

D. The formula is entirely wrong and should instead just check s[i]==s[j] without any recursive inner-substring dependency at any length

Q85. For the 'Egg Drop Puzzle' (find minimum number of trials needed in the worst case to determine the critical floor, given k eggs and n floors), the classic DP recurrence is dp[k][n] = 1 + min over x of max(dp[k−1][x−1], dp[k][n−x]). What do the two terms inside max() represent, and why is MAX (not min) used between them?

A. dp[k−1][x−1] represents the WORST CASE if the egg BREAKS when dropped from floor x (using one fewer egg, searching the x−1 floors below); dp[k][n−x] represents the WORST CASE if the egg SURVIVES (same number of eggs, searching the n−x floors above); MAX is used because you must plan for the WORST-CASE outcome of the drop (you don't control whether it breaks or survives, so you must guarantee success regardless of which happens)

B. Both terms represent the BEST case, and MAX is a typo; it should be MIN, representing optimistic planning

C. dp[k−1][x−1] and dp[k][n−x] are always equal by symmetry, so the choice between max and min never actually matters

D. The two terms represent trying the SAME floor twice with different eggs, and MAX ensures redundant verification

Q86. What is the time complexity of the direct DP implementation of the Egg Drop recurrence for k eggs and n floors (dp[k][n] = 1+min over x of max(dp[k−1][x−1], dp[k][n−x])), and what alternative formulation (reframing the DP state) achieves a better complexity?

A. Direct implementation: $O(k \cdot n^2)$ time, due to trying every floor x (O(n) choices) for each of the O(kn) states; an alternative formulation using dp[k][trials] = MAXIMUM floors distinguishable with k eggs and a given number of trials achieves O(kn) or better, by reframing the problem to avoid the inner search over x entirely

B. Both formulations are always O(kn) with no meaningful complexity difference between them

C. The direct implementation is O(k+n), since it only considers linear factors

D. There is only one possible formulation of this problem; no alternative DP state definition exists

Q87. Which statement BEST explains why Dijkstra's shortest-path algorithm is generally classified as GREEDY rather than dynamic programming, despite shortest-path problems having optimal substructure (a property DP relies on)?

A. Dijkstra's algorithm makes an irrevocable GREEDY choice at each step (permanently finalizing the shortest distance to the closest unvisited vertex, without reconsidering that choice later) rather than exploring/combining multiple possible subproblem solutions and choosing the best AFTER solving them, which is the hallmark DP approach; Dijkstra works correctly as greedy specifically because non-negative edge weights guarantee that once a vertex's shortest distance is finalized, it can never be improved by a LATER-discovered path

B. Dijkstra's algorithm doesn't actually use optimal substructure at all, unlike DP algorithms

C. The classification is arbitrary and historically inconsistent; Dijkstra's could equally correctly be called a DP algorithm with no meaningful distinction

D. Dijkstra's algorithm is actually a DP algorithm, and calling it 'greedy' is simply an outdated or incorrect naming convention

Q88. Consider Bellman-Ford's DP-style relaxation-based shortest path algorithm, which (unlike Dijkstra's) correctly handles NEGATIVE edge weights. Why does Bellman-Ford need to perform relaxation passes exactly |V|−1 times (not more, not fewer, in the standard formulation) to guarantee correctness for graphs without negative cycles?

A. Any shortest path in a graph with V vertices (and no negative cycles) contains AT MOST |V|−1 edges (since a simple path can visit each of the V vertices at most once, using at most V−1 edges to connect them); each full relaxation PASS guarantees correctly computing all shortest paths using AT MOST one additional edge compared to the previous pass, so |V|−1 passes suffice to correctly propagate information across the longest possible shortest path

B. The number |V|−1 is an arbitrary heuristic choice with no mathematical justification, chosen simply because it tends to work well in practice

C. Bellman-Ford actually requires exactly |E| passes (number of edges), not |V|−1 passes, and the standard textbook description is a simplification that happens to work by coincidence

D. |V|−1 passes are needed because that's exactly how many vertices need to be individually removed from consideration during the algorithm's execution

Q89. For the 'Word Break' DP problem (determine if a string can be segmented into space-separated dictionary words), the recurrence is dp[i] = True if there exists some j<i such that dp[j]=True AND s[j..i−1] is in the dictionary. What is the WORST-CASE time complexity, assuming O(1) dictionary lookup (via a hash set) but accounting for substring EXTRACTION cost?

A. $O(n^3)$ in a naive implementation, because there are O(n) values of i, each requiring trying O(n) values of j, and EACH such (i,j) pair requires an O(n) substring extraction (e.g., s[j:i] in many languages creates a NEW string of length up to n) before the O(1) hash lookup can even occur — this substring-copying cost, often overlooked, can dominate the complexity

B. $O(n^2)$ always, since substring extraction is universally an O(1) operation in all programming languages

C. O(n), since dictionary lookup is O(1) and there's no other significant cost

D. $O(2^n)$, since Word Break requires checking all possible ways to segment the string exhaustively without any DP structure

Q90. In the 'Maximum Product Subarray' DP problem, why must the recurrence track BOTH the maximum AND minimum product ending at each position (maxEnd[i] and minEnd[i]), rather than just tracking the maximum (as would suffice for Kadane's maximum SUM subarray)?

A. Because multiplying by a NEGATIVE number can turn the current MINIMUM (most negative) product into the new MAXIMUM (by flipping its sign to positive), so failing to track the running minimum means potentially missing the optimal solution when a negative number appears; this is a fundamental structural difference from the SUM version, where multiplying (which doesn't apply to sums) never causes this sign-flipping interaction

B. Tracking the minimum is an unnecessary optimization with no effect on correctness, only on performance

C. The minimum is tracked purely for reporting purposes at the end, not because it affects the recurrence's correctness during computation

D. Products can never be negative in this problem, so 'minimum' always just refers to the smallest positive value, unrelated to sign considerations

Q91. For 'Regular Expression Matching' via DP (supporting '.' for any single character and '*' for zero-or-more of the preceding element), what makes the handling of the '*' character SIGNIFICANTLY more complex than the '.' character in the DP recurrence?

A. '.' simply means 'match any single character,' requiring a straightforward diagonal-style dependency (dp[i][j] depends on dp[i−1][j−1]); '*' means 'zero or more of the PRECEDING character,' requiring the recurrence to consider BOTH the possibility of using ZERO occurrences (skip the preceding-char+* pattern entirely, dp[i][j−2]) AND the possibility of using ONE OR MORE occurrences (which requires the preceding pattern character to match the current text character, then recursively checking dp[i−1][j], allowing the '*' to consume MULTIPLE text characters via this same-column recursive reference)

B. Both '.' and '*' are handled with the exact same complexity; there is no meaningful difference between them in the recurrence's structure

C. '*' is actually SIMPLER than '.' because it always matches everything, requiring no conditional logic at all

D. The recurrence cannot handle '*' at all; regex matching with '*' requires abandoning DP entirely in favor of a different algorithmic paradigm like NFA simulation

Q92. Which of the following best describes the relationship between DYNAMIC PROGRAMMING and MEMOIZED RECURSION in terms of implementation, using CLRS's own framing?

A. Memoized recursion IS a form of dynamic programming (specifically, the 'top-down' variant), maintaining the natural recursive structure of the problem while adding a cache/table to avoid recomputing already-solved subproblems, in contrast to 'bottom-up' DP, which iteratively fills the table in a carefully chosen dependency-respecting order without explicit recursive calls

B. Memoized recursion and dynamic programming are entirely unrelated techniques that happen to solve similar-looking problems by coincidence

C. Dynamic programming NEVER uses recursion under any circumstances; all DP algorithms must be purely iterative by definition

D. Memoized recursion is strictly FASTER than bottom-up DP in all cases, which is why CLRS recommends always preferring it

Dynamic Programming — Answer Key & Explanations

Q50. Correct Answer: A — Optimal substructure and overlapping subproblems

CLRS identifies optimal substructure (an optimal solution to the problem contains within it optimal solutions to subproblems) and overlapping subproblems (the recursive algorithm revisits the same subproblems repeatedly, making caching worthwhile) as the two defining ingredients for DP applicability. (B) describes properties relevant to GREEDY algorithms, not DP specifically — while greedy problems also have optimal substructure, the matroid/greedy-choice property is a distinct, stronger condition that DP problems need not satisfy. (C) invents mathematical properties (convexity, monotonicity) that are sometimes relevant to specific DP problems' correctness proofs but are not the general defining criteria for DP applicability. (D) is a nonsensical pairing; DP is a technique for solving certain POLYNOMIAL-time-solvable problems, and has no direct connection to NP-hardness as a defining criterion.

Q51. Correct Answer: A — Because the recursion tree revisits the same subproblems (e.g., cutting a rod of length k) exponentially many times without caching, since each call to CUT-ROD(n) branches into recursive calls for every smaller length independently, and those calls further re-branch

CUT-ROD(n) tries every possible first-cut length i from 1 to n, recursively calling CUT-ROD(n−i) for each — but critically, the SAME value of n−i can be reached via many different paths through the recursion tree (e.g., CUT-ROD(5) might be called both directly from CUT-ROD(7) with a cut of 2, and indirectly via CUT-ROD(6) with a cut of 1, then CUT-ROD(5) again), and without memoization, each of these calls redundantly re-solves the same subproblem from scratch, leading to exponential blowup despite only n distinct subproblem VALUES existing. (B) is false; the number of distinct subproblems is exactly n (lengths 0 through n), which is precisely why memoization is so effective — the issue is redundant recomputation, not an actual explosion in distinct subproblems. (C) misattributes the cause; while recursion depth can be up to n, depth alone doesn't cause exponential time — it's the BRANCHING and overlap that causes it. (D) is false; each individual subproblem's 'own' work (excluding recursive calls) is O(n) at most (looping over cut choices), not exponential in itself.

Q52. Correct Answer: A — r(n) = max over 1≤i≤n of (p_i + r(n−i)), with n choices considered per subproblem (one for each possible first-cut length i)

CLRS's rod-cutting recurrence is $r(n) = \max_{1 \le i \le n}(p_i + r(n-i))$, reflecting the choice of where to make the FIRST cut (a piece of length i sold for p_i, plus the optimally cut remainder of length n−i); there are n possible values of i to consider for a rod of length n, giving n choices per subproblem, and Θ(n) subproblems overall, yielding the $O(n^2)$ total running time for the DP solution. (B) fabricates a halving recurrence that doesn't reflect the actual problem structure — rod cutting doesn't have a natural 'split in half' structure since cuts can be made at ANY length. (C) describes the degenerate 'sell the whole rod uncut' case, ignoring all other possible cutting strategies entirely. (D) invents a Fibonacci-like recurrence with no basis in the actual rod-cutting problem, which genuinely requires considering ALL possible first-cut lengths, not just two fixed offsets.

Q53. Correct Answer: A — Top-down memoization solves ONLY the subproblems that are actually needed (as determined by the recursive calls), potentially skipping some that bottom-up would compute; bottom-up systematically fills the entire table in a fixed order, computing every subproblem regardless of whether it's strictly necessary for the final answer

A subtle but real distinction: top-down memoization follows the natural recursive call structure and only computes/caches subproblems that are actually reached from the original call, which can sometimes save work if certain subproblems in the 'full' table are never actually needed for the specific top-level query; bottom-up tabulation, by contrast, typically fills the ENTIRE table in a systematic order (e.g., smallest to largest), computing every entry even if some end up unused by the final answer. In many classic DP problems (like rod cutting, matrix-chain, LCS) all subproblems ARE eventually needed, making this difference moot in practice, but the distinction is real and occasionally matters (e.g., in some graph or game-tree DP where pruning naturally occurs top-down). (B) overstates the equivalence; while they solve the same subproblems in many textbook examples, this is not a general guarantee. (C) is an overgeneralization; while bottom-up often avoids recursion-stack overhead, memory usage depends heavily on the specific problem and implementation, and top-down with memoization can sometimes use LESS memory if many subproblems are skipped. (D) is false; both approaches are generally interchangeable for problems with well-defined optimal substructure and overlapping subproblems, differing mainly in implementation style and sometimes constant-factor performance.

Q54. Correct Answer: A — m[i,j] = min over $i \le k < j$ of $(m[i,k] + m[k+1,j] + p_{i-1} p_k p_\gamma)$; all split points must be tried because the optimal split location isn't known in advance, and different splits can yield vastly different total multiplication counts due to how matrix dimensions interact

The correct recurrence considers splitting the chain $A_i \ldots A_\gamma$ at every possible position k (between A_k and A_{k+1}), computing the cost of the left sub-chain (m[i,k]), the right sub-chain (m[k+1,j]), plus the cost of multiplying the two resulting matrices together ($p_{i-1} p_k p_\gamma$, based on the dimensions) — and since the optimal parenthesization's split point is not known ahead of time (different splits can lead to drastically different total costs depending on the specific dimension sequence), ALL $j-i$ possible values of k must be examined and the minimum taken, which is precisely why this subproblem has '$j-i$ choices' as CLRS notes. (B) incorrectly restricts to only ONE specific split (removing just the last matrix), which is only one of many possible parenthesizations and generally not optimal. (C) is the base case formula (technically m[i,i]=0 for a single matrix, not $p_i \cdot p_\gamma$), not the general recursive case for a chain of 2+ matrices. (D) incorrectly restricts the choices to only 2 fixed options (first or last), ignoring the fact that the optimal split could occur at ANY internal position, which is exactly why the actual recurrence needs to try all $j-i$ candidates.

Q55. Correct Answer: A — Θ(n³): Θ(n²) subproblems (all pairs i≤j), each requiring up to Θ(n) choices of split point k to minimize over

There are Θ(n²) distinct subproblems m[i,j] (since i and j range independently over roughly n values each, subject to i≤j), and for each subproblem, up to Θ(n) split points k must be tried to find the minimum — multiplying subproblem count by choices-per-subproblem gives Θ(n²)×Θ(n) = Θ(n³) total time, exactly matching CLRS's stated complexity for this algorithm. (B) undercounts the number of subproblems; matrix-chain multiplication's subproblems are indexed by PAIRS (i,j), not single indices, giving Θ(n²) subproblems, not Θ(n). (C) is false; there's no logarithmic speedup available here via simple divide-and-conquer, since (unlike, say, merge sort) the optimal split point isn't fixed at the midpoint and must be searched over. (D) describes the naive brute-force enumeration of all parenthesizations (related to the Catalan number of parenthesizations, which IS exponential), which is exactly what DP avoids by exploiting overlapping subproblems — the whole point of using DP here is to avoid this exponential blowup.

Q56. Correct Answer: A — Because computing m[i,j] requires m[i,k] and m[k+1,j] for all valid k, both of which represent STRICTLY SHORTER sub-chains than [i,j]; filling by increasing chain length guarantees these shorter-chain values are already computed when needed

The dependency structure of m[i,j] = min over k of (m[i,k]+m[k+1,j]+...) means m[i,j] depends on m[i,k] and m[k+1,j], both of which correspond to sub-chains STRICTLY SHORTER than the chain from i to j (since k<j and k+1>i within the range) — so filling the table in order of increasing chain length (l=1,2,...,n−1, computing all m[i,i+l] for each l) guarantees that whenever m[i,j] is computed, all the shorter sub-chain values it depends on are already available, avoiding the use of uninitialized/incorrect data. (B) is false; row-by-row filling isn't 'impossible,' but it would visit cells in an order that doesn't respect the dependency structure, leading to using not-yet-computed values. (C) is a fabricated and false justification; the fill order doesn't change how many table entries exist — that's fixed by the problem's Θ(n²) subproblem space regardless of order. (D) is incorrect and represents a serious misunderstanding; the specific dependency structure here means an ARBITRARY fill order (like naive row-major without considering chain length) would very likely access table entries that haven't been computed yet, producing wrong (garbage/uninitialized) results.

Q57. Correct Answer: A — When $x_i=y_Y$, the matching character MUST be included in an optimal LCS ending at these positions (proven via a cut-and-paste/exchange argument), so only c[i−1,j−1]+1 is needed — including the other two terms would be redundant since they can never exceed c[i−1,j−1]+1 when characters match

CLRS proves (via an exchange/cut-and-paste argument in Theorem 14.1) that when $x_i=y_Y$, EVERY optimal LCS of X[1..i] and Y[1..j] must end with this matching character (otherwise, you could always extend a non-matching-ending LCS by this match to get a strictly longer common subsequence, contradicting optimality) — so the recurrence correctly restricts to ONLY c[i−1,j−1]+1 in this case; it is mathematically guaranteed that c[i−1,j]≤c[i−1,j−1]+1 and c[i,j−1]≤c[i−1,j−1]+1 always hold, so including them in a max() would never change the result — they're provably redundant, not merely omitted for convenience. (B) is false and represents a serious misconception; this isn't a correctness-for-speed tradeoff, it's a PROVEN optimization based on the problem's structure. (C) is false; c[i−1,j] and c[i,j−1] are always well-defined table entries regardless of whether x_i equals y_Y — they're simply not NEEDED in this particular case. (D) is misleading; while the simpler formula IS always correct, it's not merely 'stylistic' — it reflects a genuine, provable mathematical fact about the problem's optimal substructure that a naive three-way max wouldn't obviously reveal without the underlying proof.

Q58. Correct Answer: A — Θ(mn) time and Θ(mn) space for the full table; space can be reduced to Θ(min(m,n)) by keeping only the current and previous row/column, since each cell only depends on the immediately preceding row and column

The standard LCS algorithm fills an (m+1)×(n+1) table with Θ(1) work per cell, giving Θ(mn) time and Θ(mn) space for the full table — but since c[i,j] depends ONLY on c[i−1,j−1], c[i−1,j], and c[i,j−1] (i.e., only the immediately preceding row, or the current and previous row), the space can be reduced to O(min(m,n)) by only keeping two rows (or columns) at a time, a classic and important space-optimization technique — though this reduction sacrifices the ability to RECONSTRUCT the actual subsequence via backtracking, since the full table (or an alternative reconstruction method like Hirschberg's algorithm) is needed for that. (B) correctly states the reducible space bound is achievable but incorrectly claims 'no further reduction possible,' when in fact Θ(min(m,n)) (not just Θ(m+n)) is achievable and is the tighter, more standard answer. (C) is factually wrong on both counts; the time complexity is Θ(mn), not Θ(m+n) (every cell must be visited), and space is reducible below Θ(mn), not requiring the full table if only the length is needed. (D) fabricates a higher time complexity with no justification; only 3 predecessor cells are ever examined per cell (O(1) work), not requiring any additional per-cell factor of n.

Q59. Correct Answer: A — An index-shift mismatch causing comparisons between the WRONG characters (e.g., comparing X[0] when the logic assumes X[1] was intended), leading to an incorrect LCS length or subsequence due to systematically comparing shifted character pairs

This is one of the most common real implementation bugs in LCS: CLRS's mathematical formulation is naturally 1-indexed (c[i,j] considers the first i characters of X and first j characters of Y, with X[i] referring to the i-th character in 1-indexed terms), but many programming languages use 0-indexed arrays, so directly translating 'X[i]' in the pseudocode to 'X[i]' in 0-indexed code (instead of the correct 'X[i−1]') causes a systematic off-by-one shift, comparing the WRONG pair of characters at every cell — this typically doesn't crash (since indices usually stay in bounds due to the table being sized (m+1)×(n+1)) but silently produces incorrect results. (B) is unlikely for THIS specific bug pattern; because the table is still sized with the extra row/column buffer, indices generally remain in bounds even with the shift, though the comparisons are simply wrong. (C) is false; 0- and 1-indexing are NOT mathematically interchangeable without adjusting the indexing formula, precisely because the recurrence's meaning of X[i] depends on which convention is used. (D) fabricates a specific, incorrect prediction about the exact nature of the error; an indexing bug like this can cause various incorrect LCS lengths depending on the specific input strings, not a predictable 'always one too long' pattern.

Q60. Correct Answer: A — The DP table is indexed by (item index, remaining capacity), exploiting the fact that the optimal solution for a given capacity only depends on optimal solutions for smaller capacities and fewer items; it's pseudo-polynomial because W appears as a NUMBER in the input (its magnitude, not the number of bits needed to represent it), so runtime is technically exponential in the BIT-LENGTH of W

The 0/1 Knapsack DP defines dp[i][w] = the maximum value achievable using the first i items with capacity exactly (or at most) w, and the recurrence dp[i][w] = max(dp[i−1][w], dp[i−1][w−weight_i]+value_i) (if item i fits) exploits optimal substructure over both dimensions, giving O(nW) time and space — but this is called PSEUDO-polynomial because W's CONTRIBUTION to the running time scales with its numeric VALUE, not the number of BITS needed to encode it in the input (which would be $\log_2 W$); if W is given as, say, a 64-bit number representing a huge capacity, the algorithm's actual running time in terms of INPUT SIZE (which counts bits, per complexity theory convention) is exponential, not polynomial — this is precisely why 0/1 Knapsack is NP-hard (in the weak sense) despite having this seemingly efficient DP solution. (B) is false as a general statement; W is not always small in practice, and the complexity classification doesn't depend on assumptions about typical W values. (C) confuses 0/1 Knapsack's DP with the GREEDY algorithm used for the FRACTIONAL Knapsack variant, where sorting by value-to-weight ratio IS the key technique — but that greedy approach does NOT work correctly for 0/1 Knapsack. (D) is factually wrong; 0/1 Knapsack absolutely CAN be solved via DP (as described), it's simply pseudo-polynomial rather than polynomial in the strict complexity-theoretic sense.

Q61. Correct Answer: A — Items get counted MULTIPLE times (effectively turning it into an UNBOUNDED knapsack, where each item can be reused), because updating dp[w] using dp[w−weight_i] with ascending order means dp[w−weight_i] may have ALREADY been updated to include item i earlier in the same pass

This is a very well-known, subtle DP bug: in the 1D space-optimized 0/1 Knapsack (dp[w] = max(dp[w], dp[w−weight_i]+value_i) for each item i, iterating w in some order), the CORRECT approach iterates w from HIGH to LOW so that dp[w−weight_i] still refers to the value from the PREVIOUS item's pass (i.e., NOT yet including item i), correctly enforcing the 0/1 constraint (each item used at most once); iterating LOW to HIGH instead means that by the time you reach a higher w, dp[w−weight_i] may ALREADY reflect item i being included (from an earlier, smaller-w update within the SAME item's pass), effectively allowing item i to be counted multiple times, which silently transforms the problem into UNBOUNDED knapsack (where item reuse is allowed) — a completely different problem with a different (and generally higher) optimal answer. (B) is false; this bug doesn't cause negative indices or crashes, it causes a silent semantic change to a different problem. (C) is dangerously wrong; this iteration-order bug is one of the most commonly tested 'gotcha' details in DP courses precisely because the difference is completely invisible unless you understand WHY the order matters. (D) is a fabricated and unrelated failure mode with no connection to the actual bug mechanism (which is about incorrect item-reuse, not skipped indices).

Q62. Correct Answer: A — dp[i] = 1 + max(dp[j] for all j<i where A[j]<A[i]), defaulting to 1 if no such j exists; the overall LIS could END at ANY index, not necessarily the last one, so the final answer requires scanning all dp[i] values to find the true maximum

The correct recurrence considers, for each index i, all PRIOR indices j<i where A[j]<A[i] (meaning A[i] can validly extend an increasing subsequence ending at j), taking the best (longest) such extension and adding 1 for A[i] itself; since dp[i] specifically represents subsequences ENDING at index i, and the true LIS of the whole array could end at ANY position (not necessarily the last one — e.g., in [10,9,2,5,3,7,101,18], the LIS [2,3,7,101] ends at index 6, not index 7), the final answer must be max(dp[0], dp[1], ..., dp[n−1]), not simply dp[n−1]. (B) incorrectly assumes the LIS must be contiguous/include every element, which contradicts the very definition of a SUBSEQUENCE (elements need not be contiguous, and not all elements need be included). (C) fabricates an unrelated Fibonacci-style recurrence with no connection to the actual LIS problem's dependency structure, which requires comparing A[i] against ALL valid prior elements, not just the immediately preceding one or two. (D) is a common and serious misconception; nothing guarantees the overall longest increasing subsequence ends at the LAST array position — that assumption is simply false in general.

Q63. Correct Answer: A — It maintains an auxiliary array 'tails' where tails[k] stores the SMALLEST possible tail value of an increasing subsequence of length k+1 found so far, using binary search to find and update the correct position in O(log n) per element, exploiting the fact that this tails array is always sorted

The key insight behind the O(n log n) LIS algorithm is maintaining an array (often called 'tails') where tails[k] represents the smallest possible tail value among all increasing subsequences of length k+1 discovered so far; this array is provably always sorted (a non-obvious but crucial invariant), which allows using BINARY SEARCH to find where each new element should be placed (either extending the LIS or replacing an existing tail with a smaller value to keep future extensions more flexible), giving O(log n) work per element and O(n log n) total — this exploits a MONOTONICITY property (the sorted tails array) that the naive $O(n^2)$ DP's structure (all-pairs comparison) doesn't directly leverage. (B) is a fabricated and incorrect mechanism; a simple hash table lookup doesn't capture the ORDERING information needed to correctly determine increasing subsequence lengths — LIS fundamentally requires comparison-based reasoning about relative order, not just presence/absence. (C) is deeply flawed; sorting the array destroys the original ORDER information that defines what counts as a valid subsequence in the first place — the LIS problem is about subsequences of the ORIGINAL array order, not a sorted version. (D) fabricates a segment-tree-based approach with an internally inconsistent complexity claim ($O(n^2)$ preprocessing would defeat the purpose of an O(n log n) algorithm) that doesn't match any standard LIS algorithm.

Q64. Correct Answer: A — With coins in the outer loop, each coin denomination is 'fully processed' (potentially used any number of times) before moving to the next denomination, which enforces a canonical ORDER on how coins are combined (preventing the same SET of coins used in different orders from being double-counted); with amount in the outer loop, every amount reconsiders ALL coins fresh at each step, allowing the same combination to be counted once for each ORDERING of its coins, giving a permutation count instead

This is one of the most conceptually important 'gotcha' distinctions in introductory DP courses: making COINS the outer loop variable means that when processing coin c_k, you build upon a state that has ALREADY decided how many times coins $c_1,\ldots,c_{k-1}$ are used, establishing an implicit canonical order (e.g., 'always consider coin types in a fixed order, deciding how many of each to use before moving to the next type'), which naturally counts each COMBINATION (unordered multiset of coins) exactly once; making AMOUNT the outer loop instead means at every amount value, ALL coin types are reconsidered without regard to a fixed processing order, so a combination like {1,2} can be reached via the sequence 'first use a 1, then a 2' AND 'first use a 2, then a 1,' double-counting it as if order mattered — correctly computing the number of PERMUTATIONS (ordered sequences) instead of combinations. (B) is a serious and common misconception; the two loop orders provably give DIFFERENT final answers for the counting variant of coin change (though they give the SAME answer for the simpler 'minimum number of coins' or 'can it be made' variants, which is often why students are surprised the counting case differs). (C) mischaracterizes a genuine semantic/correctness difference as merely a performance detail; both orders can have similar time complexity, but they compute mathematically DIFFERENT quantities for the counting problem. (D) is a fabricated and nonsensical failure mode with no basis; there's no integer underflow issue here, the amount-outer version simply computes a different (larger, generally) but still valid non-negative count — it's just answering a different question (permutations vs. combinations).

Q65. Correct Answer: A — dp[i][j] = min(dp[i−1][j]+1, dp[i][j−1]+1, dp[i−1][j−1] + (0 if X[i]=Y[j] else 1)), representing deletion from X, insertion into X (to match Y), and substitution (or match) respectively

The classic edit-distance recurrence considers three operations at each step: deleting a character from X (moving from dp[i−1][j], costing 1), inserting a character into X to match Y (moving from dp[i][j−1], costing 1), and either substituting X[i] for Y[j] or matching them if already equal (moving from the diagonal dp[i−1][j−1], costing 1 if they differ or 0 if they already match) — taking the minimum of these three options correctly captures the cheapest way to transform one prefix into another. (B) incorrectly assumes only substitution is ever needed, completely ignoring insertion and deletion, which are frequently necessary (e.g., transforming 'cat' to 'cats' requires an insertion, not a substitution). (C) incorrectly discards the diagonal (substitution/match) option entirely, which is frequently the OPTIMAL choice, especially when characters already match (giving a free 0-cost move) — omitting it would make the algorithm systematically overestimate the true edit distance. (D) is a fabricated closed-form that is simply incorrect in general; edit distance genuinely depends on the specific CONTENT of the two strings (how many characters match at various alignments), not merely their length difference — |i−j| only provides a LOWER BOUND, not the exact answer.

Q66. Correct Answer: A — dp[0][j]=j represents transforming an EMPTY string into the first j characters of Y, requiring exactly j insertions; dp[i][0]=i represents transforming the first i characters of X into an EMPTY string, requiring exactly i deletions

The base cases have precise semantic meaning grounded in the problem definition: dp[0][j] represents the edit distance between an EMPTY prefix of X and the first j characters of Y, which can ONLY be achieved by inserting all j characters of Y one at a time (since there's nothing in X to delete, substitute, or match against), giving a cost of exactly j; symmetrically, dp[i][0] represents transforming the first i characters of X into an empty string, requiring exactly i deletions, giving cost i. These are not arbitrary values but are directly derivable from the problem's definition applied to the degenerate empty-string case. (B) is false; far from being arbitrary, these values are mathematically FORCED by the problem's semantics — there is exactly one way to transform an empty string into a j-character string using only insertions (or vice versa with deletions), so the cost is uniquely determined. (C) is factually incorrect and conflates a DIFFERENT concept (the trivial fact that a string has distance 0 to ITSELF) with the actual base case here (distance from EMPTY to a non-empty string, which is NOT zero in general). (D) is a fabricated and incorrect description; these are not merely 'safety' placeholder values to be overwritten — they are genuine, correct, final answers for the (0,j) and (i,0) subproblems, used directly by the recurrence to build up the rest of the table.

Q67. Correct Answer: A — Suppose Z is a longest common subsequence (LCS) of X and Y, but its prefix Z' (with the last character removed) is NOT the longest common subsequence of the corresponding prefixes of X and Y; then there would exist a STRICTLY LONGER common subsequence Z'' of those prefixes, and appending the removed final character to Z'' would produce a common subsequence of X and Y strictly longer than Z — contradicting that Z was the LCS

This is precisely the cut-and-paste argument style CLRS Theorem 14.1 formalizes: assuming for contradiction that a proper prefix of an optimal solution is NOT itself optimal for the corresponding subproblem, you show that substituting in a better subproblem solution ('pasting in' the improvement) yields a strictly better solution to the original problem, contradicting the original solution's assumed optimality — this rigorously establishes that optimal solutions to subproblems must be used within optimal solutions to the larger problem, which is the definition of optimal substructure. (B) is a dangerous oversimplification; NOT all algorithmic problems automatically have optimal substructure (the longest SIMPLE PATH problem is a classic CLRS counterexample where it FAILS), so it must be proven for each specific problem, not assumed axiomatically. (C) is a fabricated and false restriction; LCS's optimal substructure holds for ANY two strings, regardless of whether their characters happen to be in alphabetical order — sortedness is irrelevant to this proof. (D) is a non-sequitur; finiteness of the strings alone says nothing about whether optimal solutions to subproblems compose into optimal solutions to the whole problem — that requires the specific structural argument given in option A.

Q68. Correct Answer: A — Because the two 'optimal' sub-paths, if chosen independently to each be the longest simple path between their endpoints, might SHARE vertices with each other, violating the SIMPLICITY (no-repeated-vertex) constraint of the combined path — so the pieces can't always be selected independently and then combined

This is the exact subtlety CLRS highlights: for SHORTEST paths, if you decompose an optimal path through an intermediate vertex, the sub-paths are guaranteed to be optimal (shortest) BECAUSE shortest paths (with appropriately non-negative structure or no negative cycles) can be freely combined without conflict — but for LONGEST SIMPLE paths, the simplicity constraint (no vertex repeated) means that if you pick the longest simple path from u to w and separately the longest simple path from w to v, these two sub-paths might overlap at some vertex OTHER than w, which would make the combined path invalid (not simple) — so you cannot simply combine independently-optimal sub-paths, and the greedy 'solve subproblems optimally and combine' strategy that works for DP breaks down. (B) is false; decompositions through intermediate vertices certainly exist for longest simple paths (any path has intermediate vertices) — the issue is that these decompositions don't have the needed OPTIMAL SUBSTRUCTURE property, not that they don't exist. (C) is a fabricated and irrelevant explanation; the failure of optimal substructure here has nothing to do with edge weight signs — it's purely about the SIMPLICITY (vertex non-repetition) constraint interacting badly with independent sub-path optimization. (D) is factually false and backwards; longest simple path is actually NP-hard in general (related to the Hamiltonian path problem), which is CONSISTENT with (and indeed partially explained by) its lack of clean optimal substructure — it is emphatically not polynomial-time solvable in general.

Q69. Correct Answer: A — Because whichever key k_r is chosen as the root of the subtree spanning i to j, EVERY key in that range has its depth increased by 1 compared to if it were the root itself, so the TOTAL additional cost across all these keys (weighted by their probabilities) is exactly w(i,j), and this must be added at EVERY level of the recursion where the subtree is 're-rooted' deeper in the overall tree

This reflects a genuinely important and often confusing aspect of the OBST recurrence: e[i,j] = min over r of (e[i,r−1] + e[r+1,j] + w(i,j)), where the reasoning is that when you build a subtree spanning keys i to j with root k□, every key OTHER than k□ in that range sits ONE LEVEL DEEPER than it would if it were somehow at the 'root position' of just its own sub-subtree — specifically, when a subtree becomes a CHILD of another node (rather than the overall root), every key within it incurs one additional comparison/depth level in the full tree, and since this happens recursively at every level of nesting, the w(i,j) term (sum of probabilities in the range, representing the TOTAL weighted cost of that extra depth increment for ALL keys in the range) must be added at every level where a subtree gets nested one level deeper — this cumulative addition across all recursive levels correctly captures the TOTAL expected search cost, which inherently depends on the DEPTH of each key, not just which keys are grouped together. (B) is false; this is a mathematically necessary term derivable from the definition of expected search cost as (probability × depth) summed over all keys, not a stylistic or formatting choice. (C) is incorrect; w(i,j) is a sum of PROBABILITIES (weighted by access frequency), not a raw count of keys, and it directly relates to the cost consequences of depth. (D) is a fabricated and false claim; OBST trees are not constrained to have exactly $\log_2 n$ depth (they can be, and often are for OBST specifically, quite unbalanced if that minimizes expected cost given non-uniform probabilities), so this explanation has no basis.

Q70. Correct Answer: A — Θ(n³), structurally analogous to matrix-chain multiplication: Θ(n²) subproblems (pairs i,j) each requiring O(n) choices of root to minimize over

OBST shares the exact same complexity STRUCTURE as matrix-chain multiplication: there are Θ(n²) subproblems e[i,j] (for all valid ranges i≤j), and for each one, up to O(n) choices of root r must be tried to find the minimum expected cost, giving Θ(n²)×O(n) = O(n³) total time (and this is indeed tight, Θ(n³), matching CLRS's stated bound) — this deep structural similarity (both are 'choose a split/root point over a range' DP problems) is a commonly tested conceptual connection between these two classic DP examples. (B) drastically undercounts the choices needed; while there are 'only' n keys overall, EACH of the Θ(n²) different ranges independently requires trying multiple possible roots, not just considering each key a single time globally. (C) is false; there's no known Θ(n log n) algorithm for general OBST with arbitrary probabilities (though various optimized variants like using Knuth's optimization on the choice of optimal root CAN reduce it to Θ(n²) — but not Θ(n log n)). (D) describes the naive brute-force enumeration of ALL possible tree shapes (related to the Catalan numbers, hence exponential), which is exactly the inefficiency that the DP formulation with overlapping subproblems avoids.

Q71. Correct Answer: A — DP gives 3+3=2 coins, strictly better than greedy's 3-coin solution; greedy fails because it makes locally optimal choices (largest coin first) without considering how that choice affects the REMAINING amount's own optimal decomposition, and for this particular denomination set, greedy's early commitment to '4' forecloses the better '3+3' option

This is a canonical counterexample used to teach WHY greedy algorithms don't always work even when a problem has optimal substructure: with denominations {1,3,4}, greedy's rule 'always take the largest fitting coin' picks 4 first (since 4≤6), leaving 2, which then requires two more 1-coins (since 3>2), for a total of 4+1+1=3 coins; but true DP examines ALL possibilities (not just the locally-largest choice) and discovers that 3+3=6 achieves the same target with only 2 coins, which is strictly better — greedy's failure specifically stems from its irrevocable early commitment to using a 4, which then constrains the remaining sub-problem (amount=2) to a worse outcome than if a different first choice had been made. (B) is a dangerous overgeneralization; while greedy DOES happen to work correctly for certain 'nice' denomination sets (like standard currency systems: 1,5,10,25), it is NOT guaranteed to work for ARBITRARY denomination sets, and {1,3,4} targeting 6 is a standard textbook counterexample proving this. (C) is simply computationally wrong; using six 1-coins is a valid but clearly suboptimal solution, and correctly reflects neither the optimal DP answer (2 coins) nor even the greedy answer (3 coins) — it doesn't correspond to any of the algorithms described. (D) is a non-sequitur; there's no requirement that denominations evenly divide the target amount for a DP (or any correct) solution to exist — DP handles arbitrary denominations and targets, as long as making the target is possible using coins of value 1 (guaranteeing solvability here).

Q72. Correct Answer: A — Θ(n²·2ⁿ) time; while still exponential, this is a massive improvement over the brute-force Θ(n!) approach (trying every permutation of cities), since 2ⁿ grows dramatically slower than n! for even moderately large n

The bitmask DP approach has 2^n possible subsets (masks) times n possible ending cities, giving $\Theta(n \cdot 2^n)$ STATES, and for each state, transitioning requires trying up to n possible 'previous city' options, giving an additional factor of n, for a total of $\Theta(n^2 \cdot 2^n)$ — while this remains exponential in n (since 2^n is exponential), it represents an enormous practical improvement over the brute-force Θ(n!) approach of trying every permutation, since for even modest n (like n=20), $2^{20} \approx 10^6$ is vastly more tractable than $20! \approx 2.4 \times 10^{18}$, extending the practically solvable problem size considerably (this exemplifies how DP can convert an intractable brute-force exponential into a 'smaller' but still exponential complexity via avoiding REDUNDANT recomputation across overlapping subsets/states). (B) is false; bitmask DP is a DP technique exploiting overlapping subproblems (indexed by subsets), fundamentally different from divide-and-conquer's independent-subproblem structure, and it is absolutely NOT polynomial (Θ(n log n)) — TSP remains NP-hard. (C) undercounts the per-state work; simply having 2^n (or $n \cdot 2^n$) states doesn't account for the WORK done to compute each state's value, which requires trying up to n transition options per state. (D) is false and misses the entire point of using DP here; while both are technically 'exponential,' 2^n and n! have vastly different growth rates (n! grows much faster), and this difference is exactly why bitmask DP is a meaningful, valuable improvement, not merely a reformulation of the same brute-force complexity.

Q73. Correct Answer: A — The default dictionary `memo={}` is created ONCE when the function is DEFINED (not each time it's called), so it PERSISTS across separate top-level calls to f() unless a fresh memo is explicitly passed each time — this can cause STALE or unexpectedly shared cached results across logically distinct invocations, though for a pure function like Fibonacci this happens to be harmless (even beneficial for caching) but would be dangerous if memo needed to be reset between independent uses

This tests a classic, famous Python 'gotcha': default argument values (like `memo={}`) are evaluated exactly ONCE at function DEFINITION time, not fresh on every call, so the SAME dictionary object persists across ALL calls to f() that don't explicitly pass their own memo — for a pure, input-only-dependent function like Fibonacci, this accidental persistence is actually harmless or even a free optimization (results from previous calls remain valid and get reused), but this pattern is a well-known DANGEROUS bug in general Python code whenever the mutable default is intended to be fresh per call (e.g., if the memo needed to be cleared between uses, or if the function had side effects depending on a clean state) — this question tests recognition of the underlying Python semantics regardless of whether it happens to cause a problem in this SPECIFIC benign case. (B) is factually wrong about Python's actual default-argument evaluation semantics, which is precisely the well-documented gotcha being tested. (C) is false; the code runs correctly and does NOT crash — empty memo is a valid starting state, and the function correctly falls through to compute base cases as needed. (D) is a fabricated and incorrect failure mode with no grounding in how the code actually executes; there's no mechanism by which the memo would override the return value to always be 0.

Q74. Correct Answer: A — Θ(n), because this function makes only ONE recursive call per invocation (not two, like Fibonacci), so there is no branching/overlap at all — it's simply a linear chain of calls, structurally nothing like the exponential-blowup pattern of naive Fibonacci

This function makes exactly ONE recursive call per invocation (mystery(n−1)), forming a simple linear CHAIN of n calls (mystery(n) → mystery(n−1) → ... → mystery(0)), each doing O(1) additional work (the addition), giving Θ(n) total time — there is no branching, no overlapping subproblems, and no redundant recomputation, fundamentally different from Fibonacci's TWO-way branching recursive structure (F(n)=F(n−1)+F(n−2)) which creates an exponentially-sized call tree due to massive subproblem overlap. This question specifically tests whether students can distinguish 'this is recursive' from 'this must be exponential' — NOT all unmemoized recursion is exponential; the KEY factor is whether the recursion BRANCHES into multiple calls that overlap. (B) reflects a common but serious misconception that ALL naive recursion is automatically exponential; the branching factor and subproblem overlap are what matter, not merely 'being recursive.' (C) fabricates an incorrect complexity with no basis; each individual call does only O(1) work (a single addition), not O(n) work. (D) is a nonsensical blanket generalization; there is no such universal rule, and this specific function is clearly linear, not logarithmic, by direct inspection of its single-decrement recursive structure.

Q75. Correct Answer: A — A_3 has dimensions 15×5 (using p[2]×p[3]); left-to-right naive multiplication would require substantially MORE than 15,125 multiplications (the specific naive total for this exact sequence, computable by summing p[i-1]·p[i]·p[6] style costs along a fixed left-associative order, is known to be significantly higher, illustrating why the choice of parenthesization dramatically affects efficiency even though the FINAL matrix product is identical regardless of association

By the standard convention, matrix A_k has dimensions p[i−1]×p[i], so A_3 (with i=3) has dimensions p[2]×p[3] = 15×5 — this dimension-indexing convention (where consecutive matrices share a 'linking' dimension, p[i], ensuring A_k and A_{i+1} are compatible for multiplication) is foundational to the entire matrix-chain DP formulation. Regarding the naive left-to-right approach: CLRS's own worked example for this EXACT dimension sequence demonstrates that different parenthesizations can require dramatically different total multiplication counts (this is literally the motivating example in CLRS Chapter 14, illustrating why finding the optimal parenthesization matters immensely for efficiency, even though mathematically every valid parenthesization computes the IDENTICAL final product due to matrix multiplication's associativity) — the optimal here is stated as 15,125, and a naive fixed-order approach can require substantially more multiplications for unfavorable dimension sequences. (B) gives a wrong dimension calculation (misindexing the p array) and additionally makes the false, sweeping claim that left-to-right is 'always optimal,' directly contradicting the entire motivating point of the matrix-chain problem. (C) gives yet another incorrect dimension pairing and falsely claims naive always matches optimal, which would make the entire DP algorithm pointless — contradicting the well-established fact that parenthesization choice matters significantly. (D) is incorrect; the dimension array p directly and completely determines every matrix's dimensions via the standard convention, so A_3's dimensions ARE fully determined by the given information.

Q76. Correct Answer: A — Naive: Θ(n) space; optimized: Θ(1) space, since fib[i] only depends on fib[i−1] and fib[i−2] — this two-variable trick works because Fibonacci's dependency is on a FIXED, small, constant number of previous states, whereas problems needing full BACKTRACKING (like reconstructing the actual LCS string, not just its length) require retaining more of the table's history to trace the solution path, which the O(1)-space version discards

The naive bottom-up array stores all n+1 Fibonacci values, using Θ(n) space, but since fib[i] depends ONLY on the two immediately preceding values (fib[i−1] and fib[i−2]), you can discard older values and keep just two rolling variables, achieving Θ(1) space — however, this space-reduction trick fundamentally works because you only need the FINAL value, not any INTERMEDIATE path/choice information; problems like LCS, when the goal is to reconstruct the actual optimal SEQUENCE (not just its length), require tracing BACK through the choices made at each step, which necessitates retaining (at least) enough of the table's history to perform this backtracking — simply keeping the 'last couple of rows' as in space-optimized LENGTH-only LCS doesn't preserve the information needed to recover the actual subsequence without additional techniques (like Hirschberg's algorithm, which cleverly achieves both reconstruction AND reduced space via a divide-and-conquer approach on the DP table itself). (B) is false; Fibonacci's rolling-variable optimization is one of the most standard, well-known space optimizations in introductory DP teaching. (C) is a fabricated and internally inconsistent claim; an ITERATIVE bottom-up implementation, by definition, does not use a recursive call stack at all — conflating this with a different (top-down recursive) implementation style is a category error. (D) is an overly broad and false generalization; MANY multi-state-variable DP problems (like certain grid-based DPs, or even 2D DPs like LCS's LENGTH-only computation) DO admit space optimizations by keeping only a limited number of previous rows/columns, when the dependency structure allows it.

Q77. Correct Answer: A — dp[i][j] = dp[i−1][j] + dp[i][j−1] (with dp[0][0]=1 and appropriate boundary handling), and the closed-form answer equals C(m+n−2, m−1), since any path consists of exactly (m−1) DOWN moves and (n−1) RIGHT moves in SOME order, and choosing WHICH positions (out of the total m+n−2 moves) are DOWN moves is a combinatorial selection problem

The number of ways to reach cell (i,j) is the sum of ways to reach it from ABOVE (dp[i−1][j], then move down) or from the LEFT (dp[i][j−1], then move right), correctly captured by dp[i][j]=dp[i−1][j]+dp[i][j−1] — and this DP recurrence has a beautiful and well-known CLOSED-FORM equivalent: since every valid path from top-left to bottom-right requires EXACTLY (m−1) down-moves and (n−1) right-moves (in some interleaved order) out of a TOTAL of (m+n−2) moves, the number of distinct paths equals the number of ways to choose WHICH (m−1) of those (m+n−2) total move-slots are 'down' moves (the rest being 'right' by elimination), which is EXACTLY the binomial coefficient C(m+n−2, m−1) — this is a classic and important illustration of how DP solutions can sometimes be verified against, or even replaced by, elegant closed-form combinatorial formulas when such a formula exists. (B) fabricates an incorrect, overly simplistic recurrence that doesn't correctly model the actual TWO-WAY (from-above OR from-left) path-counting structure of the problem. (C) is a nonsensical direct-multiplication formula with no grounding in the actual combinatorics of path-counting (i×j does not equal the number of grid paths in general). (D) is factually incorrect; while the recurrence IS correct as stated, this is precisely a case where a clean closed-form DOES exist (the binomial coefficient), demonstrating that DP and combinatorial closed-forms are often two equivalent ways of solving the same underlying counting problem, not mutually exclusive approaches.

Q78. Correct Answer: A — dp[i][j] = 0 if cell (i,j) is an obstacle, else dp[i−1][j] + dp[i][j−1] (with appropriate boundary handling); this breaks the closed form because the binomial-coefficient formula assumed EVERY possible interleaving of moves was a VALID path, but obstacles selectively INVALIDATE specific paths in a way that generally has no simple combinatorial closed-form expression, especially when obstacle positions are arbitrary

The natural and correct modification sets dp[i][j]=0 for any obstacle cell (since no valid path can pass through it, contributing zero ways to reach further cells via that position), while non-obstacle cells still sum contributions from above and left (using the now-corrected zeroed-out values for blocked neighbors) — this modification is precisely why the elegant closed-form binomial coefficient (which counts ALL possible move-orderings without regard to obstacles) no longer applies: the combinatorial formula fundamentally assumed every interleaving of down/right moves traces a VALID path, but once certain grid cells are forbidden, SPECIFIC move-orderings become invalid (those that would pass through a blocked cell), and there's no simple, general closed-form expression for 'count binomial arrangements EXCLUDING those that hit an arbitrary set of forbidden points' — this beautifully illustrates a broader principle in DP: adding CONSTRAINTS to a problem can destroy an existing closed-form solution while the DP recurrence itself remains easily adaptable (just add a conditional check), which is exactly why DP's flexibility is often preferred over closed-form combinatorics for problems with irregular constraints. (B) is false and represents a serious misunderstanding of how obstacles fundamentally change which paths are valid, hence changing the resulting COUNT. (C) is a fabricated and mathematically nonsensical modification (multiplying by −1 would produce meaningless negative path counts, and doesn't correspond to any correct handling of obstacles). (D) is false; the modified DP approach (setting obstacle cells to 0) is a well-known, efficient, and entirely standard solution — exhaustive enumeration is unnecessary and dramatically less efficient.

Q79. Correct Answer: A — dp[i][0] = True for all i, including i=0, because the EMPTY subset (choosing NONE of the available elements) always sums to exactly 0, regardless of how many elements are available to choose from; this must be correctly initialized, or the recurrence (which builds on dp[i-1][s] and dp[i-1][s-a_i]) would incorrectly propagate False for achievable sums that rely on 'not including' certain elements to reach 0 partway through a larger sum calculation

The base case dp[i][0]=True for EVERY i (not just i=0) reflects the simple but crucial fact that the EMPTY subset (selecting zero elements from whatever pool of i elements is available) always achieves a sum of exactly 0, no matter how many elements you have access to — this must be correctly set as a TRUE base case across the entire first column of the table, because the general recurrence dp[i][s] = dp[i−1][s] OR (dp[i−1][s−a_i] if s≥a_i) relies on being able to correctly look up dp[i−1][0]=True whenever a partial sum calculation needs to confirm that 'sum=0 is achievable using a subset of the first i−1 elements' (which is always trivially true via the empty selection) — getting this base case wrong (e.g., only setting dp[0][0]=True and leaving dp[i][0] for i>0 uninitialized or False) would cause the recurrence to incorrectly report FALSE for various achievable sums that depend on this lookup. (B) is factually incorrect and reflects a serious misunderstanding; a sum of exactly 0 is ALWAYS trivially achievable via the empty subset, for any i, making this a TRUE base case, not False. (C) is an evasive non-answer; the base case is both well-defined and essential to state explicitly for the recurrence to function correctly. (D) incorrectly restricts the True base case to ONLY i=0, but the recurrence's correctness for i>0 typically depends on ALSO having dp[i][0]=True (not just dp[0][0]) readily available at each row, since the empty-subset-sums-to-zero fact holds independently of how many elements are available to potentially include.

Q80. Correct Answer: A — Θ(nT) time and Θ(nT) space (reducible to Θ(T) space via rolling rows); pseudo-polynomial because T's numeric MAGNITUDE (not its bit-length, $\log_2 T$) directly determines the running time, so for exponentially large T (relative to its bit representation), the algorithm's true runtime relative to INPUT SIZE becomes exponential

Subset-sum's DP table has dimensions (n+1)×(T+1), with O(1) work per cell, giving Θ(nT) time and space (reducible to Θ(T) space via the same rolling-row/careful-iteration-order techniques discussed for 0/1 Knapsack) — and exactly like Knapsack, this is PSEUDO-polynomial because T contributes to the running time based on its VALUE, not the number of BITS needed to represent it (which would be Θ(log T)); if T is specified as an extremely large number requiring only a modest number of bits to encode (e.g., a 128-bit integer representing an astronomically large target), the Θ(nT) algorithm becomes impractically slow despite the INPUT SIZE (in bits) being small, which is the technical definition of pseudo-polynomial time and directly relates to subset-sum's status as NP-complete in the weak sense. (B) fabricates an incorrect, overly optimistic polynomial time bound; there's no known algorithm achieving Θ(n log T) for exact subset-sum in general (that would imply a truly polynomial algorithm for an NP-complete problem, which is not known/believed to exist). (C) describes the naive brute-force approach (trying all subsets), which is exactly what the pseudo-polynomial DP AVOIDS via exploiting overlapping subproblems — conflating the two approaches misses the entire point of using DP here. (D) incorrectly claims independence from T for the TIME complexity; T directly and substantially affects both time and space in this DP formulation, since the table's width scales with T.

Q81. Correct Answer: A — With weights, a job with a very high weight but a slightly later finish time might be far more valuable to include than several lower-weight jobs with earlier finish times, so greedily prioritizing EARLIEST FINISH alone (ignoring weight entirely) can lead to a SUBOPTIMAL total weight, whereas the correct DP approach (sorting by finish time, then using p(j) = the last job compatible with j, with recurrence $OPT(j)=\max(OPT(j-1), w_v+OPT(p(j)))$ correctly considers BOTH options (include or exclude each job) at every step

The unweighted interval scheduling problem is a classic case where greedy (earliest finish time) provably achieves the optimal MAXIMUM NUMBER of non-overlapping jobs, but once JOB WEIGHTS (values) are introduced, this greedy strategy is no longer guaranteed optimal, since it's entirely possible for a single high-weight job to be worth MORE than several lower-weight jobs combined, even if greedy would have selected those several smaller jobs first due to their earlier finish times — the correct approach uses DP: after sorting jobs by finish time and precomputing p(j) (the index of the latest job that doesn't conflict with job j), the recurrence $OPT(j) = \max(OPT(j-1), w_v+OPT(p(j)))$ correctly considers, for EACH job in sorted order, both the option of EXCLUDING it (inheriting $OPT(j-1)$) and INCLUDING it (adding its weight to the best compatible solution before it, $OPT(p(j))$), guaranteeing the true optimal total weight is found by exhaustively (but efficiently, via DP) considering all relevant combinations. (B) is false and represents a serious failure to recognize that weight fundamentally changes the OPTIMALITY criterion away from simple job COUNT, breaking the guarantees that made the unweighted greedy approach correct. (C) is false; this is one of the most standard and classic DP problems taught specifically to illustrate how DP EXTENDS a simpler greedy-solvable problem to handle a more general (weighted) version — it absolutely does NOT require abandoning DP for integer linear programming. (D) is deeply incorrect; weights fundamentally change WHICH SPECIFIC set of jobs is optimal to select, not merely how the final numeric total is computed — the actual selected job set can be entirely different once weights are considered.

Q82. Correct Answer: A — Job j is included if and only if w_Y+OPT(p(j)) > OPT(j−1) (strictly better to include it, given the best achievable using only jobs compatible with j); p(j) is the LARGEST index i<j such that job i's finish time is ≤ job j's start time (i.e., the latest job that doesn't overlap/conflict with job j)

The recurrence's max() directly encodes the fundamental DP decision at each job j: EXCLUDE job j (yielding OPT(j−1), the best solution considering only the first j−1 jobs) versus INCLUDE job j (yielding w_Y PLUS the best solution among jobs compatible with j, which is OPT(p(j)) since p(j) is defined as the latest-indexed job that finishes before j starts, i.e., doesn't conflict with j) — whichever term is larger determines the optimal choice for that specific subproblem, and p(j) is precisely this 'latest non-conflicting predecessor' index, typically precomputed via binary search (since jobs are sorted by finish time) in O(log n) per job, contributing to an overall O(n log n) algorithm. (B) is a naive and incorrect greedy-style rule that ignores the actual comparison needed; a job should NOT be automatically included just because its weight is positive — it might conflict with an even MORE valuable combination of other jobs. (C) is a fabricated and incorrect definition; p(j) is specifically about finding the ONE latest compatible predecessor job (a single index), not a count of conflicting jobs. (D) is absurd and irrelevant; this is a standard DETERMINISTIC DP algorithm with a well-defined, provably correct decision rule at each step, with no randomization involved.

Q83. Correct Answer: A — Checking whether an arbitrary substring is a palindrome naively takes O(length) time, and doing this check REPEATEDLY inside the minimum-cuts recurrence (without precomputation) would multiply the overall complexity unnecessarily; precomputing a 2D boolean table isPalin[i][j] in O(n²) time upfront allows each subsequent palindrome-check during the cuts-DP to be an O(1) lookup instead, keeping the overall complexity at O(n²) instead of a worse O(n³)

This reflects a genuinely important DP OPTIMIZATION TECHNIQUE (precomputation/preprocessing to avoid redundant repeated sub-computations): if you tried to check palindrome status NAIVELY (via a direct character-by-character comparison, itself an O(length) operation) EVERY time it's needed within the minimum-cuts DP recurrence (which itself considers O(n²) substring possibilities, and for EACH one might need a palindrome check), you'd get an overall complexity multiplied by this extra factor, potentially reaching O(n³); by first precomputing a table isPalin[i][j] (indicating whether the substring from i to j is a palindrome) using ITS OWN efficient DP recurrence (isPalin[i][j] = (s[i]==s[j]) AND isPalin[i+1][j−1], itself just O(n²) states with O(1) work each), all SUBSEQUENT lookups during the cuts-computation phase become O(1), keeping the combined two-phase approach at a better overall O(n²) time complexity — this pattern (precompute one DP table, then use it to accelerate a second DP) is a common and valuable technique across many DP problems. (B) drastically understates the significance; this genuinely affects the TIME COMPLEXITY (potentially O(n²) versus O(n³)), not merely code organization/style. (C) is false; there is no logical contradiction preventing a single-phase approach — it's simply LESS EFFICIENT (or would require inlining an O(n) check at every step, degrading complexity), not impossible. (D) is incorrect and misses the crucial DEPENDENCY between the two phases; the cuts-DP explicitly RELIES ON the palindrome-check results as a necessary INPUT/subroutine, they are not merely coincidentally related problems sharing input.

Q84. Correct Answer: A — Substrings of length 1 (i==j) are trivially palindromes (isPalin[i][i]=True) and substrings of length 2 (j==i+1) require checking ONLY s[i]==s[j] (since isPalin[i+1][j-1] would reference an INVALID range where i+1>j−1, i.e., an empty or nonsensical range); without handling these separately, the formula would either access out-of-bounds/undefined table entries or incorrectly require satisfying a condition on a non-existent inner substring

The general recurrence isPalin[i][j]=(s[i]==s[j]) AND isPalin[i+1][j−1] implicitly assumes a well-defined 'inner' substring from i+1 to j−1 exists to check — but for LENGTH-1 substrings (i==j), there's no meaningful comparison to make (a single character is trivially a palindrome), and for LENGTH-2 substrings (j=i+1), the 'inner' range i+1 to j−1 becomes i+1 to i, which is an EMPTY (and thus invalid/out-of-bounds if not handled) range since i+1>i; these cases must be explicitly special-cased (isPalin[i][i]=True always, and isPalin[i][i+1]=(s[i]==s[i+1]) directly, without recursing into a nonexistent inner substring) before the general formula can be safely applied for longer substrings (length≥3), where the inner range i+1 to j−1 is guaranteed to be valid and well-defined. (B) is factually incorrect and represents a failure to recognize a genuinely necessary edge case; without this special handling, table lookups for these short lengths would be undefined/incorrect. (C) is incomplete; BOTH length-1 AND length-2 cases require special handling, not just one of them, since both involve degenerate or non-existent 'inner' substring references. (D) is an overly simplistic and INCORRECT alternative that discards the genuinely necessary recursive structure needed for longer palindromes (checking only the endpoints, without the AND isPalin[i+1][j-1] condition, would incorrectly classify strings like 'abca' as palindromic based on endpoints alone, when the middle characters must also form a palindrome).

Q85. Correct Answer: A — dp[k−1][x−1] represents the WORST CASE if the egg BREAKS when dropped from floor x (using one fewer egg, searching the x−1 floors below); dp[k][n−x] represents the WORST CASE if the egg SURVIVES (same number of eggs, searching the n−x floors above); MAX is used because you must plan for the WORST-CASE outcome of the drop (you don't control whether it breaks or survives, so you must guarantee success regardless of which happens)

This recurrence directly encodes the two possible OUTCOMES of dropping an egg from floor x: either the egg BREAKS (in which case you've learned the critical floor is BELOW x, and must continue searching the x−1 floors below using one FEWER egg, since one just broke — giving dp[k−1][x−1]), or the egg SURVIVES (in which case the critical floor is AT or ABOVE x, and you continue searching the remaining n−x floors above, still with all k eggs since none broke — giving dp[k][n−x]); since you cannot control or predict which outcome occurs on any given drop, you must plan for the WORST CASE (whichever outcome requires MORE additional trials), hence taking the MAX of the two — and then, since you get to CHOOSE which floor x to drop from, you take the MIN over all possible choices of x to find the best (minimum worst-case) strategy, giving the full recurrence's outer min-over-x combined with the inner max-of-outcomes structure. (B) is factually incorrect; you don't get to assume optimistic outcomes — the adversarial/worst-case framing of the problem (guaranteeing success regardless of when the egg breaks) is exactly why MAX (not MIN) is used for the two branch outcomes. (C) is false in general; dp[k−1][x−1] and dp[k][n−x] are generally NOT equal (they depend on different remaining floor counts and different numbers of remaining eggs), and the specific VALUE of x is deliberately chosen (via the outer min) partly to balance/minimize this max, which would be meaningless if the two terms were always trivially equal. (D) is a fabricated and nonsensical interpretation with no connection to the actual mechanics of the egg-drop problem, which involves a SINGLE drop's binary outcome (break or survive), not two separate drops from the same floor.

Q86. Correct Answer: A — Direct implementation: O(k·n²) time, due to trying every floor x (O(n) choices) for each of the O(kn) states; an alternative formulation using dp[k][trials] = MAXIMUM floors distinguishable with k eggs and a given number of trials achieves O(kn) or better, by reframing the problem to avoid the inner search over x entirely

The direct recurrence, as stated, has O(kn) distinct (k,n) states, and for EACH state, must try up to O(n) different floor choices x to find the minimizing option, giving O(k·n²) total time (this can be improved to O(kn log n) using binary search on x, since dp[k−1][x−1] increases and dp[k][n−x] decreases as x increases, making the max of the two roughly unimodal/searchable) — but a cleverer REFRAMING of the DP state entirely avoids this inner search: instead of asking 'minimum trials for n floors,' define dp[k][t] = the MAXIMUM number of floors distinguishable using k eggs and AT MOST t trials, with the recurrence dp[k][t] = dp[k−1][t−1] + dp[k][t−1] + 1 (combining the floors coverable if the egg breaks, if it survives, plus the current floor itself), which requires only O(kn) (or O(k· the actual max needed trials, often much smaller than n) states with O(1) work each, then finding the smallest t such that dp[k][t]≥n, achieving a SIGNIFICANTLY better overall complexity — this exemplifies a broader and important DP technique of REFRAMING what the DP state represents to expose a more efficient recurrence. (B) is false; the alternative formulation provides a genuine and substantial complexity improvement, not a negligible difference. (C) drastically and incorrectly underestimates the direct approach's complexity, ignoring the necessary inner search over floor choices entirely. (D) is false and reflects a limited view of DP problem-solving; REFRAMING what the DP state represents (here, switching from 'floors→trials' to 'trials→max floors') is a genuinely powerful and commonly used technique across many DP problems, not a nonexistent option.

Q87. Correct Answer: A — Dijkstra's algorithm makes an irrevocable GREEDY choice at each step (permanently finalizing the shortest distance to the closest unvisited vertex, without reconsidering that choice later) rather than exploring/combining multiple possible subproblem solutions and choosing the best AFTER solving them, which is the hallmark DP approach; Dijkstra works correctly as greedy specifically because non-negative edge weights guarantee that once a vertex's shortest distance is finalized, it can never be improved by a LATER-discovered path

This tests the nuanced relationship between greedy algorithms and DP: BOTH rely on optimal substructure, but they differ in HOW they exploit it — DP-style approaches (like Bellman-Ford, which handles negative edges) explore MULTIPLE possible ways to reach each vertex and relax/update distances repeatedly as better paths are discovered, essentially computing and comparing several subproblem solutions; Dijkstra's, by contrast, makes a single GREEDY, IRREVOCABLE commitment at each step (permanently finalizing the shortest-known distance to whichever unvisited vertex currently has the smallest tentative distance), never reconsidering that choice — this greedy strategy is PROVABLY correct specifically because non-negative edge weights guarantee that no future-discovered path could ever provide a SHORTER route to an already-finalized vertex (a path through unvisited vertices can only ADD non-negative weight, never reduce total distance), which is precisely the 'greedy-choice property' that justifies never revisiting a decision. (B) is false; shortest-path problems (including those Dijkstra's solves) absolutely DO exhibit optimal substructure (a shortest path's sub-paths are themselves shortest paths) — that's not what distinguishes greedy from DP here. (C) dismisses a genuinely meaningful and well-established algorithmic distinction (greedy vs. DP) that CLRS and the broader algorithms community carefully and consistently draws, based on precisely the mechanism described in option A. (D) is factually incorrect; while greedy and DP are related (both exploit optimal substructure), Dijkstra's specific ALGORITHM STRUCTURE (irrevocable, single-choice-per-step) is the defining characteristic of greedy algorithms, not DP's characteristic approach of exploring/comparing multiple subproblem solutions.

Q88. Correct Answer: A — Any shortest path in a graph with V vertices (and no negative cycles) contains AT MOST |V|−1 edges (since a simple path can visit each of the V vertices at most once, using at most V−1 edges to connect them); each full relaxation PASS guarantees correctly computing all shortest paths using AT MOST one additional edge compared to the previous pass, so |V|−1 passes suffice to correctly propagate information across the longest possible shortest path

This is a precise, provable mathematical guarantee (proven via induction in CLRS): since a SIMPLE shortest path in any graph (with no negative-weight cycles, which would otherwise make 'shortest path' undefined by allowing arbitrarily negative totals via repeated cycling) can use AT MOST |V|−1 edges (because a simple path visits each of the |V| vertices at most once, and connecting k vertices in sequence requires at most k−1 edges), Bellman-Ford's relaxation process is proven (by induction on the number of passes) to correctly compute the shortest distance using paths of AT MOST i edges after the i-th pass — so after |V|−1 complete passes, ALL shortest paths (which use at most |V|−1 edges, per the argument above) are guaranteed to have been correctly computed, making further passes UNNECESSARY (though a |V|-th pass is often added specifically as a NEGATIVE-CYCLE DETECTION mechanism: if distances STILL improve on this extra pass, a negative cycle must exist, since otherwise convergence would have already occurred). (B) is false; this is a precisely PROVEN bound via a rigorous inductive argument about the maximum possible length of simple shortest paths, not an empirical heuristic. (C) is factually incorrect; the standard, provably correct formulation is |V|−1 passes, not |E| passes — conflating these two different graph-size parameters reflects a misunderstanding of the actual complexity/correctness argument. (D) is a fabricated and incorrect mechanism; the algorithm doesn't 'remove' vertices from consideration in this sense — it repeatedly relaxes ALL edges across the ENTIRE graph in each pass, and the |V|−1 count relates to maximum SIMPLE PATH LENGTH, not vertex removal.

Q89. Correct Answer: A — O(n³) in a naive implementation, because there are O(n) values of i, each requiring trying O(n) values of j, and EACH such (i,j) pair requires an O(n) substring extraction (e.g., s[j:i] in many languages creates a NEW string of length up to n) before the O(1) hash lookup can even occur — this substring-copying cost, often overlooked, can dominate the complexity

This tests a subtle but practically important complexity trap: while the DP STATE SPACE is O(n) (positions i) and the TRANSITION considers O(n) choices of j per state (giving a naively expected $O(n^2)$ DP structure), many implementations OVERLOOK that extracting a substring s[j:i] in languages where strings are immutable and substring operations COPY the underlying characters (common in Python, Java, and others, unless specific optimizations/views are used) itself costs O(i−j) time, which can be up to O(n) in the worst case — multiplying this into the $O(n^2)$ DP structure gives a WORST-CASE $O(n^3)$ TOTAL complexity if this substring-extraction cost isn't carefully accounted for or optimized away (e.g., via using string hashing, or a Trie-based dictionary check that avoids full substring materialization). This question specifically tests whether students recognize that 'the dictionary lookup is O(1)' does NOT automatically mean 'the whole operation involving that lookup is O(1)' — the cost of PREPARING the input to the lookup (substring extraction) must also be counted. (B) is factually wrong about substring extraction being universally O(1); this depends heavily on the specific language/data structure (e.g., some specialized string-view or rope-based implementations CAN achieve O(1) or O(log n) substring access, but this is NOT the default/universal case, especially for standard string types in common languages). (C) drastically underestimates the complexity by ignoring BOTH the O(n) choices of j per state AND the substring extraction cost entirely. (D) incorrectly claims Word Break has no DP structure at all, contradicting the explicitly given DP recurrence in the question itself, and fabricates an unrelated exponential complexity that doesn't match the actual (polynomial, if properly analyzed) nature of this classic DP problem.

Q90. Correct Answer: A — Because multiplying by a NEGATIVE number can turn the current MINIMUM (most negative) product into the new MAXIMUM (by flipping its sign to positive), so failing to track the running minimum means potentially missing the optimal solution when a negative number appears; this is a fundamental structural difference from the SUM version, where multiplying (which doesn't apply to sums) never causes this sign-flipping interaction

This is THE defining insight that distinguishes Maximum Product Subarray from the simpler Maximum SUM Subarray (Kadane's algorithm): because MULTIPLICATION (unlike addition) can flip signs, a sequence like [..., large_negative, negative_number] can produce a LARGE POSITIVE product when the negative_number multiplies with the large_negative value — meaning the BEST maximum-product-ending-here might actually come from extending the WORST (most negative) product-ending-at-the-previous-position, not the best one; therefore the correct recurrence must maintain BOTH maxEnd[i] and minEnd[i] at every step, computing maxEnd[i] = max(A[i], A[i]×maxEnd[i−1], A[i]×minEnd[i−1]) (and similarly minEnd[i] considering the same three candidates but taking the min), since you genuinely cannot know in advance which of maxEnd[i−1] or minEnd[i−1], when multiplied by a potentially-negative A[i], will yield the better result — this sign-flipping phenomenon has NO analog in the pure-SUM version, where adding a negative number simply decreases the value without ever converting a 'worst' running sum into a 'best' one via multiplication's sign-inversion property. (B) is false and represents a serious misunderstanding; omitting the minimum tracking can cause the algorithm to produce an INCORRECT (too low) answer, not merely a slower one — specific test cases with negative numbers would fail without this tracking. (C) is incorrect; the minimum is used ACTIVELY DURING the recurrence's computation at every step (as a potential source for the NEXT maximum, via sign-flipping), not merely reported passively at the end. (D) is factually wrong; products in this problem absolutely CAN be negative (whenever an odd number of negative factors are multiplied together), and the entire point of tracking the minimum relates directly to this sign consideration, not merely finding 'the smallest positive value.'

Q91. Correct Answer: A — '.' simply means 'match any single character,' requiring a straightforward diagonal-style dependency (dp[i][j] depends on dp[i−1][j−1]); '*' means 'zero or more of the PRECEDING character,' requiring the recurrence to consider BOTH the possibility of using ZERO occurrences (skip the preceding-char+* pattern entirely, dp[i][j−2]) AND the possibility of using ONE OR MORE occurrences (which requires the preceding pattern character to match the current text character, then recursively checking dp[i−1][j], allowing the '*' to consume MULTIPLE text characters via this same-column recursive reference)

The '.' character has simple, LOCAL semantics (match exactly one arbitrary character), fitting naturally into a straightforward diagonal DP dependency similar to basic string matching; the '*' character, by contrast, has fundamentally more complex semantics tied to REPETITION of the PRECEDING pattern element, requiring the recurrence to branch into two genuinely different cases: (1) treat the preceding-element+* combination as matching ZERO times (effectively SKIPPING both characters in the pattern, referencing dp[i][j−2]), or (2) treat it as matching ONE OR MORE times, which requires the preceding pattern element to successfully match the CURRENT text character (via '.' or exact match), and then recursively check dp[i−1][j] (NOT dp[i−1][j−1] or dp[i][j-2]) — crucially staying at the SAME pattern position j (since '*' can be reused to match ADDITIONAL text characters), which is what allows this recurrence to correctly handle patterns like 'a*' matching 'aaaa' (repeatedly consuming text characters while remaining at the same point in the pattern). This dual-case branching (zero vs. one-or-more, with the one-or-more case having a same-column self-referential dependency) is precisely what makes '*' handling substantially trickier than '.' handling. (B) is false and understates the genuine additional complexity that '*' introduces via its repetition semantics, which '.' entirely lacks. (C) is factually backwards and incorrect; '*' does NOT 'always match everything' — it specifically means repetition of the PRECEDING element, requiring careful conditional handling, not a trivial universal match. (D) is false; DP-based regex matching (for this specific restricted subset of regex features: '.' and '*') IS a well-established, correct, and standard technique — it does not require abandoning DP for NFA simulation, though NFA-based approaches are an alternative valid technique for MORE GENERAL regex features.

Q92. Correct Answer: A — Memoized recursion IS a form of dynamic programming (specifically, the 'top-down' variant), maintaining the natural recursive structure of the problem while adding a cache/table to avoid recomputing already-solved subproblems, in contrast to 'bottom-up' DP, which iteratively fills the table in a carefully chosen dependency-respecting order without explicit recursive calls

CLRS explicitly presents memoization (top-down) and tabulation (bottom-up) as the TWO standard ways of implementing dynamic programming, both exploiting the same underlying optimal-substructure and overlapping-subproblems properties — memoized recursion preserves the problem's natural top-down recursive formulation but adds a cache (memo table) checked at the start of each call to avoid redoing work for previously-solved subproblems, while bottom-up tabulation instead builds the solution iteratively from smaller subproblems upward, in an explicitly chosen order that guarantees dependencies are satisfied before they're needed; both are legitimate, standard DP IMPLEMENTATION STRATEGIES, not distinct paradigms. (B) is false and represents a fundamental misunderstanding; memoized recursion is not merely 'similar' to DP by coincidence — it IS one of the two standard, textbook-recognized ways of implementing dynamic programming. (C) is factually incorrect; PLENTY of DP algorithms are implemented via memoized recursion, which explicitly and necessarily uses recursion — DP is defined by the underlying algorithmic STRATEGY (exploiting optimal substructure/overlapping subproblems), not by whether the implementation happens to be iterative or recursive. (D) is an overgeneralization not supported by CLRS or general algorithmic practice; while top-down CAN sometimes save work by skipping genuinely unnecessary subproblems, bottom-up often has LOWER constant-factor overhead (no recursive call/stack management cost) and CLRS does not make a blanket claim that one is always strictly faster — the choice often depends on the specific problem and implementation context.

Floyd-Warshall (All-Pairs Shortest Paths) — Practice Questions

Q93. In the Floyd-Warshall algorithm, $d_{ij}^{(k)}$ is defined as the weight of a shortest path from i to j with all INTERMEDIATE vertices restricted to the set {1,2,...,k}. What is the base case $d_{ij}^{(0)}$, and why?

A. $d_{ij}^{(0)} = w_{ij}$ (the direct edge weight, or ∞ if no edge exists), because with k=0 (no intermediate vertices allowed at all), a path from i to j can have AT MOST one edge — i.e., it must be the direct edge itself, if one exists

B. $d_{ij}^{(0)} = 0$ always, since k=0 means the path hasn't started yet

C. $d_{ij}^{(0)} = \infty$ always, since zero intermediate vertices means no path can possibly exist

D. $d_{ij}^{(0)}$ is undefined and must be computed via the general recurrence even for k=0

Q94. What is the Floyd-Warshall recurrence for $d_{ij}^{(k)}$ when k≥1, and what does each of the two terms inside the min() represent?

A. $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$; the first term represents NOT using vertex k as an intermediate vertex (reusing the best path found with only {1,...,k−1} available), the second represents USING vertex k as an intermediate vertex (combining the best i-to-k and k-to-j paths, each restricted to {1,...,k−1} as intermediates)

B. $d_{ij}^{(k)} = d_{ik}^{(k)} + d_{kj}^{(k)}$ always, unconditionally using vertex k as an intermediate on every path

C. $d_{ij}^{(k)} = \max(d_{ij}^{(k-1)}, d_{ik}^{(k-1)}+d_{kj}^{(k-1)})$, taking the MAXIMUM of the two options rather than minimum

D. $d_{ij}^{(k)} = w_{ij}$ unconditionally, ignoring all previously computed values entirely

Q95. The bottom-up FLOYD-WARSHALL procedure computes $D^{(k)}$ matrices for k=1 to n using a TRIPLY NESTED loop (over k, then i, then j). Why must the OUTERMOST loop be over k specifically, rather than i or j?

A. Because computing $d_{ij}^{(k)}$ for ANY pair (i,j) requires $d_{ik}^{(k-1)}$ and $d_{kj}^{(k-1)}$ to ALREADY be fully finalized from the PREVIOUS value of k; making k the outermost loop guarantees that the ENTIRE $D^{(k-1)}$ matrix is complete before any $D^{(k)}$ entries are computed, correctly respecting this dependency across ALL (i,j) pairs simultaneously

B. The loop order doesn't actually matter for correctness; any of the six possible orderings of i, j, k produces the same correct result

C. k must be outermost purely for performance reasons (cache locality), with no correctness implications

D. i must always be the outermost loop for any triply-nested dynamic programming algorithm, as a general rule

Q96. What is the time complexity of the Floyd-Warshall algorithm for a graph with n vertices, and how does this compare structurally to running Bellman-Ford from EVERY single vertex as an alternative all-pairs shortest-path approach?

A. Floyd-Warshall: Θ(n³), from the triply-nested loop with O(1) work per innermost iteration; running Bellman-Ford (O(VE)=O(n³) for dense graphs) from every one of the n vertices gives O(n·VE) = O(n&sup4;) for dense graphs, making Floyd-Warshall asymptotically BETTER for dense graphs specifically

B. Floyd-Warshall is Θ(n²), always strictly faster than any Bellman-Ford-based approach regardless of graph density

C. Floyd-Warshall is Θ(n⁴), making it always WORSE than running Bellman-Ford from every vertex

D. Both approaches have identical Θ(n³) complexity in all cases, so the choice between them never matters

Q97. Why does the Floyd-Warshall algorithm, as presented in CLRS, work correctly even in the presence of NEGATIVE-weight edges, but explicitly requires the ABSENCE of negative-weight CYCLES?

A. The algorithm's correctness proof relies only on the existence of well-defined SHORTEST (simple) paths between vertex pairs, which negative EDGES alone don't prevent; but a negative-weight CYCLE would allow a 'path' to repeatedly loop through the cycle, decreasing total weight WITHOUT BOUND, making 'shortest path' UNDEFINED (no finite minimum exists) for any pair of vertices reachable via that cycle

B. Negative-weight edges are actually NOT permitted by Floyd-Warshall at all; the algorithm requires strictly non-negative weights, identical to Dijkstra's requirement

C. Negative-weight cycles are permitted as long as there are an EVEN number of negative edges within the cycle

D. The algorithm handles negative cycles correctly by simply ignoring them and returning the best result found before encountering the cycle

Q98. Post-hoc negative cycle DETECTION using Floyd-Warshall's output: after running the standard algorithm, how can you determine whether the input graph contained a negative-weight cycle, purely by examining the FINAL $D^{(n)}$ matrix?

A. Check the DIAGONAL entries $d_{ii}^{(n)}$ for any i; if ANY diagonal entry is NEGATIVE ($d_{ii}^{(n)} < 0$), this indicates vertex i lies on a negative-weight cycle, since a legitimate shortest 'path' from a vertex to ITSELF (with no cycling) should have weight exactly 0, and a negative value can only arise if the algorithm found a way to 'return' to i via a net-negative-weight cycle

B. Check if ANY off-diagonal entry equals exactly 0; this always indicates a negative cycle exists somewhere in the graph

C. Negative cycles can never be detected from the OUTPUT matrix alone; a separate, completely different algorithm must be run from scratch

D. Check if the matrix is symmetric; asymmetry in the final D matrix always indicates a negative cycle

Q99. For the TRANSITIVE CLOSURE computation via Floyd-Warshall-style DP (using boolean logical operations instead of arithmetic min/+), the recurrence is $t_{ij}^{(k)} = t_{ij}^{(k-1)}$ OR ($t_{ik}^{(k-1)}$ AND $t_{kj}^{(k-1)}$). What is the PRECISE structural correspondence between this and the standard Floyd-Warshall recurrence?

A. OR directly replaces MIN (both select the 'better'/broader of two options: minimum weight, or logical truth if EITHER path-existence option holds), and AND directly replaces + (both COMBINE two sub-results into a single joint result: summing weights along a path, or requiring BOTH sub-paths to exist for the combined path to exist)

B. The two recurrences are entirely unrelated; transitive closure requires a fundamentally different, unrelated algorithmic approach with no structural connection to shortest paths

C. OR replaces +, and AND replaces MIN, which is the OPPOSITE (swapped) correspondence from the correct one

D. Transitive closure cannot be computed via any Floyd-Warshall-style DP approach; it requires a completely different graph traversal method like repeated BFS/DFS from every vertex

Q100. What is the practical ADVANTAGE of the boolean-based TRANSITIVE-CLOSURE algorithm over simply running standard Floyd-Warshall with all edge weights set to 1 and checking if $d_{ij} < n$ (as an alternative way to compute transitive closure)?

A. The direct boolean approach can be more space- and potentially time-efficient in practice, since boolean (single-bit) values typically require less STORAGE than full integer/numeric weights, and on many computer architectures, logical operations (AND/OR) on single bits execute FASTER than arithmetic operations (addition, comparison) on full integer words

B. There is no practical advantage whatsoever; both approaches always have identical performance characteristics in every conceivable implementation

C. The boolean approach is asymptotically faster, achieving $O(n^2)$ instead of $O(n^3)$

D. The weight-based approach (setting weights to 1) is mathematically INCORRECT and doesn't actually compute transitive closure at all, making the boolean approach the only valid option

Q101. In the predecessor matrix Π construction for Floyd-Warshall (used to reconstruct actual shortest PATHS, not just their weights), the recurrence for $\pi_{ij}^{(k)}$ when $k \geq 1$ depends on comparing $d_{ij}^{(k-1)}$ to $d_{ik}^{(k-1)}+d_{kj}^{(k-1)}$. What does $\pi_{ij}^{(k)}$ represent, and how is it updated in each of the two cases?

A. $\pi_{ij}^{(k)}$ is the PREDECESSOR of vertex j on a shortest path from i, restricted to intermediates in {1,...,k}; if k IS used as an intermediate (i.e., $d_{ij}^{(k-1)} > d_{ik}^{(k-1)}+d_{kj}^{(k-1)}$), then $\pi_{ij}^{(k)} = \pi_{kj}^{(k-1)}$ (inheriting the predecessor from the k-to-j sub-path, since j's immediate predecessor on the FULL path is the same as its predecessor on just the final leg from k to j); otherwise, $\pi_{ij}^{(k)} = \pi_{ij}^{(k-1)}$ (predecessor unchanged, since the best path doesn't route through k)

B. $\pi_{ij}^{(k)}$ always equals k directly, regardless of whether k actually lies on the optimal path

C. $\pi_{ij}^{(k)}$ represents the TOTAL NUMBER of edges in the shortest path, not an actual vertex reference

D. $\pi_{ij}^{(k)}$ is computed completely independently of the D matrix, using an entirely separate recurrence with no relationship to the distance computations

Q102. Given the base case $\pi_{ij}^{(0)}$ = NIL if i=j or $w_{ij}=\infty$, and $\pi_{ij}^{(0)}$ = i if i≠j and $w_{ij}<\infty$, what is the intuition behind these TWO distinct sub-cases?

A. If i=j (trivial self-path) or there's NO direct edge ($w_{ij}=\infty$), there's no meaningful predecessor to record, so NIL correctly signals 'no path/predecessor exists at this base level'; if a DIRECT edge from i to j exists (i≠j and $w_{ij}<\infty$), then the immediate predecessor of j on this single-edge path is simply i itself (the path is just the one edge i→j, so j's predecessor is trivially i)

B. Both cases are set to NIL universally, since predecessors are always only defined for k≥1, never for the base case k=0

C. The value i (in the second case) is a labeling ERROR and should actually be j, referring to the destination vertex instead of the source

D. These base cases are entirely arbitrary/placeholder values with no real semantic meaning, used only to prevent runtime errors during table initialization

Q103. Consider tracing Floyd-Warshall by hand on a SPECIFIC small graph: vertices {1,2,3}, edges with weights $w_{12}=3$, $w_{23}=-2$, $w_{31}=2$, and all other non-diagonal entries ∞ (no direct edge), diagonal entries 0. After the k=1 iteration (allowing vertex 1 as an intermediate), does $d_{23}^{(1)}$ change from its base value $d_{23}^{(0)}$?

A. No, $d_{23}^{(1)}$ remains −2 (unchanged from $d_{23}^{(0)}=-2$, the direct edge weight), because checking the alternative path 2→1→3 would require $d_{21}^{(0)}+d_{13}^{(0)}$, and $d_{21}^{(0)}=\infty$ (no direct edge from 2 to 1 exists in this graph), so this alternative path is not viable, leaving the direct edge as the best (and only) option

B. Yes, $d_{23}^{(1)}$ becomes 5, by incorrectly using $w_{12}+w_{31}$ instead of properly checking the path THROUGH vertex 1 in the correct direction (2 to 1, then 1 to 3)

C. Yes, $d_{23}^{(1)}$ becomes 0, by incorrectly assuming all shortest paths must eventually reduce to 0 after passing through any intermediate vertex

D. This cannot be determined without additional information about the graph beyond what's given

Q104. For the same graph as the previous question (vertices {1,2,3}, $w_{12}=3$, $w_{23}=-2$, $w_{31}=2$), what is $d_{12}^{(3)}$ (the final shortest distance from vertex 1 to vertex 2, using ALL vertices as potential intermediates)?

A. 3, unchanged from the direct edge $d_{12}^{(0)}=3$, since NO alternative path from 1 to 2 (via 3 as an intermediate, i.e., 1→3→2, requiring an edge FROM 3 TO 2, which is not given/is ∞) provides an improvement

B. 1, computed incorrectly by summing all three given edge weights and dividing by 3

C. 0, since the algorithm always eventually converges all distances to 0 once every vertex is used as an intermediate

D. −2, incorrectly using the value of w_{23} (a completely different edge) instead of correctly computing d_{12}

Q105. What is the SPACE complexity of the STRAIGHTFORWARD Floyd-Warshall implementation that explicitly stores ALL n+1 matrices $D^{(0)}$, $D^{(1)}$, ..., $D^{(n)}$ (as literally described by the superscripted recurrence), and how can this be reduced?

A. $\Theta(n^3)$ space for storing all n+1 separate n×n matrices; this can be reduced to $\Theta(n^2)$ by OBSERVING that the recurrence for $d_{ij}^{(k)}$ only ever needs values from the IMMEDIATELY PRECEDING matrix $D^{(k-1)}$, so a SINGLE matrix can be reused/overwritten in place (dropping the superscripts entirely) across iterations, as explicitly discussed in a CLRS exercise (23.2-4)

B. The space complexity is always exactly $\Theta(n^2)$, with no possibility of using more space even in a naive implementation, since matrices are inherently n×n regardless of implementation choice

C. Space complexity cannot be reduced below $\Theta(n^3)$ under any circumstances, since all n+1 matrices are mathematically necessary and cannot be discarded

D. The space complexity is $\Theta(1)$, since Floyd-Warshall only tracks a single running total, not any matrices

Q106. A buggy implementation of Floyd-Warshall INITIALIZES the diagonal entries d_{ii} to ∞ instead of the CORRECT value of 0. What is the most likely CONSEQUENCE of this bug on the algorithm's final output?

A. The algorithm may incorrectly compute LONGER (or entirely incorrect/infinite) shortest paths for various vertex pairs, since paths that legitimately should be able to 'pass through' a vertex back to a point that connects back to itself (weight 0 contribution) would instead be treated as requiring an INFINITE-cost detour, potentially causing valid, genuinely shorter paths to be missed or miscalculated during the min() comparisons throughout the algorithm's execution

B. This bug has absolutely no effect on the algorithm's correctness, since diagonal entries are never actually used or referenced during the computation

C. The algorithm will crash immediately with a division-by-zero-style error due to the infinite diagonal values

D. This bug only affects the SPACE complexity of the algorithm, not the correctness of the computed distances

Q107. Consider a DENSE graph (E close to V^2) versus a SPARSE graph (E close to V) for the ALL-PAIRS shortest path problem. Why does Floyd-Warshall's FIXED $\Theta(n^3)$ complexity make it PARTICULARLY well-suited (relatively speaking) for DENSE graphs specifically, compared to running Dijkstra's from every vertex?

A. Running Dijkstra's (with a Fibonacci heap) from every vertex gives $O(V^2 \log V + VE)$ total time; for a DENSE graph where E approaches V^2, this becomes $O(V^3)$, essentially MATCHING Floyd-Warshall's complexity (making Floyd-Warshall's simplicity and ability to handle negative edges an attractive, comparably-efficient choice), whereas for a SPARSE graph where E is much smaller (closer to V), the Dijkstra-from-every-vertex approach becomes substantially FASTER than Floyd-Warshall's FIXED n^3 bound, which doesn't improve/adapt based on sparsity

B. Floyd-Warshall is ALWAYS faster than any Dijkstra-based approach, regardless of graph density, making this comparison moot

C. Dijkstra's algorithm cannot be run multiple times from different sources under any circumstances, making this comparison technically invalid

D. Floyd-Warshall's complexity actually DECREASES for dense graphs specifically, unlike its complexity for sparse graphs

Q108. Which statement correctly explains why Floyd-Warshall is classified as a DYNAMIC PROGRAMMING algorithm specifically, connecting this classification to the recurrence's structure?

A. Floyd-Warshall exhibits BOTH optimal substructure (a shortest path's SUB-PATHS, when decomposed at the highest-numbered intermediate vertex k, are themselves provably shortest paths using only smaller-numbered intermediates, as directly proven via the CLRS optimal-substructure argument) AND overlapping subproblems (the SAME $d_{ik}^{(k-1)}$ and $d_{kj}^{(k-1)}$ values get REUSED across MANY different (i,j) pairs at each iteration of k, making memoization/tabulation via the D matrix genuinely valuable rather than redundant recomputation)

B. Floyd-Warshall is classified as DP purely by historical convention/naming tradition, with no actual structural justification connecting it to the formal DP criteria

C. Floyd-Warshall is NOT actually a DP algorithm; it's more accurately classified as a GREEDY algorithm, similar to Dijkstra's

D. Floyd-Warshall qualifies as DP solely because it uses a MATRIX data structure, and any matrix-based algorithm is automatically classified as DP

Q109. For a graph where Floyd-Warshall is run and it's later discovered that a NEGATIVE-weight cycle exists (violating the algorithm's stated assumptions), what specifically happens to the computed d_{ij} values for vertex pairs (i,j) where BOTH i and j can reach and be reached by that negative cycle?

A. The computed distances for such pairs become MEANINGLESS/INCORRECT (the true 'shortest path' is undefined, being negative infinity in principle, since the cycle could be traversed arbitrarily many times), though the algorithm will still MECHANICALLY produce SOME finite numeric output (not necessarily an explicit error), which is WHY the post-hoc diagonal-checking technique is a valuable, necessary safeguard when negative edges are present and cycle-freedom isn't otherwise guaranteed

B. The algorithm automatically detects the negative cycle DURING execution and immediately halts with a clear, explicit error message, requiring no additional post-processing

C. The computed distances remain PERFECTLY CORRECT and meaningful even with negative cycles present, since Floyd-Warshall's min() operations inherently handle this case correctly with no special consideration needed

D. Negative cycles only affect the PREDECESSOR matrix Π, leaving the DISTANCE matrix D completely unaffected and reliably correct

Q110. A student claims: 'Floyd-Warshall can be trivially parallelized across the OUTER loop (over k) since each iteration is independent, just like the inner (i,j) loops can be parallelized.' Evaluate this specific claim about the OUTER (k) loop's parallelizability.

A. This claim is FALSE; the outer k-loop iterations are FUNDAMENTALLY SEQUENTIAL (NOT independent), since computing $D^{(k)}$ REQUIRES the FULLY COMPLETED $D^{(k-1)}$ matrix as input — you cannot correctly compute results for k=5 before k=4 is finished, unlike the INNER (i,j) pairs at a FIXED k, which genuinely CAN be computed independently/in parallel with each other since they only depend on the (already complete) PREVIOUS iteration's matrix, not on each other

B. This claim is entirely TRUE; all n iterations of the k-loop can be run simultaneously on separate processors with no synchronization needed whatsoever

C. Neither the k-loop NOR the inner (i,j) loops can EVER be parallelized in any implementation of Floyd-Warshall

D. Parallelization of Floyd-Warshall is only theoretically possible but has never been practically implemented or studied in real systems

Q111. What is the CORRECT way to interpret the claim 'Floyd-Warshall solves the SINGLE-PAIR shortest path problem as a special case,' and why is this generally considered a POOR practical choice despite being technically POSSIBLE?

A. Technically, running FULL Floyd-Warshall and then simply extracting ONE specific d_{ij} entry from the final matrix WOULD give the correct single-pair shortest-path answer, but this wastes an enormous amount of computation ($\Theta(n^3)$ total work to answer just ONE query) compared to a dedicated single-source algorithm like Dijkstra's (O(E+VlogV) with a good heap) or even Bellman-Ford (O(VE)), both of which are specifically DESIGNED for this narrower single-source (or even more targeted single-PAIR) query and thus far more EFFICIENT when the ALL-PAIRS information isn't actually needed

B. Floyd-Warshall CANNOT be used for single-pair queries under any circumstances; it strictly requires ALL pairs to be queried simultaneously as an inseparable batch operation

C. Using Floyd-Warshall for a single-pair query is actually the MOST efficient approach in ALL cases, strictly better than any single-source alternative

D. Single-pair shortest path queries are fundamentally UNSOLVABLE by any algorithm; only ALL-PAIRS problems have well-defined solutions

Q112. Consider modifying Floyd-Warshall to find the shortest path with AT MOST K EDGES (not just any number of edges), for some fixed parameter K much smaller than n. Does the standard Floyd-Warshall recurrence, AS GIVEN, directly solve this MODIFIED problem, and why or why not?

A. No, the standard recurrence does NOT directly solve this variant; the standard $d_{ij}^{(k)}$ tracks shortest paths based on which VERTICES are allowed as intermediates, NOT based on the NUMBER OF EDGES used, so a completely DIFFERENT DP formulation (typically tracking BOTH the destination AND the number of edges used so far as separate DP state dimensions, e.g., dp[edges][i][j]) would be needed to correctly enforce an edge-COUNT constraint

B. Yes, the standard recurrence automatically and directly enforces an edge-count limit of exactly K by design, with K implicitly equal to n in the standard formulation

C. This modified problem is mathematically equivalent to the original, unmodified problem in every case, requiring no changes to the recurrence whatsoever

D. The standard algorithm can be trivially adapted by simply stopping the k-loop early after K iterations, which correctly and directly solves the edge-count-limited variant

Q113. For a graph represented with adjacency MATRIX W where $w_{ij}=\infty$ indicates 'no edge,' what happens during Floyd-Warshall's execution if a NAIVE IMPLEMENTATION uses a very large but FINITE sentinel value (e.g., INT_MAX in a language with fixed-width integers) to represent ∞, WITHOUT any overflow-checking, when computing $d_{ik}^{(k-1)}+d_{kj}^{(k-1)}$?

A. If BOTH $d_{ik}^{(k-1)}$ and $d_{kj}^{(k-1)}$ happen to be (or are close to) the large sentinel value simultaneously, their SUM can OVERFLOW the integer type's maximum representable value, WRAPPING AROUND to an unexpectedly SMALL (or even NEGATIVE) number, which could then be INCORRECTLY selected as the 'minimum' in the min() comparison, silently CORRUPTING the algorithm's output with a completely bogus, artificially-small distance value

B. This scenario never actually occurs in practice, since Floyd-Warshall never adds two large sentinel values together under any circumstances

C. Using a large finite sentinel value INSTEAD of true mathematical infinity is always PERFECTLY SAFE with no possible complications in any implementation

D. The overflow, if it occurs, only affects the PREDECESSOR matrix, leaving the DISTANCE matrix completely unaffected

Q114. Which of the following BEST describes the relationship between Floyd-Warshall's algorithm and the concept of MATRIX MULTIPLICATION over the (min,+) SEMIRING (sometimes called the 'min-plus' or 'tropical' semiring), as an alternative theoretical framing of the all-pairs shortest path problem?

A. Computing $D^{(n)}$ can be conceptually framed as repeatedly 'multiplying' the weight matrix W by itself using MIN in place of standard addition's role and PLUS (+) in place of standard multiplication's role (i.e., $(A\otimes B)_{ij}$ = min over k of $(A_{ik}+B_{kj})$, directly analogous to standard matrix multiplication's $(AB)_{ij}$ = sum over k of $(A_{ik}\times B_{kj})$); this reframing connects all-pairs shortest paths to FAST MATRIX MULTIPLICATION techniques, offering an ALTERNATIVE (though not identical in practical implementation) theoretical algorithmic approach distinct from Floyd-Warshall's specific triply-nested DP structure

B. There is no meaningful mathematical connection between Floyd-Warshall and matrix multiplication concepts of any kind

C. Floyd-Warshall IS literally, directly, standard matrix multiplication (using regular +,× operations) applied n times in sequence, with no special semiring reinterpretation needed

D. The (min,+) semiring framing only applies to problems with strictly POSITIVE edge weights, making it entirely inapplicable whenever negative weights (as Floyd-Warshall explicitly permits) are present

Q115. In code-tracing terms: given a Floyd-Warshall implementation with a bug where the innermost update line is WRITTEN AS `d[i][j] = d[i][k] + d[k][j]` (an UNCONDITIONAL assignment, OMITTING the min() comparison against the existing d[i][j] entirely), what is the specific, predictable CONSEQUENCE of this bug?

A. The algorithm will FORCE every single d[i][j] entry to be OVERWRITTEN by the (potentially much WORSE) path THROUGH vertex k, even when the EXISTING, previously-computed value (representing a path NOT using k) was ALREADY better/shorter; this systematically DISCARDS potentially-optimal existing solutions in favor of unconditionally routing through k, generally producing INCORRECT (too large, or nonsensical if involving unreachable/infinite sub-paths) final distances

B. This bug has NO effect on correctness, since d[i][k]+d[k][j] always equals the CORRECT minimum in every case regardless of the min() comparison being omitted

C. This specific bug causes an infinite loop, since the assignment creates a circular self-reference

D. This bug only affects PERFORMANCE (making the algorithm slower), not the CORRECTNESS of the final computed distances

Floyd-Warshall (All-Pairs Shortest Paths) — Answer Key & Explanations

Q93. Correct Answer: A — $d_{ij}^{(0)} = w_{ij}$ (the direct edge weight, or ∞ if no edge exists), because with k=0 (no intermediate vertices allowed at all), a path from i to j can have AT MOST one edge — i.e., it must be the direct edge itself, if one exists

As CLRS directly states, with k=0, a path from i to j whose intermediate vertices are restricted to the empty set {1,...,0}=∅ has NO intermediate vertices at all, meaning it consists of AT MOST ONE EDGE — the direct edge from i to j; hence $d_{ij}^{(0)} = w_{ij}$, the direct edge weight (using the convention that $w_{ij}=\infty$ if no direct edge exists, and $w_{ii}=0$ for the trivial self-loop case). This base case is foundational to the entire recursive definition, serving as the starting point (k=0) before intermediate vertices 1,2,...,n are progressively allowed. (B) is factually wrong; setting all base-case distances to 0 would incorrectly claim every pair of vertices has zero distance regardless of actual edge weights, destroying the algorithm's correctness from the start. (C) is also wrong and overly pessimistic; DIRECT edges (paths with zero intermediate vertices) certainly CAN exist and should be captured, not universally treated as infinite/nonexistent. (D) is false; the base case is explicitly and directly defined by the direct edge weights, not derived from the general min-based recurrence (which specifically applies for k≥1).

Q94. Correct Answer: A — $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$; the first term represents NOT using vertex k as an intermediate vertex (reusing the best path found with only {1,...,k−1} available), the second represents USING vertex k as an intermediate vertex (combining the best i-to-k and k-to-j paths, each restricted to {1,...,k−1} as intermediates)

This is the CENTRAL recurrence of the Floyd-Warshall algorithm, directly grounded in the case analysis of whether vertex k IS or IS NOT an intermediate vertex of the true shortest path from i to j (restricted to intermediates in {1,...,k}): if k is NOT used, the shortest such path is exactly the shortest path with intermediates restricted to {1,...,k−1}, giving $d_{ij}^{(k-1)}$; if k IS used, the path decomposes into a sub-path from i to k and a sub-path from k to j, each of which (crucially, by the optimal substructure argument) must ITSELF be a shortest path using only intermediates from {1,...,k−1} (since k itself, appearing only once as the 'peak'/connector vertex in a SIMPLE path, cannot be an intermediate vertex of either sub-path), giving $d_{ik}^{(k-1)}+d_{kj}^{(k-1)}$; taking the MINIMUM of these two mutually exclusive cases correctly captures the true shortest path once vertex k becomes available as a potential intermediate. (B) incorrectly forces every path to use k, ignoring the case where NOT using k might be better, which is one of the two essential cases the recurrence must consider. (C) uses MAX instead of MIN, completely inverting the optimization goal (shortest PATHS require minimizing weight, not maximizing). (D) discards all recursive structure entirely, ignoring the whole point of building up the solution incrementally as k increases from 0 to n.

Q95. Correct Answer: A — Because computing $d_{ij}^{(k)}$ for ANY pair (i,j) requires $d_{ik}^{(k-1)}$ and $d_{kj}^{(k-1)}$ to ALREADY be fully finalized from the PREVIOUS value of k; making k the outermost loop guarantees that the ENTIRE $D^{(k-1)}$ matrix is complete before any $D^{(k)}$ entries are computed, correctly respecting this dependency across ALL (i,j) pairs simultaneously

This is a critical correctness requirement (and a very commonly tested 'trick' in Floyd-Warshall specifically, since many students ASSUME loop order doesn't matter, unlike simpler doubly-nested DP loops): computing $d_{ij}^{(k)}$ for any (i,j) pair fundamentally requires knowing $d_{ik}^{(k-1)}$ and $d_{kj}^{(k-1)}$ from the PREVIOUS iteration (k−1) — and different (i,j) pairs may depend on DIFFERENT (ik) and (kj) entries, all of which must be from the SAME, fully-completed $D^{(k-1)}$ matrix; making k the OUTERMOST loop ensures the entire $D^{(k-1)}$ matrix (across ALL i,j pairs) is finalized before ANY $D^{(k)}$ computation begins, correctly respecting this global dependency; if i or j were outermost instead, you could end up using a MIX of already-updated $D^{(k)}$ values and not-yet-updated $D^{(k-1)}$ values within the same 'k iteration,' which (perhaps surprisingly) doesn't actually break Floyd-Warshall's specific correctness due to a subtle mathematical property (using $D^{(k)}$ values 'early' for the SAME k can't make things WORSE, since $d_{ik}^{(k)}=d_{ik}^{(k-1)}$ and $d_{kj}^{(k)}=d_{kj}^{(k-1)}$ always hold, a special self-referential property unique to Floyd-Warshall) — BUT this is a delicate, algorithm-specific exception, and the STANDARD, textbook-safe, always-correct implementation strictly requires k as the outer loop precisely to avoid needing to reason through this subtlety. (B) is a dangerous oversimplification for an exam context; while Floyd-Warshall happens to have this specific tolerance for SOME reordering due to its unique self-referential property, this is NOT true in general for arbitrary triply-nested DP algorithms, and asserting 'any of the six orderings works' without the specific justification is not a safe, general claim to test. (C) understates the correctness implications; while cache locality IS a real secondary consideration, the PRIMARY and non-negotiable reason for k being outermost is the fundamental dependency-respecting correctness argument. (D) is a false, overly broad generalization with no general basis; the correct loop nesting order depends entirely on the SPECIFIC dependency structure of whatever particular DP recurrence is being implemented, not a universal rule about which variable position is always 'i'.

Q96. Correct Answer: A — Floyd-Warshall: $\Theta(n^3)$, from the triply-nested loop with O(1) work per innermost iteration; running Bellman-Ford (O(VE)=O(n³) for dense graphs) from every one of the n vertices gives O(n·VE) = O(n&sup4;) for dense graphs, making Floyd-Warshall asymptotically BETTER for dense graphs specifically

As explicitly stated in CLRS, the Floyd-Warshall algorithm's triply-nested loop structure (over k, i, j, with O(1) work at line 6 per innermost iteration) directly gives $\Theta(n^3)$ time; comparing this to running Bellman-Ford (itself O(VE) time, which for a DENSE graph with $E=\Theta(V^2)=\Theta(n^2)$ edges becomes $O(n^3)$) from EACH of the n vertices as sources gives a TOTAL of $O(n)\times O(n^3)=O(n^4)$ for dense graphs — making Floyd-Warshall's SINGLE $\Theta(n^3)$ pass strictly more efficient than this alternative 'run single-source Bellman-Ford from everywhere' approach specifically for dense graphs, which is one of the key motivations CLRS gives for Floyd-Warshall's practical value (though for SPARSE graphs, running Dijkstra's, if weights permit, or a more carefully optimized approach from every vertex might actually be competitive with or better than Floyd-Warshall's fixed $\Theta(n^3)$, since Dijkstra's with a good heap implementation can be much faster than $O(n^3)$ for sparse graphs). (B) drastically understates Floyd-Warshall's actual complexity; $\Theta(n^2)$ would be WAY too fast for an ALL-PAIRS shortest path algorithm that needs to consider all n^2 pairs AND intermediate vertex possibilities. (C) incorrectly computes Floyd-Warshall's complexity as n^4 (that's the BELLMAN-FORD-FROM-EVERY-VERTEX alternative's complexity for dense graphs, not Floyd-Warshall's own complexity) and then draws the exact opposite (backwards) conclusion about which approach is better. (D) is false; the two approaches have DIFFERENT complexities as computed above (n^3 vs n^4 for dense graphs), directly contradicting the premise that they're 'identical in all cases.'

Q97. Correct Answer: A — The algorithm's correctness proof relies only on the existence of well-defined SHORTEST (simple) paths between vertex pairs, which negative EDGES alone don't prevent; but a negative-weight CYCLE would allow a 'path' to repeatedly loop through the cycle, decreasing total weight WITHOUT BOUND, making 'shortest path' UNDEFINED (no finite minimum exists) for any pair of vertices reachable via that cycle

As explicitly noted in the CLRS text, Floyd-Warshall (like the related approach in Section 23.1) permits negative-weight EDGES, but explicitly assumes NO negative-weight CYCLES exist — the fundamental reason is that the notion of a 'SHORTEST PATH' is only well-defined if there's a finite minimum achievable weight; with a negative-weight cycle reachable from i and able to reach j, you could construct paths that traverse the cycle an ARBITRARY number of times, each traversal further DECREASING the total path weight without any lower bound, making the INFIMUM of possible path weights negative infinity — there is NO actual shortest (finite, simple) path in this scenario, and the entire mathematical framework of the algorithm (defined in terms of computing finite shortest-path weights) breaks down. Individual negative EDGES, by contrast, don't cause this issue as long as they don't form an exploitable cycle, since any SIMPLE path (visiting each vertex at most once) is finite in length and has a well-defined finite total weight regardless of individual edge signs. (B) is factually incorrect and conflates Floyd-Warshall's requirements with a DIFFERENT algorithm's (Dijkstra's) requirements; Floyd-Warshall (unlike Dijkstra's) explicitly CAN handle negative EDGES correctly, that's one of its notable strengths as a DP-based (rather than greedy-based) approach. (C) is a fabricated and mathematically nonsensical condition; the PARITY (even/odd count) of negative edges within a cycle has no bearing on whether the CYCLE ITSELF has negative total weight, which is the actual relevant condition, and even a single sufficiently negative edge combined with other edges can create a negative-weight cycle regardless of edge count parity. (D) is false; the STANDARD Floyd-Warshall algorithm does NOT automatically detect or gracefully handle negative cycles by 'ignoring' them — if negative cycles are present, the algorithm's output becomes MEANINGLESS/incorrect (potentially the diagonal entries d_{ii} would go negative, which is a KNOWN post-hoc detection technique, but the algorithm doesn't 'ignore' the cycle during its normal execution).

Q98. Correct Answer: A — Check the DIAGONAL entries $d_{ii}^{(n)}$ for any i; if ANY diagonal entry is NEGATIVE ($d_{ii}^{(n)} < 0$), this indicates vertex i lies on a negative-weight cycle, since a legitimate shortest 'path' from a vertex to ITSELF (with no cycling) should have weight exactly 0, and a negative value can only arise if the algorithm found a way to 'return' to i via a net-negative-weight cycle

This is a well-known and elegant POST-HOC detection technique directly exploitable from Floyd-Warshall's own output structure: under NORMAL circumstances (no negative cycles), the shortest 'path' from any vertex i to ITSELF should have weight exactly 0 (the trivial empty path, doing nothing), so ALL diagonal entries $d_{ii}^{(n)}$ should equal 0 in a well-behaved graph; however, if a negative-weight cycle passing through vertex i exists, the algorithm (even though technically its underlying assumptions are violated) would still MECHANICALLY compute increasingly negative values by chaining through this exploitable cycle, resulting in $d_{ii}^{(n)}$ becoming NEGATIVE — so a simple O(n) post-processing scan of just the diagonal entries after running the standard $O(n^3)$ algorithm can definitively detect the PRESENCE of at least one negative cycle (though not necessarily pinpoint every affected vertex/cycle precisely without further work) with minimal additional computational cost. (B) is a fabricated and incorrect detection criterion; ZERO off-diagonal entries are entirely NORMAL and expected in many graphs (e.g., if there happen to be two DIFFERENT vertices with a genuinely zero-weight shortest path between them via some combination of positive and compensating negative edges, without ANY cycling issue at all) and have no reliable connection to negative cycle presence. (C) is false and unnecessarily pessimistic; this exact diagonal-checking technique IS a legitimate, efficient, well-known way to detect negative cycles using Floyd-Warshall's ALREADY-COMPUTED output, without needing an entirely separate algorithm (like Bellman-Ford's own negative-cycle detection mechanism, though that's an alternative valid approach for different scenarios). (D) is a fabricated and irrelevant criterion; matrix SYMMETRY relates to whether the underlying graph is UNDIRECTED (or has symmetric edge weights) versus DIRECTED with differing i-to-j and j-to-i weights — it has no direct, reliable connection to negative cycle detection.

Q99. Correct Answer: A — OR directly replaces MIN (both select the 'better'/broader of two options: minimum weight, or logical truth if EITHER path-existence option holds), and AND directly replaces + (both COMBINE two sub-results into a single joint result: summing weights along a path, or requiring BOTH sub-paths to exist for the combined path to exist)

As CLRS explicitly derives, the transitive closure recurrence is STRUCTURALLY ISOMORPHIC to the shortest-path recurrence, with a precise algebraic correspondence: MIN (which selects the smaller/better of two WEIGHT options in the shortest-path context) corresponds to OR (which selects TRUE if EITHER of two path-existence options holds, capturing 'is there SOME way to get from i to j, either without using k or by using k'); and + (which COMBINES two path segments' weights by SUMMING them) corresponds to AND (which combines two path-existence facts by requiring BOTH the i-to-k AND k-to-j sub-paths to exist for the combined i-to-k-to-j path to exist) — this beautiful correspondence (sometimes formalized via the mathematical concept of a SEMIRING, where different algebraic structures like (min,+) versus (OR,AND) can be plugged into the SAME underlying recursive algorithmic template) explains why the SAME $O(n^3)$ triply-nested-loop STRUCTURE directly solves both problems, just with different operators substituted in. (B) is false and misses this elegant, well-established, and pedagogically important structural connection that CLRS specifically highlights as a direct analogy. (C) describes the EXACT OPPOSITE (incorrectly swapped) correspondence from the genuinely correct one; MIN corresponds to OR (not AND), and + corresponds to AND (not MIN) — getting this backwards would produce a nonsensical, incorrect recurrence. (D) is factually wrong; the WHOLE POINT of CLRS's presentation of transitive closure immediately AFTER Floyd-Warshall is to demonstrate that the identical algorithmic template DOES directly apply, with only the algebraic operators changed, avoiding the need for a completely different approach.

Q100. Correct Answer: A — The direct boolean approach can be more space- and potentially time-efficient in practice, since boolean (single-bit) values typically require less STORAGE than full integer/numeric weights, and on many computer architectures, logical operations (AND/OR) on single bits execute FASTER than arithmetic operations (addition, comparison) on full integer words

As CLRS explicitly notes, while BOTH approaches achieve the same asymptotic $\Theta(n^3)$ time complexity, the DIRECT boolean-based transitive closure algorithm offers PRACTICAL (constant-factor) advantages: since it uses only BOOLEAN values (existence/non-existence of a path) rather than full numeric weights, its SPACE requirement is smaller by a factor corresponding to the size of a machine word (e.g., using 1 bit per entry instead of 32 or 64 bits for a full integer), and on many computer architectures, LOGICAL operations (bitwise AND/OR) on single-bit values can execute FASTER than ARITHMETIC operations (addition, comparison) on full-width integers — making the direct boolean approach a genuinely useful practical optimization even though it doesn't change the asymptotic complexity CLASS. (B) is false; while asymptotically IDENTICAL, there ARE meaningful practical (constant-factor) differences in space usage and potentially execution speed, as explicitly discussed in the textbook. (C) is factually incorrect; BOTH the boolean transitive-closure algorithm and the weight-based Floyd-Warshall-with-unit-weights approach share the SAME $\Theta(n^3)$ asymptotic time complexity — there's no achievable $O(n^2)$ speedup being described here. (D) is false; setting all edge weights to 1 and checking $d_{ij}<n$ IS a mathematically VALID (if less elegant/efficient in practice) way to compute transitive closure, as CLRS itself explicitly describes as 'one way' to solve this problem — it's not incorrect, just potentially less practically efficient than the direct boolean approach.

Q101. Correct Answer: A — $\pi_{ij}^{(k)}$ is the PREDECESSOR of vertex j on a shortest path from i, restricted to intermediates in {1,...,k}; if k IS used as an intermediate (i.e., $d_{ij}^{(k-1)} > d_{ik}^{(k-1)}+d_{kj}^{(k-1)}$), then $\pi_{ij}^{(k)} = \pi_{kj}^{(k-1)}$ (inheriting the predecessor from the k-to-j sub-path, since j's immediate predecessor on the FULL path is the same as its predecessor on just the final leg from k to j); otherwise, $\pi_{ij}^{(k)} = \pi_{ij}^{(k-1)}$ (predecessor unchanged, since the best path doesn't route through k)

This directly mirrors CLRS's precise formal definition (equation 23.8): $\pi_{ij}^{(k)}$ tracks the PREDECESSOR of j (the vertex immediately before j) on the current best-known shortest path from i to j, restricted to intermediates in {1,...,k}; when vertex k becomes a BETTER intermediate choice (i.e., the path THROUGH k, with cost $d_{ik}^{(k-1)}+d_{kj}^{(k-1)}$, is STRICTLY better than the previous best $d_{ij}^{(k-1)}$), the predecessor of j on this NEW, improved path is INHERITED from the k-to-j SUB-PATH specifically (since the overall path is now i→...→k→...→j, and j's IMMEDIATE predecessor is determined entirely by how the k-to-j portion reaches j, which is exactly $\pi_{kj}^{(k-1)}$); when k is NOT a better choice, the predecessor simply CARRIES OVER unchanged from the previous iteration ($\pi_{ij}^{(k-1)}$), since the best path itself hasn't changed. This precise predecessor-tracking mechanism allows full path RECONSTRUCTION (not just weight computation) by following predecessor pointers backward from j to i after the algorithm completes. (B) is factually wrong; $\pi_{ij}^{(k)}$ represents the predecessor of J specifically (the vertex whose predecessor we're tracking), NOT simply k itself — k is only relevant as an intermediate 'connector' vertex, and the true predecessor of j could be some vertex much closer to j along the k-to-j sub-path, not necessarily k directly (unless j happens to be immediately adjacent to k on that sub-path). (C) misunderstands the entire PURPOSE of the predecessor matrix; it explicitly represents VERTEX REFERENCES for path RECONSTRUCTION, not edge counts. (D) is false; the π recurrence is DIRECTLY and INTIMATELY tied to the D matrix's own computations (comparing $d_{ij}^{(k-1)}$ to $d_{ik}^{(k-1)}+d_{kj}^{(k-1)}$ is EXACTLY the same comparison driving the D matrix's own update decision), not an independent, unrelated computation.

Q102. Correct Answer: A — If i=j (trivial self-path) or there's NO direct edge ($w_{ij}=\infty$), there's no meaningful predecessor to record, so NIL correctly signals 'no path/predecessor exists at this base level'; if a DIRECT edge from i to j exists (i≠j and $w_{ij}<\infty$), then the immediate predecessor of j on this single-edge path is simply i itself (the path is just the one edge i→j, so j's predecessor is trivially i)

These base cases directly and sensibly reflect the SIMPLEST possible path scenarios at k=0 (no intermediates allowed): when i=j, there's no meaningful 'path' to speak of (a vertex trivially 'reaches' itself with zero edges), so there's no predecessor to record, hence NIL; when $w_{ij}=\infty$ (no direct edge exists between DISTINCT i and j), there's similarly NO valid path at all at this base level, so again NIL correctly indicates 'undefined/no path'; but when a genuine DIRECT edge exists (i≠j specifically, AND w_{ij} is finite), the path consists of EXACTLY that single edge i→j, and by the very definition of 'predecessor of j on this path,' the vertex immediately preceding j is i itself (there's nothing else on this minimal one-edge path) — hence $\pi_{ij}^{(0)}=i$ correctly captures this. (B) is factually wrong; the predecessor matrix IS meaningfully defined even at the base case k=0, as explicitly shown by the two distinct sub-cases given in the question itself — NIL is not universal across both sub-cases. (C) is incorrect; setting the value to i (the SOURCE vertex) is precisely correct for representing 'the predecessor of j is i' when the path is the single direct edge i→j — using j instead would nonsensically claim j is its own predecessor, which is logically wrong. (D) is a serious misunderstanding; these values are NOT arbitrary placeholders but carry precise, meaningful semantic content directly derived from the definition of what a 'predecessor' represents at the simplest possible path length (zero or one edges).

Q103. Correct Answer: A — No, $d_{23}^{(1)}$ remains −2 (unchanged from $d_{23}^{(0)}=-2$, the direct edge weight), because checking the alternative path 2→1→3 would require $d_{21}^{(0)}+d_{13}^{(0)}$, and $d_{21}^{(0)}=\infty$ (no direct edge from 2 to 1 exists in this graph), so this alternative path is not viable, leaving the direct edge as the best (and only) option

Let's carefully trace this: $d_{23}^{(0)}=w_{23}=-2$ (the given direct edge). Checking whether vertex 1 improves this at k=1: the recurrence requires comparing $d_{23}^{(0)}$ against $d_{21}^{(0)}+d_{13}^{(0)}$ (the potential path 2→1→3, using vertex 1 as the intermediate connector) — critically, $d_{21}^{(0)}$ represents the DIRECT edge weight from vertex 2 TO vertex 1 specifically (respecting DIRECTIONALITY, since this is presumably a DIRECTED graph, as is standard for Floyd-Warshall problems), and based on the given edges (only w_{12}, w_{23}, w_{31} are specified as finite; there's no mention of an edge FROM 2 TO 1), $d_{21}^{(0)}=\infty$, making the alternative path's cost $\infty+d_{13}^{(0)}=\infty$ (regardless of $d_{13}^{(0)}$'s specific value), which is certainly NOT better than the existing $d_{23}^{(0)}=-2$ — so $d_{23}^{(1)}$ correctly remains −2, UNCHANGED from the base case. (B) makes a critical DIRECTIONALITY error, incorrectly using w_{12} (the edge FROM 1 TO 2) instead of correctly checking for an edge FROM 2 TO 1 (which doesn't exist in this specific graph) — a very common student mistake when working with DIRECTED graphs, where edge direction matters critically and A→B does NOT imply B→A exists or has the same weight. (C) reflects a fundamental misunderstanding of the recurrence; there's no rule stating shortest paths 'reduce to 0' after passing through intermediates — path weights are determined by SUMMING actual edge weights along the path, not some arbitrary reset-to-zero convention. (D) is incorrect; the given information (the three specific edge weights, with the explicit statement that all OTHER entries are ∞) IS fully sufficient to definitively trace through this specific step of the algorithm.

Q104. Correct Answer: A — 3, unchanged from the direct edge $d_{12}^{(0)}=3$, since NO alternative path from 1 to 2 (via 3 as an intermediate, i.e., 1→3→2, requiring an edge FROM 3 TO 2, which is not given/is ∞) provides an improvement

Tracing carefully: $d_{12}^{(0)}=w_{12}=3$ (the given direct edge). At k=1 (using vertex 1 as intermediate): checking $d_{11}^{(0)}+d_{12}^{(0)}=0+3=3$, no improvement (this check is actually somewhat trivial/redundant when k=i, but doesn't cause incorrect results). At k=2: checking $d_{12}^{(1)}$ vs $d_{12}^{(1)}$... (using vertex 2 as an intermediate to improve i=1,j=2 doesn't make sense since j=2=k here, another trivial/non-improving case). At k=3 (using vertex 3 as intermediate): checking $d_{12}^{(2)}=3$ against $d_{13}^{(2)}+d_{32}^{(2)}$ — we'd need to know $d_{32}^{(2)}$ (edge/path FROM 3 TO 2), but the given edges are $w_{12}=3$, $w_{23}=-2$ (2 TO 3), and $w_{31}=2$ (3 TO 1) — there is NO specified edge FROM 3 TO 2 directly, and no INDIRECT path from 3 to 2 either (3's only outgoing edge goes to 1, and 1's only outgoing edge goes to 2, so 3→1→2 IS actually a valid path with weight $w_{31}+w_{12}=2+3=5$, meaning d_{32} is actually 5, not ∞) — but checking $d_{13}^{(2)}+d_{32}^{(2)}$: we need $d_{13}^{(2)}$ too (is there a path from 1 to 3? Only via 1→2→3, weight $3+(-2)=1$), so $d_{13}^{(2)}=1$, giving $d_{13}^{(2)}+d_{32}^{(2)}=1+5=6$, which is WORSE than the existing $d_{12}^{(2)}=3$, so NO improvement occurs, and the final $d_{12}^{(3)}$ remains 3. (B), (C), and (D) all represent various computational/conceptual errors (incorrect averaging, a false convergence-to-zero belief, and directly substituting an unrelated edge's weight) with no grounding in the actual recurrence-based computation traced above.

Q105. Correct Answer: A — Θ(n³) space for storing all n+1 separate n×n matrices; this can be reduced to Θ(n²) by OBSERVING that the recurrence for $d_{ij}^{(k)}$ only ever needs values from the IMMEDIATELY PRECEDING matrix $D^{(k-1)}$, so a SINGLE matrix can be reused/overwritten in place (dropping the superscripts entirely) across iterations, as explicitly discussed in a CLRS exercise (23.2-4)

As explicitly given in CLRS Exercise 23.2-4 (directly referenced in the source material), the STRAIGHTFORWARD reading of the recurrence (with explicit superscripts $d_{ij}^{(k)}$ for k=0,1,...,n) suggests needing Θ(n³) space to store ALL n+1 separate matrices — but the exercise specifically asks students to show that the simplified procedure FLOYD-WARSHALL', which 'simply DROPS all the superscripts' (i.e., uses a SINGLE matrix D that gets overwritten/updated IN PLACE across iterations, rather than maintaining n+1 distinct matrices), is STILL CORRECT, requiring only Θ(n²) space; this works because, crucially, when computing the update for d_{ij} at iteration k, even though you're technically supposed to use ONLY values from $D^{(k-1)}$, the specific structure of the recurrence means that using the 'new' (already-updated-this-iteration) values for d_{ik} and d_{kj} specifically doesn't cause incorrect results, since $d_{ik}^{(k)}=d_{ik}^{(k-1)}$ and $d_{kj}^{(k)}=d_{kj}^{(k-1)}$ ALWAYS hold (updating row/column k using vertex k itself as an intermediate never actually CHANGES those specific values, a special self-referential mathematical property unique to this exact recurrence structure) — making the space-efficient single-matrix implementation both CORRECT and the STANDARD, practically-used version of the algorithm. (B) is false; a NAIVE, literal implementation storing ALL matrices explicitly WOULD indeed require Θ(n³) space, and recognizing this reducibility to Θ(n²) is precisely the valuable insight this question tests, not an assumption that space is 'always' minimal. (C) is factually incorrect and directly contradicts the well-established, standard optimization explicitly discussed in CLRS itself (Exercise 23.2-4). (D) drastically understates the necessary space; Floyd-Warshall fundamentally needs to track PAIRWISE distances between ALL vertex pairs, requiring at least Θ(n²) space for this information, not merely a single running total.

Q106. Correct Answer: A — The algorithm may incorrectly compute LONGER (or entirely incorrect/infinite) shortest paths for various vertex pairs, since paths that legitimately should be able to 'pass through' a vertex back to a point that connects back to itself (weight 0 contribution) would instead be treated as requiring an INFINITE-cost detour, potentially causing valid, genuinely shorter paths to be missed or miscalculated during the min() comparisons throughout the algorithm's execution

This is a subtle but genuinely impactful correctness bug: the diagonal entries d_{ii} (representing the trivial 'distance from a vertex to itself') are used throughout the algorithm's min() comparisons whenever i=k or j=k in various sub-computations (e.g., when k happens to equal i or j during certain iterations, the recurrence implicitly references d_{ii} or d_{jj}-style entries as part of the broader computation) — incorrectly setting these to ∞ instead of the mathematically correct 0 could cause the algorithm to WRONGLY REJECT or fail to properly consider certain valid path combinations that legitimately rely on this zero-cost self-loop baseline, potentially leading to INCORRECT (too large, or even erroneously infinite) final shortest-path distances for various vertex pairs, since the min() operations throughout the algorithm would be comparing against artificially inflated (infinite) diagonal values instead of the correct zero baseline in relevant sub-computations. (B) is factually wrong; while the diagonal entries might seem 'trivial' at first glance, they DO play a genuine role in the broader matrix computation's correctness, particularly in edge cases and specific intermediate calculations throughout the n^3 triply-nested loop structure. (C) is incorrect; using ∞ as a value (assuming it's represented as, e.g., a very large sentinel number or a proper infinity representation in the specific programming language/environment) doesn't inherently cause a division-by-zero or crash-style error — Floyd-Warshall's core operations are addition, comparison, and min-selection, none of which inherently require division. (D) is false; this is fundamentally a CORRECTNESS bug affecting the computed VALUES in the distance matrix, not merely a space/memory-related issue — the bug doesn't change how much memory is used, it changes what VALUES end up stored there.

Q107. Correct Answer: A — Running Dijkstra's (with a Fibonacci heap) from every vertex gives $O(V^2 \log V + VE)$ total time; for a DENSE graph where E approaches V^2, this becomes $O(V^3)$, essentially MATCHING Floyd-Warshall's complexity (making Floyd-Warshall's simplicity and ability to handle negative edges an attractive, comparably-efficient choice), whereas for a SPARSE graph where E is much smaller (closer to V), the Dijkstra-from-every-vertex approach becomes substantially FASTER than Floyd-Warshall's FIXED n^3 bound, which doesn't improve/adapt based on sparsity

This reflects an important PRACTICAL/comparative complexity consideration frequently tested in courses covering all-pairs shortest paths: Dijkstra's algorithm (with an efficient Fibonacci-heap-based priority queue implementation) runs in $O(V \log V + E)$ per single-source execution; running it from ALL V vertices gives $O(V^2 \log V + VE)$ total — for a DENSE graph ($E \approx V^2$), this becomes $O(V^2 \log V + V^3) \approx O(V^3)$ (since the V^3 term dominates for large V), essentially MATCHING Floyd-Warshall's fixed $O(V^3)$ bound, meaning Floyd-Warshall's algorithmic SIMPLICITY (no complex heap data structure needed) and its ADDITIONAL capability of correctly handling NEGATIVE edge weights (which Dijkstra's fundamentally CANNOT do) make it an attractive, comparably-efficient choice for dense graphs specifically; but for a SPARSE graph (E much smaller, e.g., $E = \Theta(V)$), the Dijkstra-based approach's complexity DROPS substantially (closer to $O(V^2 \log V)$, much better than $O(V^3)$), making it the CLEARLY SUPERIOR choice for sparse graphs, SPECIFICALLY BECAUSE Floyd-Warshall's complexity remains FIXED at $\Theta(n^3)$ regardless of how sparse the actual graph is (it doesn't 'adapt' or improve for sparse inputs, unlike Dijkstra's-based approaches whose complexity DIRECTLY depends on E). (B) is false and directly contradicts the nuanced density-dependent tradeoff being tested; Floyd-Warshall is NOT universally faster — for sparse graphs specifically, Dijkstra-based approaches become substantially better. (C) is a fabricated and incorrect claim; running a single-source algorithm like Dijkstra's REPEATEDLY from every vertex as a source is a well-established, valid, and commonly-used technique for solving the ALL-PAIRS problem, explicitly discussed and compared against Floyd-Warshall in CLRS and virtually every algorithms course. (D) is factually incorrect; Floyd-Warshall's complexity is a FIXED $\Theta(n^3)$ REGARDLESS of the actual number of edges E (dense or sparse) — it does not decrease for dense graphs; rather, dense graphs are exactly where its FIXED complexity happens to be COMPETITIVE with alternatives, not where its own complexity improves.

Q108. Correct Answer: A — Floyd-Warshall exhibits BOTH optimal substructure (a shortest path's SUB-PATHS, when decomposed at the highest-numbered intermediate vertex k, are themselves provably shortest paths using only smaller-numbered intermediates, as directly proven via the CLRS optimal-substructure argument) AND overlapping subproblems (the SAME $d_{ik}^{(k-1)}$ and $d_{kj}^{(k-1)}$ values get REUSED across MANY different (i,j) pairs at each iteration of k, making memoization/tabulation via the D matrix genuinely valuable rather than redundant recomputation)

Floyd-Warshall genuinely and structurally satisfies BOTH of the two defining DP criteria discussed extensively in the DP chapter: OPTIMAL SUBSTRUCTURE is directly established by the careful case-analysis proof (whether k is or isn't an intermediate vertex of the optimal path), showing that optimal solutions to the sub-paths (i-to-k and k-to-j, restricted to smaller intermediate sets) must themselves be optimal, precisely mirroring the general DP optimal-substructure pattern; OVERLAPPING SUBPROBLEMS is evident from the algorithm's very structure, since for a SPECIFIC value of k, the SAME values $d_{ik}^{(k-1)}$ (for various j) and $d_{kj}^{(k-1)}$ (for various i) get referenced REPEATEDLY across the n^2 different (i,j) pairs being updated at that iteration, making it genuinely valuable to have these values PRECOMPUTED and stored in the D matrix (rather than naively recomputed from scratch for each pair), directly matching the 'overlapping subproblems, worth caching' hallmark of DP. (B) is false and represents a serious misunderstanding of algorithmic classification; the DP label is NOT arbitrary/historical — it's grounded in genuine, provable structural properties (optimal substructure + overlapping subproblems) that Floyd-Warshall demonstrably satisfies. (C) is factually incorrect; Floyd-Warshall's ENTIRE algorithmic approach (building up the D matrix incrementally via k=1,2,...,n, comparing MULTIPLE possible options via min() at each step) is fundamentally DIFFERENT from Dijkstra's GREEDY, irrevocable-choice-based approach (finalizing one vertex's distance at a time, never reconsidering) — conflating these two fundamentally different algorithmic PARADIGMS is a significant conceptual error. (D) is a nonsensical, fabricated classification criterion; MANY algorithms use matrices for entirely different reasons (e.g., representing adjacency information) without being DP algorithms, and the mere PRESENCE of a matrix data structure has no bearing on whether an algorithm exhibits the actual defining DP properties.

Q109. Correct Answer: A — The computed distances for such pairs become MEANINGLESS/INCORRECT (the true 'shortest path' is undefined, being negative infinity in principle, since the cycle could be traversed arbitrarily many times), though the algorithm will still MECHANICALLY produce SOME finite numeric output (not necessarily an explicit error), which is WHY the post-hoc diagonal-checking technique is a valuable, necessary safeguard when negative edges are present and cycle-freedom isn't otherwise guaranteed

As established earlier, when a negative-weight cycle exists and is reachable from/can reach a given vertex pair (i,j), the TRUE 'shortest path weight' between i and j is mathematically UNDEFINED (there's no finite minimum, since traversing the cycle repeatedly drives the weight toward negative infinity) — but Floyd-Warshall, as a MECHANICAL, finite-iteration algorithm (running for exactly a fixed n iterations of k, without any explicit awareness of or special-casing for negative cycles during its core execution), will STILL produce SOME specific finite numeric value in d_{ij} at the end (NOT an explicit error, infinity symbol, or crash) — this computed value is simply INCORRECT/MEANINGLESS relative to the true (undefined) shortest path, since the algorithm's underlying mathematical assumptions (no negative cycles) have been violated, even though it doesn't 'know' this internally and will happily proceed producing seemingly normal-looking output; this is PRECISELY why the earlier-discussed post-hoc diagonal-checking technique (looking for $d_{ii}^{(n)}<0$) serves as a valuable, NECESSARY safeguard specifically when negative edges are present and cycle-freedom cannot otherwise be guaranteed a priori. (B) is factually incorrect; the STANDARD Floyd-Warshall algorithm, as typically presented and implemented, does NOT include automatic negative-cycle detection/halting logic built into its core execution — it simply runs its fixed triply-nested loop structure regardless, which is precisely why the SEPARATE post-hoc check is needed as an additional safeguard. (C) is dangerously wrong and represents a fundamental misunderstanding of why the negative-cycle assumption matters in the first place; the computed values are NOT meaningful/correct when this assumption is violated, even though the algorithm doesn't inherently signal this issue during its normal execution. (D) is false; the negative cycle issue fundamentally corrupts the underlying DISTANCE values themselves (which is exactly what makes 'shortest path' undefined in the first place), and any predecessor-matrix issues would be a DOWNSTREAM consequence of this same fundamental problem, not an independent, isolated issue affecting only Π while leaving D reliable.

Q110. Correct Answer: A — This claim is FALSE; the outer k-loop iterations are FUNDAMENTALLY SEQUENTIAL (NOT independent), since computing $D^{(k)}$ REQUIRES the FULLY COMPLETED $D^{(k-1)}$ matrix as input — you cannot correctly compute results for k=5 before k=4 is finished, unlike the INNER (i,j) pairs at a FIXED k, which genuinely CAN be computed independently/in parallel with each other since they only depend on the (already complete) PREVIOUS iteration's matrix, not on each other

This question specifically and importantly distinguishes between the parallelizability of the OUTER (k) loop versus the INNER (i,j) loops, a genuinely important structural/practical distinction: the OUTER k-loop iterations are STRICTLY SEQUENTIAL because $D^{(k)}$ is DEFINED IN TERMS OF the FULLY COMPLETED $D^{(k-1)}$ matrix (every entry of $D^{(k)}$ potentially depends on ANY entry of $D^{(k-1)}$, via the $d_{ik}^{(k-1)}$ and $d_{kj}^{(k-1)}$ terms for VARYING i and j across the whole matrix) — you fundamentally CANNOT start computing k=5's matrix before k=4's matrix is ENTIRELY finished, since you don't know in advance which entries will be needed; HOWEVER, for a FIXED value of k, the INNER computation across DIFFERENT (i,j) pairs genuinely CAN be parallelized, since computing $d_{ij}^{(k)}$ for one specific (i,j) pair doesn't depend on the SIMULTANEOUSLY-being-computed $d_{i'j'}^{(k)}$ value for some OTHER (i',j') pair at the SAME k — each (i,j) pair's update at a fixed k only reads from the ALREADY-COMPLETE $D^{(k-1)}$ matrix, making these inner updates INDEPENDENT of each other and genuinely parallelizable across multiple processors/threads, which is a well-established, practically-used parallelization strategy for Floyd-Warshall in real high-performance computing implementations. (B) is factually incorrect and reflects a serious misunderstanding of the OUTER loop's fundamental sequential dependency structure, directly contradicting the reasoning given above. (C) is also false; while the OUTER loop cannot be parallelized (as correctly explained), the INNER (i,j) loops absolutely CAN be, and this partial parallelizability is a genuinely valuable, well-studied property of Floyd-Warshall in practice. (D) is factually wrong; parallelized implementations of Floyd-Warshall (exploiting exactly the inner-loop independence described above) are well-documented, extensively studied, and PRACTICALLY IMPLEMENTED in real high-performance and GPU-based computing contexts, not merely theoretical curiosities.

Q111. Correct Answer: A — Technically, running FULL Floyd-Warshall and then simply extracting ONE specific d_{ij} entry from the final matrix WOULD give the correct single-pair shortest-path answer, but this wastes an enormous amount of computation ($\Theta(n^3)$ total work to answer just ONE query) compared to a dedicated single-source algorithm like Dijkstra's (O(E+VlogV) with a good heap) or even Bellman-Ford (O(VE)), both of which are specifically DESIGNED for this narrower single-source (or even more targeted single-PAIR) query and thus far more EFFICIENT when the ALL-PAIRS information isn't actually needed

This tests understanding of an important PRACTICAL EFFICIENCY consideration (choosing the RIGHT algorithm for the RIGHT problem SCOPE): while it's certainly TECHNICALLY TRUE that running the complete Floyd-Warshall algorithm and then simply LOOKING UP one specific d_{ij} entry from the resulting matrix would give a mathematically CORRECT answer to a single-pair query, doing so involves computing the ENTIRE $\Theta(n^3)$-time, $\Theta(n^2)$-space ALL-PAIRS solution just to extract ONE piece of information from it — a substantial, unnecessary WASTE of computational resources when a dedicated, purpose-built algorithm like Dijkstra's (for non-negative weights, achieving O(E+VlogV) with an efficient priority queue) or Bellman-Ford (for graphs potentially containing negative edges, achieving O(VE)) could answer the SAME single-pair (or even single-SOURCE, to-all-destinations) query far more EFFICIENTLY, since these algorithms are specifically DESIGNED and OPTIMIZED for this narrower problem scope, without the overhead of computing distances for EVERY possible pair when only ONE (or a few) is actually needed. (B) is factually incorrect; there's no technical BARRIER preventing Floyd-Warshall from being used this way — it's simply INEFFICIENT, not IMPOSSIBLE, to do so. (C) is false and directly contradicts the well-established EFFICIENCY comparison; Floyd-Warshall is specifically LESS efficient for THIS narrower use case precisely because it computes far MORE information than actually needed for a single query. (D) is a nonsensical, false claim; single-pair shortest path problems are extremely WELL-STUDIED and have MULTIPLE well-established, efficient algorithmic solutions (Dijkstra's, Bellman-Ford, BFS for unweighted graphs, etc.), directly contradicting any claim that they're 'unsolvable.'

Q112. Correct Answer: A — No, the standard recurrence does NOT directly solve this variant; the standard $d_{ij}^{(k)}$ tracks shortest paths based on which VERTICES are allowed as intermediates, NOT based on the NUMBER OF EDGES used, so a completely DIFFERENT DP formulation (typically tracking BOTH the destination AND the number of edges used so far as separate DP state dimensions, e.g., dp[edges][i][j]) would be needed to correctly enforce an edge-COUNT constraint

This question tests a subtle but important conceptual distinction: the STANDARD Floyd-Warshall recurrence's parameter k controls WHICH VERTICES are permitted as INTERMEDIATE vertices (progressively allowing MORE vertices as potential 'connectors' as k increases from 0 to n) — this is FUNDAMENTALLY DIFFERENT from constraining the NUMBER OF EDGES used in the path itself; a shortest path using intermediates restricted to {1,...,k} could still use ANY NUMBER of edges (potentially using ALL k+1 available vertices in a long chain, or just a single direct edge, depending on what's optimal) — there's NO direct connection between the k parameter (controlling vertex AVAILABILITY) and an edge-COUNT limit; to correctly solve the edge-count-limited variant, you'd need an ENTIRELY DIFFERENT DP formulation with a state space that EXPLICITLY tracks the number of edges used (e.g., a 3D DP table dp[m][i][j] = shortest path from i to j using AT MOST m edges, with a recurrence like dp[m][i][j]=min(dp[m−1][i][j], min over intermediate vertex v of (dp[m−1][i][v]+w_{vj})), which is a genuinely DIFFERENT algorithmic approach, closely related to certain formulations used in some versions of the Bellman-Ford algorithm's own iterative structure). (B) is factually incorrect; there's no such automatic correspondence between k (intermediate-vertex availability) and an edge-count limit — conflating these two entirely different DP state dimensions is a serious conceptual error. (C) is false; these are GENUINELY DIFFERENT problems (unrestricted-vertex shortest path vs. edge-count-limited shortest path) that can have DIFFERENT optimal answers for the SAME graph, depending on the specific value of K relative to the graph's structure. (D) is incorrect; simply STOPPING the k-loop early after K iterations does NOT correctly enforce an edge-count limit — it instead limits WHICH VERTICES may serve as intermediates (only vertices numbered 1 through K), which is a COMPLETELY DIFFERENT constraint from limiting the TOTAL NUMBER OF EDGES in the resulting path, and these two different restrictions generally produce DIFFERENT (not equivalent) results.

Q113. Correct Answer: A — If BOTH $d_{ik}^{(k-1)}$ and $d_{kj}^{(k-1)}$ happen to be (or are close to) the large sentinel value simultaneously, their SUM can OVERFLOW the integer type's maximum representable value, WRAPPING AROUND to an unexpectedly SMALL (or even NEGATIVE) number, which could then be INCORRECTLY selected as the 'minimum' in the min() comparison, silently CORRUPTING the algorithm's output with a completely bogus, artificially-small distance value

This is a genuinely important, PRACTICAL implementation bug/consideration frequently tested regarding Floyd-Warshall (and shortest-path algorithms in general): when representing 'no edge/no path' using a large FINITE sentinel value (a common practical necessity in languages using fixed-width integer types, since a TRUE mathematical infinity isn't directly representable), you must be CAREFUL that ARITHMETIC operations (specifically, the ADDITION $d_{ik}^{(k-1)}+d_{kj}^{(k-1)}$ in the recurrence) don't cause INTEGER OVERFLOW — if BOTH terms happen to be large sentinel values (representing 'currently unreachable' in BOTH sub-paths), their SUM could EXCEED the maximum representable integer value for the given data type, causing UNDEFINED BEHAVIOR or, in many common overflow-handling conventions, WRAPPING AROUND to a SMALL or even NEGATIVE number — which would then be INCORRECTLY interpreted as a 'very good' (small) distance by the subsequent min() comparison, potentially causing the algorithm to select this BOGUS, artificially-small value as if it were a genuine, valid shorter path, thereby SILENTLY CORRUPTING the algorithm's final output in a way that might not be immediately obvious without careful testing/debugging; the standard mitigation is to explicitly CHECK if either operand equals the sentinel infinity value BEFORE attempting the addition, and skip/avoid the addition entirely in that case (treating the sum as still 'infinity' rather than actually computing a numeric sum that risks overflow). (B) is factually incorrect; this scenario (both sub-path distances being 'infinity,' i.e., currently unreachable) is a COMMON, EXPECTED occurrence throughout the algorithm's early iterations, especially for SPARSE graphs or before enough intermediate vertices have been processed to establish connectivity between certain pairs. (C) is dangerously wrong and represents a failure to recognize a well-known, PRACTICALLY IMPORTANT implementation pitfall that real-world Floyd-Warshall implementations must explicitly guard against. (D) is incorrect; an overflow-corrupted DISTANCE value would directly and primarily corrupt the DISTANCE matrix D itself (since the overflow occurs during the distance CALCULATION), with any predecessor-matrix issues being, at most, a SECONDARY, downstream consequence of this same underlying distance-corruption problem, not an issue isolated to Π alone.

Q114. Correct Answer: A — Computing $D^{(n)}$ can be conceptually framed as repeatedly 'multiplying' the weight matrix W by itself using MIN in place of standard addition's role and PLUS (+) in place of standard multiplication's role (i.e., $(A \otimes B)_{ij} = \min_k (A_{ik} + B_{kj})$, directly analogous to standard matrix multiplication's $(AB)_{ij} = \sum_k (A_{ik} \times B_{kj})$); this reframing connects all-pairs shortest paths to FAST MATRIX MULTIPLICATION techniques, offering an ALTERNATIVE (though not identical in practical implementation) theoretical algorithmic approach distinct from Floyd-Warshall's specific triply-nested DP structure

This is a genuinely elegant and mathematically rich alternative theoretical PERSPECTIVE on the all-pairs shortest path problem, connecting it to abstract algebra's SEMIRING concept: by defining a special 'multiplication-like' operation ⊗ where $(A \otimes B)_{ij} = \min_k (A_{ik} + B_{kj})$ (replacing STANDARD matrix multiplication's usual SUM-of-PRODUCTS with a MIN-of-SUMS structure), repeatedly applying this operation to the weight matrix W (specifically, computing W, then W⊗W, then (W⊗W)⊗W, and so on) EVENTUALLY converges to the all-pairs shortest-path distance matrix once enough 'multiplications' have been performed (specifically, after $\lceil \log_2 n \rceil$ repeated SQUARINGS, i.e., W→W⊗W→(W⊗W)⊗(W⊗W)→..., since each squaring DOUBLES the maximum PATH LENGTH being correctly accounted for) — this REPEATED-SQUARING approach can achieve $O(n^3 \log n)$ time using NAIVE min-plus matrix multiplication (SLIGHTLY worse than Floyd-Warshall's clean $O(n^3)$), but theoretically opens the door to potentially FASTER algorithms if efficient min-plus matrix multiplication techniques (analogous to Strassen's algorithm for STANDARD matrix multiplication) can be developed, an ACTIVE area of ongoing algorithmic research; this connection is a valuable, sophisticated theoretical framing distinct from (though related in spirit to) Floyd-Warshall's own specific triply-nested-loop DP structure. (B) is false and dismisses a genuinely well-established, mathematically rich connection actively studied and discussed in advanced algorithms coursework and research literature. (C) is incorrect; Floyd-Warshall does NOT use STANDARD (+,×) matrix multiplication operations at all — it's the SPECIAL (min,+) SEMIRING reinterpretation that provides this alternative connection, not literal standard arithmetic matrix multiplication. (D) is a fabricated and false restriction; the (min,+) semiring framing works PERFECTLY WELL with NEGATIVE edge weights present (as long as no negative CYCLES exist, mirroring Floyd-Warshall's own stated requirement) — there's no inherent limitation to strictly positive weights in this theoretical framework.

Q115. Correct Answer: A — The algorithm will FORCE every single d[i][j] entry to be OVERWRITTEN by the (potentially much WORSE) path THROUGH vertex k, even when the EXISTING, previously-computed value (representing a path NOT using k) was ALREADY better/shorter; this systematically DISCARDS potentially-optimal existing solutions in favor of unconditionally routing through k, generally producing INCORRECT (too large, or nonsensical if involving unreachable/infinite sub-paths) final distances

This is a critical, very commonly tested implementation bug: OMITTING the min() comparison and UNCONDITIONALLY assigning d[i][j]=d[i][k]+d[k][j] means EVERY (i,j) pair gets FORCIBLY updated to route THROUGH vertex k, REGARDLESS of whether this is actually BENEFICIAL — if the EXISTING value of d[i][j] (representing the best path found so far, NOT necessarily using k) was ALREADY better (shorter) than the alternative path through k, this bug WRONGLY DISCARDS that better existing solution and REPLACES it with the (now KNOWN to be worse, by assumption) through-k alternative; additionally, if d[i][k] or d[k][j] happen to be INFINITY (representing 'currently unreachable' via the vertices considered SO FAR), this unconditional assignment could even PROPAGATE bogus infinite (or overflow-corrupted, per the earlier discussed overflow issue) values into d[i][j], further corrupting the computation; the CORRECT algorithm's min() comparison is NOT optional decorative syntax — it's the CENTRAL mechanism by which the algorithm correctly chooses between the 'don't use k' option (keeping the existing d[i][j]) and the 'DO use k' option (the new i-k-j path), and removing it fundamentally breaks the algorithm's core correctness guarantee. (B) is factually WRONG; there's no mathematical guarantee that d[i][k]+d[k][j] is ALWAYS the minimum/best option — that's PRECISELY the comparison the min() function is designed to correctly evaluate, and OFTEN the EXISTING d[i][j] (not routing through k) IS the better choice. (C) is incorrect; this bug doesn't create any LOOPING behavior (infinite or otherwise) — it's a straightforward (though INCORRECT) computational ASSIGNMENT that completes normally, just produces WRONG final results. (D) is false; this is UNAMBIGUOUSLY a CORRECTNESS bug (producing WRONG final shortest-path values), not merely a PERFORMANCE issue — the algorithm would still run in the same asymptotic TIME ($\Theta(n^3)$), just PRODUCE INCORRECT OUTPUT.